Master IMA – RDFIA 2026-2027
Reconnaissance des formes pour l'analyse et l'interprétation d'images
Neural Networks and Deep Learning

Clément Rambour

clement.rambour@isir.upmc.fr

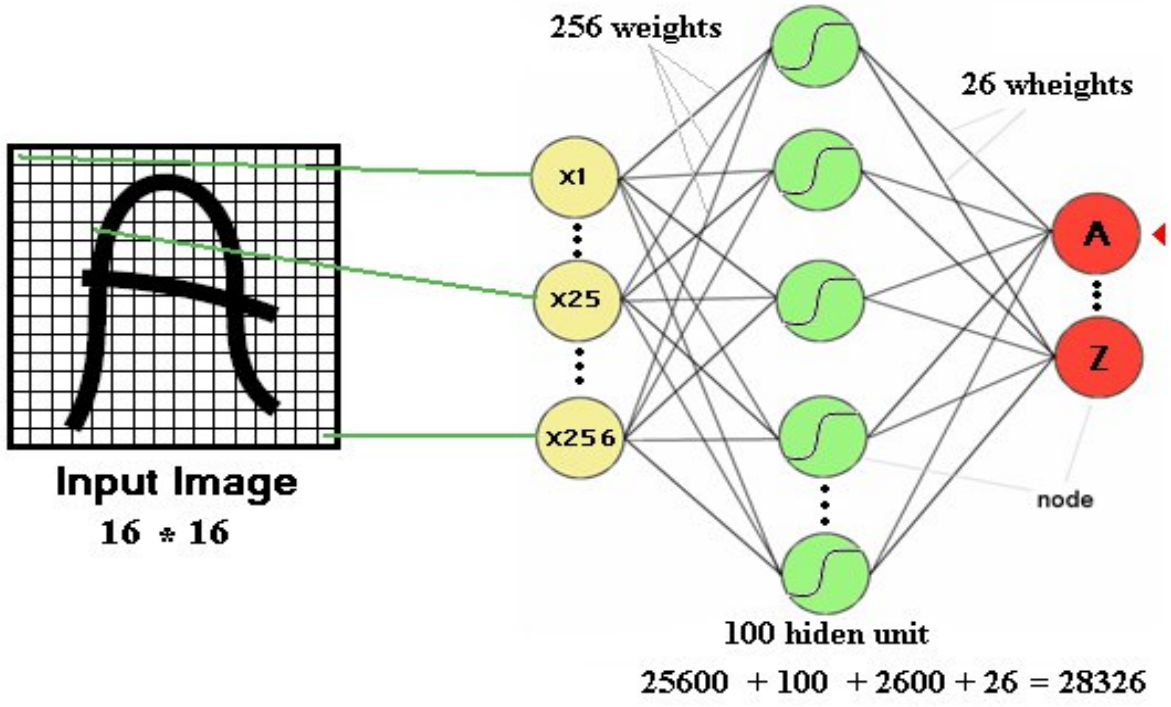
Sorbonne Université
ISIR - équipe MLIA

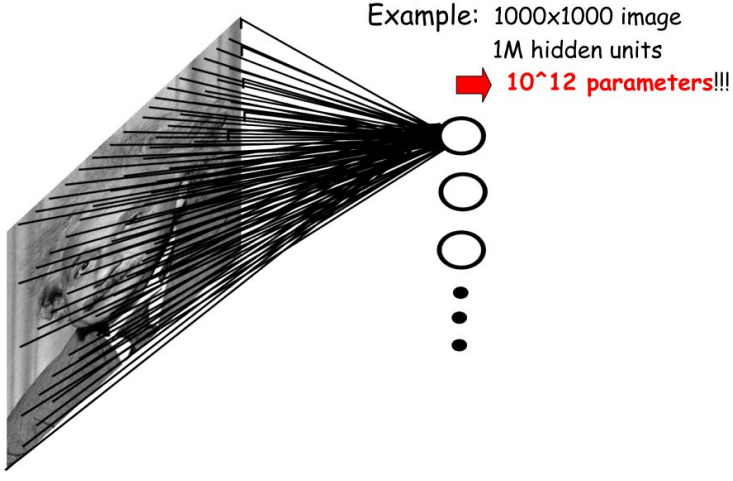
Année universitaire 2026-2027

Outline

1. Convolutional Neural Networks
2. Modern Deep Architectures
3. Implementation
4. Transfer Learning

MLP example: brute force connection





First Pb: Scalability

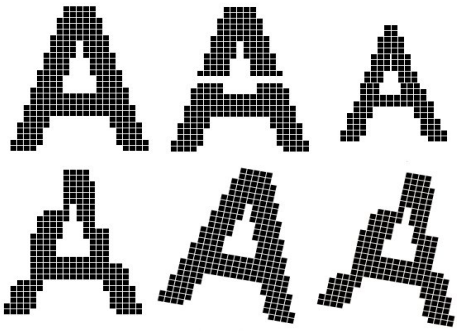
Large images => extremely large number of trainable parameters

MLP example: brute force connection

2d Pb: *Stability* of the representation

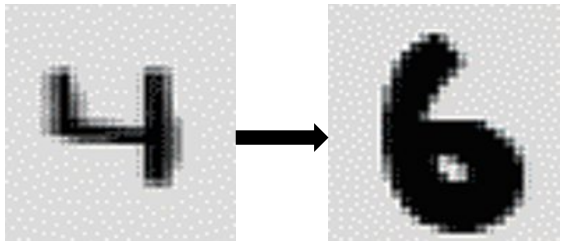
Expectation:

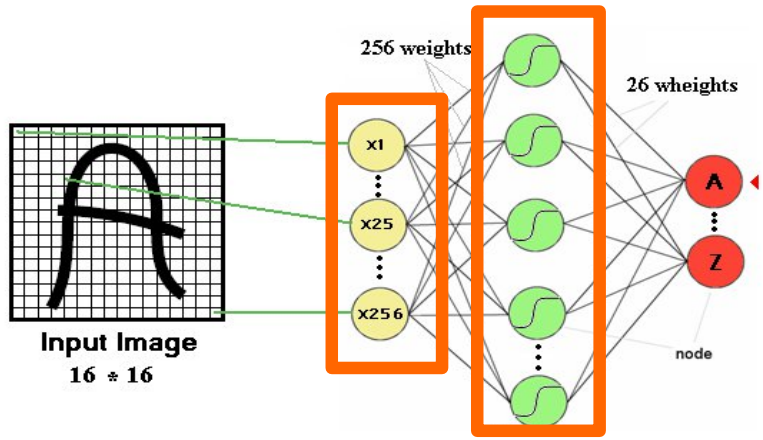
- *Small deformation in the input space*
  => similar representations



- *Large (or unexpected) transfo in the input*
  => very dissimilar representations

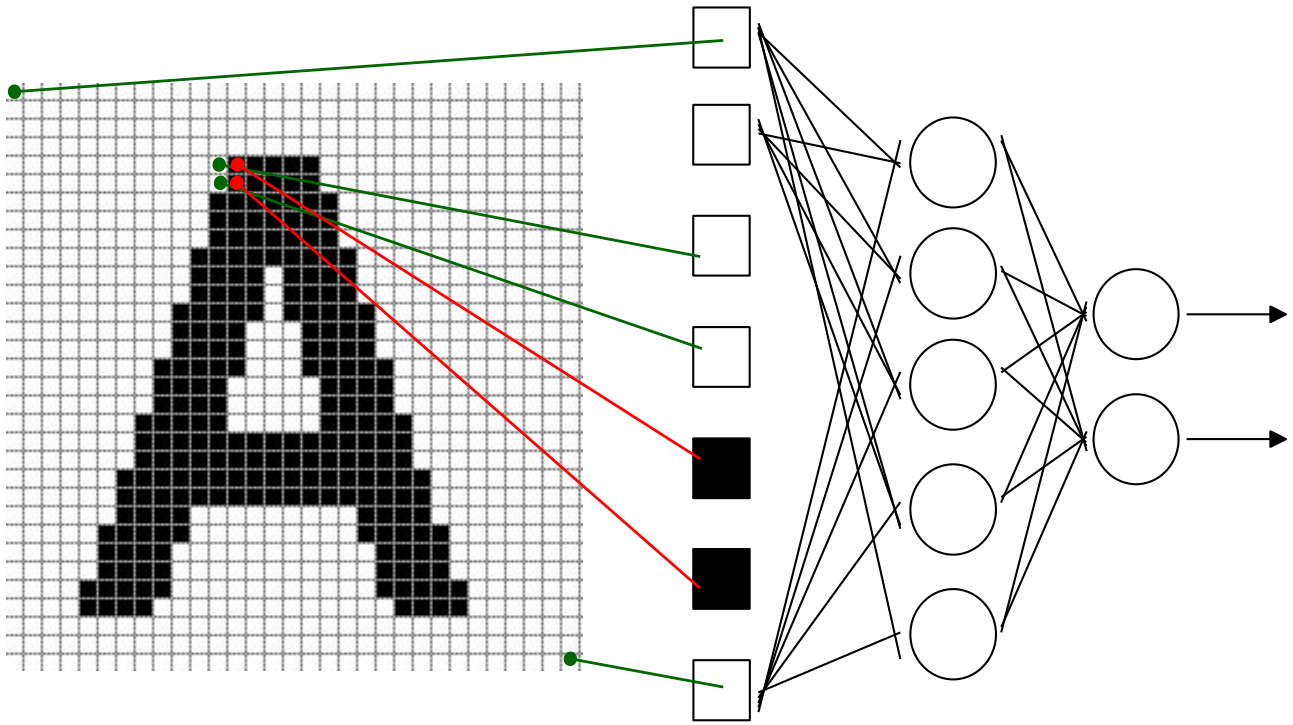
Representations:





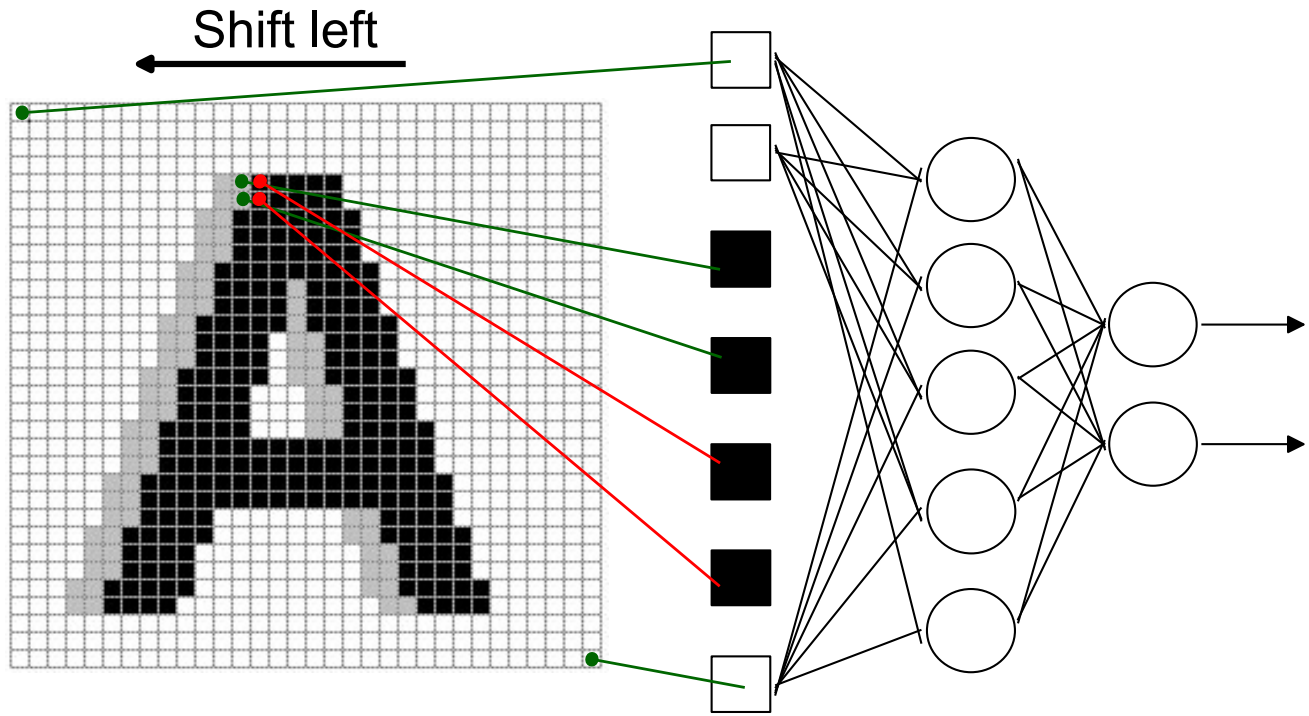
MLP example: brute force connection

Stability: Invariance/Robustness to (local) shifting, scaling, and other forms of (small) distortions?

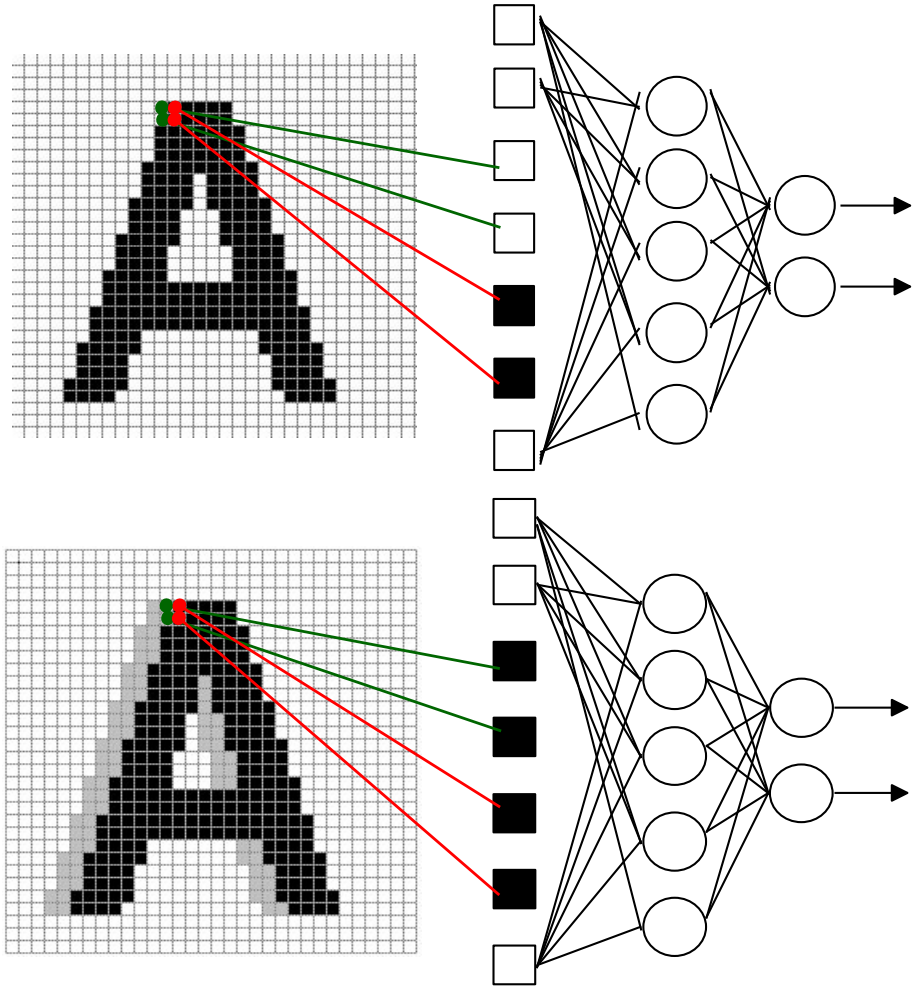


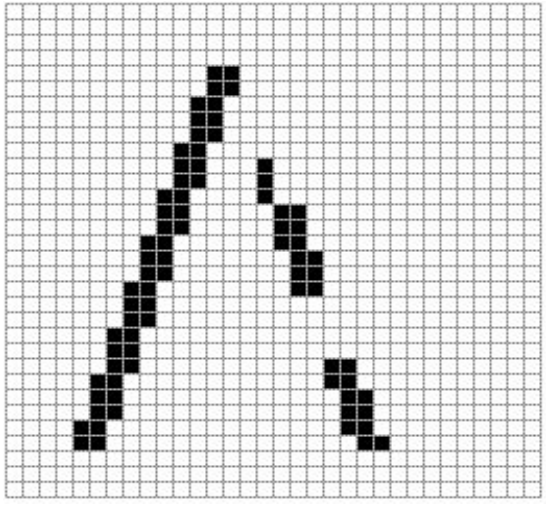
MLP example: brute force connection

Little or no invariance to shifting, scaling, and other forms of distortion



MLP example: brute force connection





154 input change
from 2 shift left
77 : black to white
77 : white to black

@LeCun

MLP example: brute force connection

Scaling and other forms of distortions => same pb



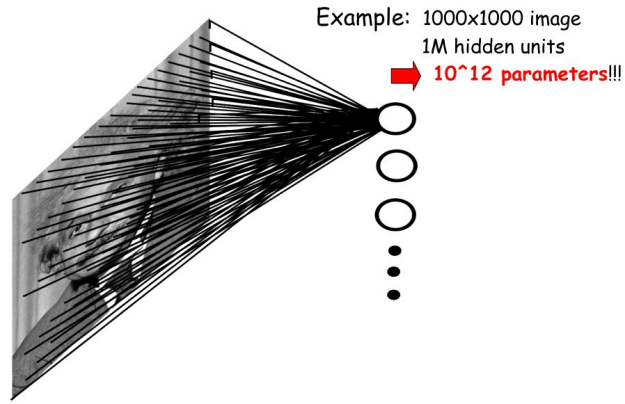
Conclusion of MLP on raw data

Brute force connection of images as input of MLP NOT a good idea

- No Invariance/Robustness of the representation because topology of the input data completely ignored
- Nb of weights grows largely with the size of the input image

How keep spatial topology?

How to limit the weight number?



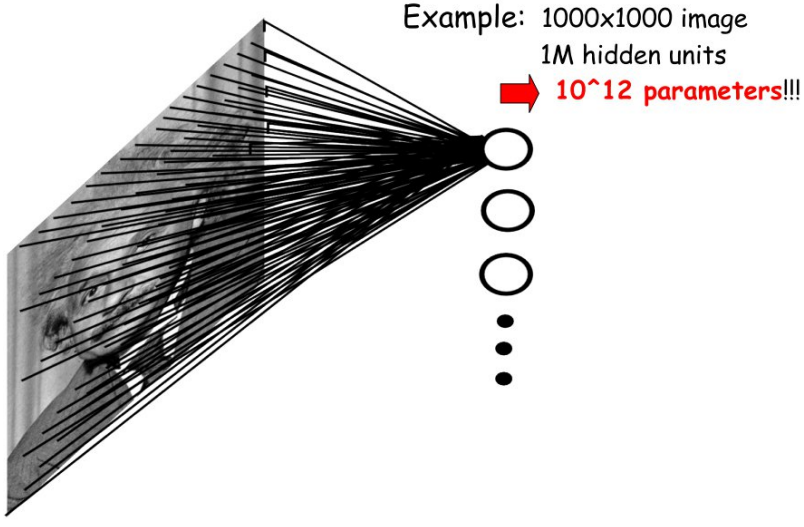
Outline
Convolutional Nets for visual classification

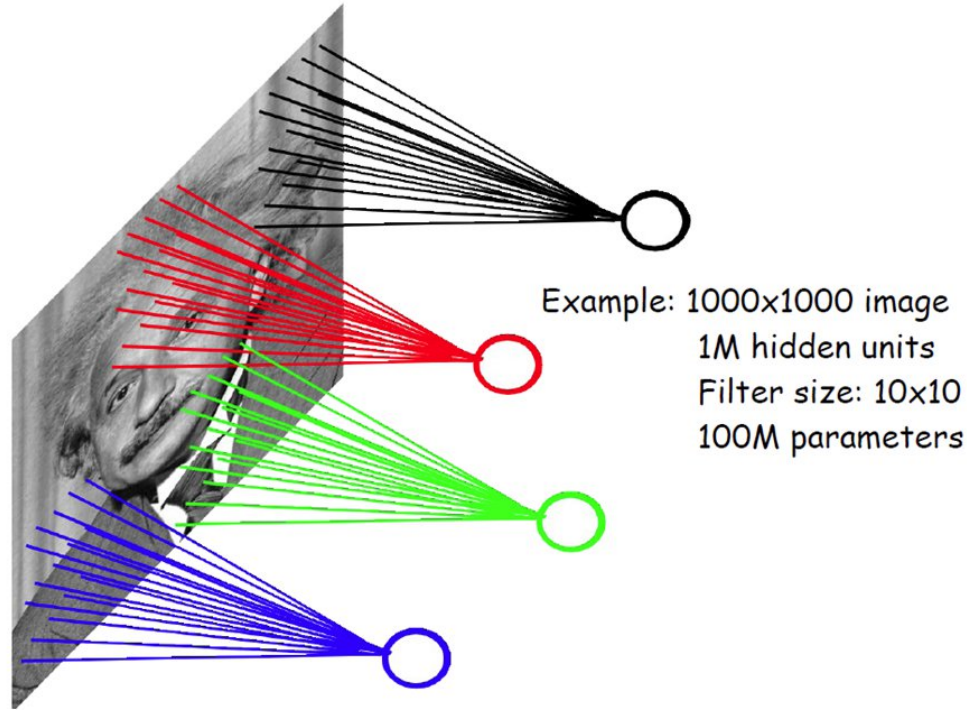
1. Recap MLP
2. **Convolutional Neural Networks**

How to limit the weight numbers?

1/ Locally connected neural networks

- **Sparse connectivity**: a hidden unit is only connected to a local patch (weights connected to the patch are called filter or kernel)
- Inspired by biological systems, where a cell is sensitive to a small sub-region of the input space, called a receptive field. Many cells are tiled to cover the entire visual field

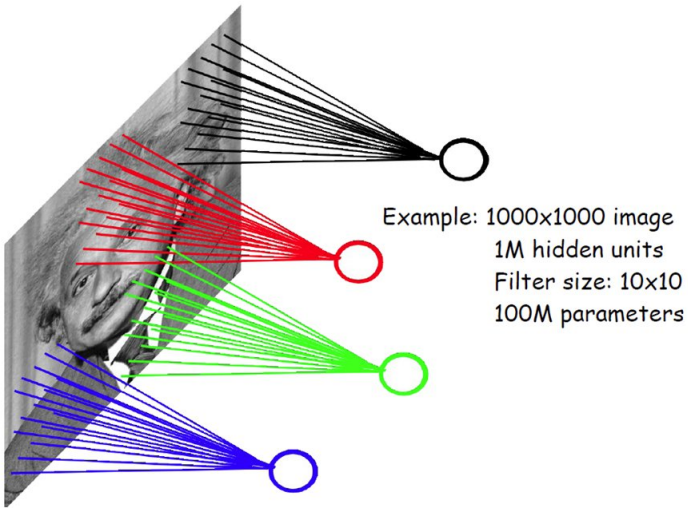


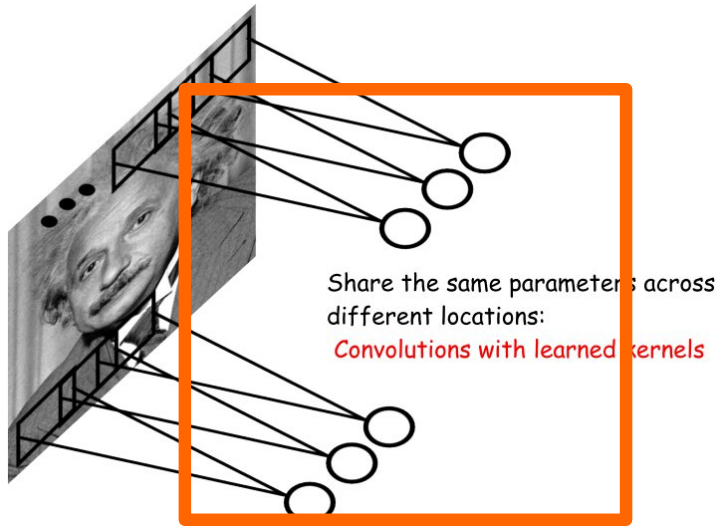


How to limit the weight numbers?

2/ Shared Weights

- Hidden nodes at different locations share the same weights
    - greatly reduces the number of parameters to learn
- Keep spatial information in a 2D feature map (hidden layer map)





- Computing responses at hidden nodes equivalent to convoluting input image with a linear filter (learned)
- A learned filter as a feature detector

Recap (1D/2D) convolution

1D discrete convolution of input signal x[n], with filter impulse response h[n], and output y[n]:

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$

2D discrete convolution of input signal x[m,n], with filter impulse response h[m,n] (*kernel*), and output y[m,n]:

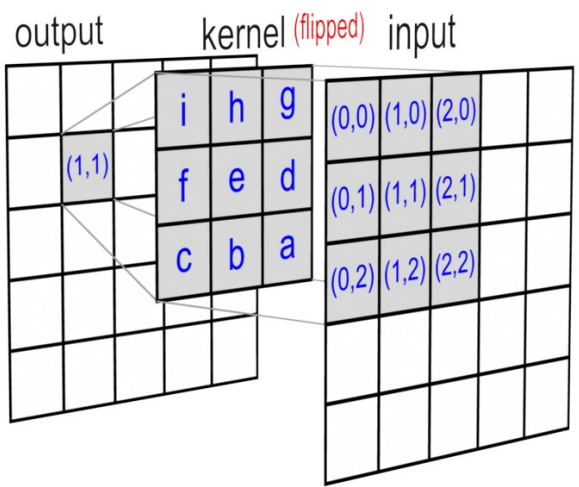
$$y[m,n] = x[m,n] * h[m,n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i,j] \cdot h[m-i, n-j]$$

Example with impulse response (kernel) 3x3, and it's values are a, b, c, d,... : (0,0) located in the center of the kernel

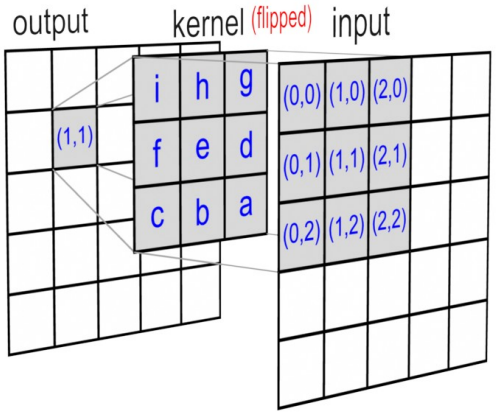
m / n	-1	0	1
-1	a	b	c
0	d	e	f
1	g	h	i

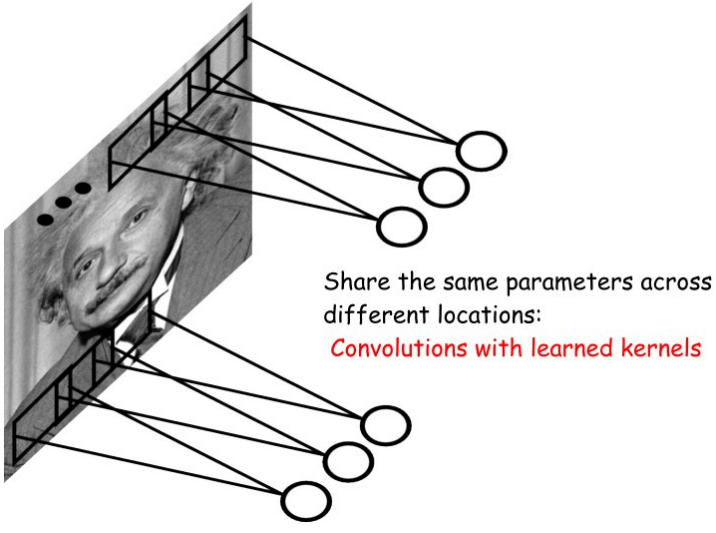
$$y[1,1] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i,j] \cdot h[1-i, 1-j]$$

$$\begin{aligned} = \quad & x[0,0] \cdot h[1,1] \quad + x[1,0] \cdot h[0,1] \quad + x[2,0] \cdot h[-1,1] \\ & + x[0,1] \cdot h[1,0] \quad + x[1,1] \cdot h[0,0] \quad + x[2,1] \cdot h[-1,0] \\ & + x[0,2] \cdot h[1,-1] + x[1,2] \cdot h[0,-1] + x[2,2] \cdot h[-1,-1] \end{aligned}$$



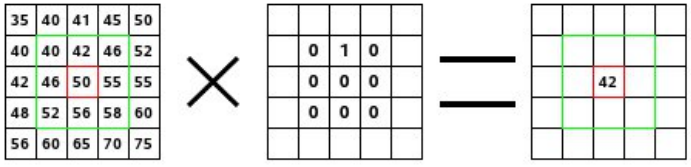
Ex. of convolution operator

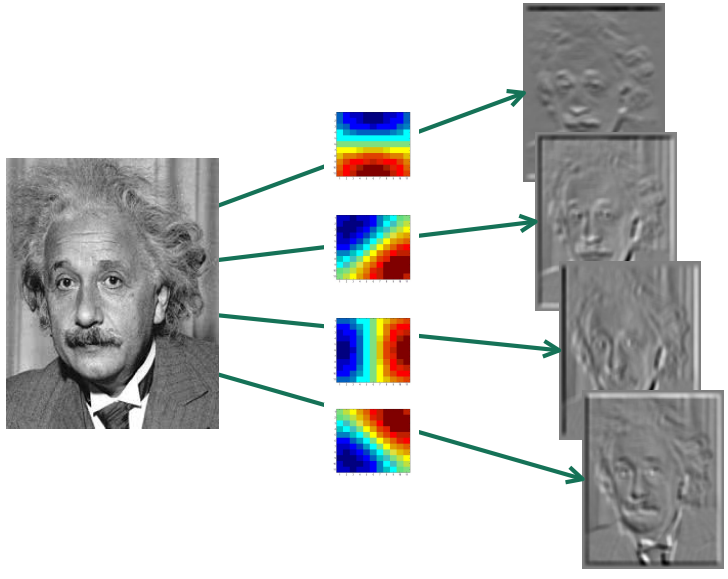




Convolution

2D





Convolution filters

unit impulse $\delta[n] = \mathbb{1}[n = 0]$

every signal x expressed as

$$x[n] = \sum_k x[k]\delta[n-k] = \sum_k x[k]s_k(\delta)[n]$$

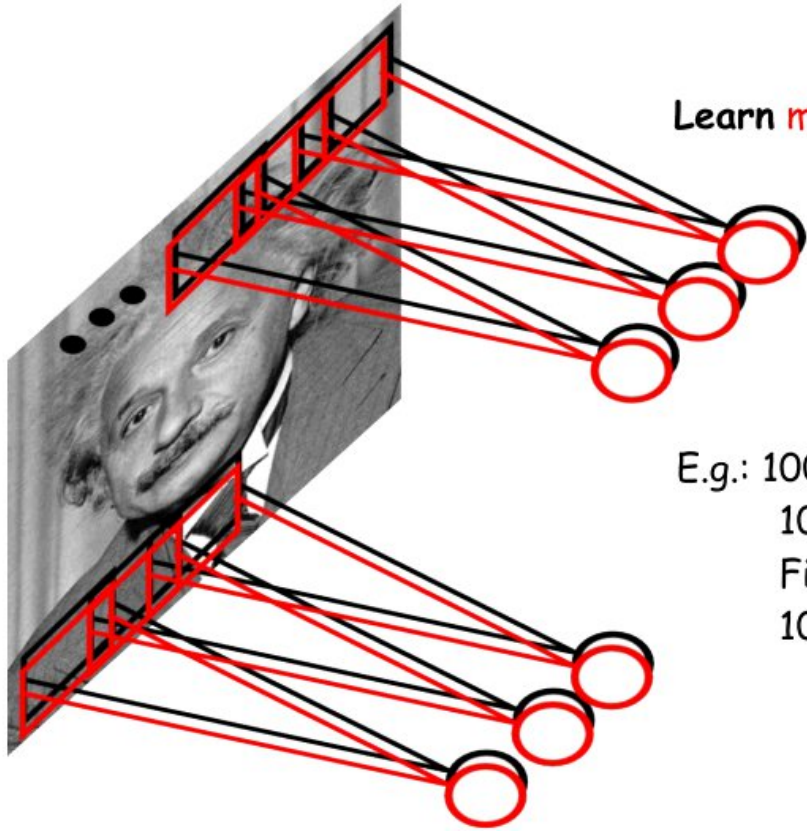
if f is LTI with impulse response $h = f(\delta)$, then $f(x) = x * h$:

$$f(x)[n] = f\left(\sum_k x[k]s_k(\delta)\right)[n] = \sum_k x[k]s_k(f(\delta))[n]$$

$$= \sum_k x[k]h[n-k] := (x * h)[n]$$

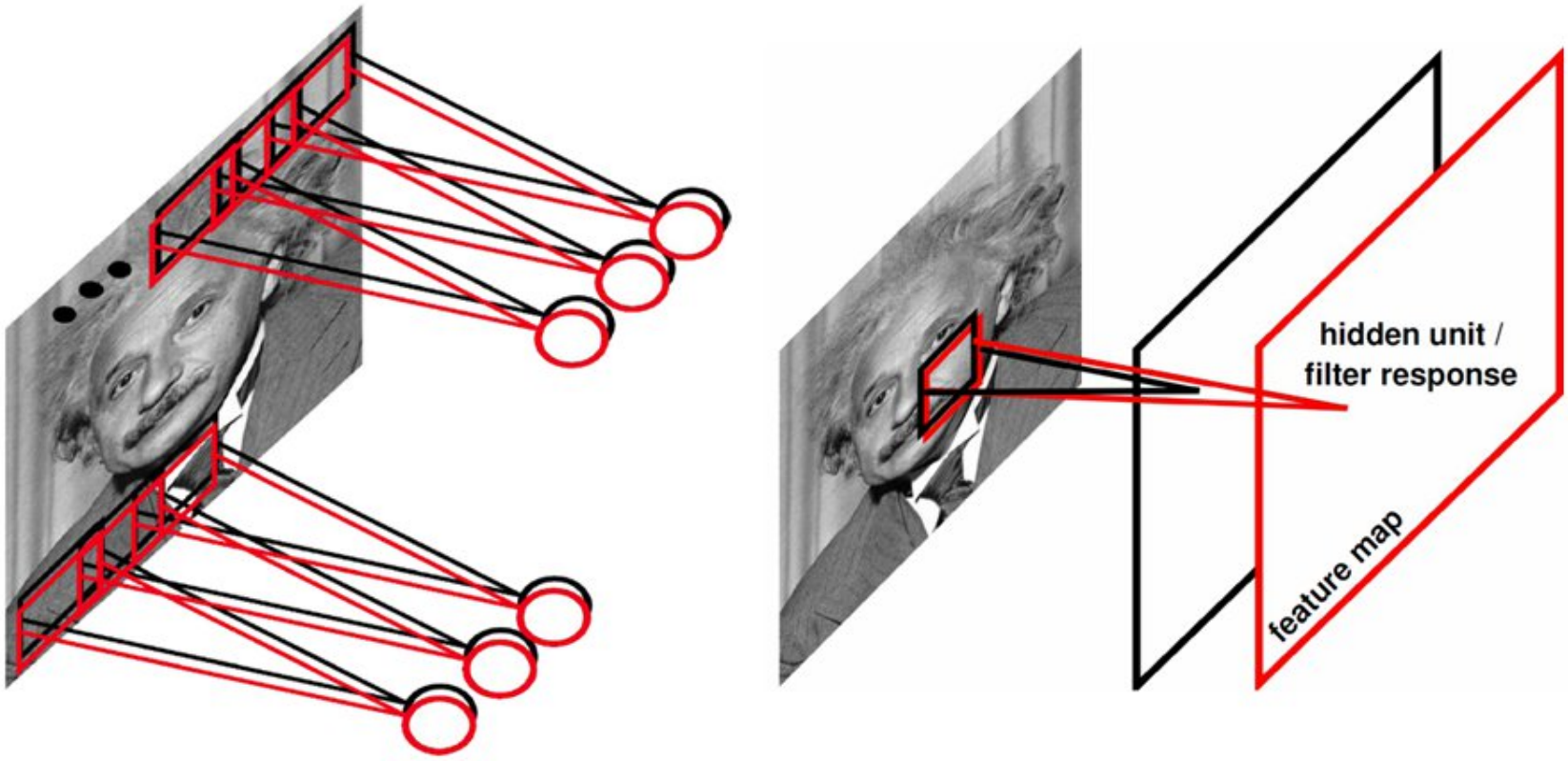
From one to many filters

1 filter => 1 feature map (corresponding to 1 visual pattern)
To detect spatial distributions of multiple visual patterns: Multiple filters
M filters => M feature maps! Get richer description



From one to many filters

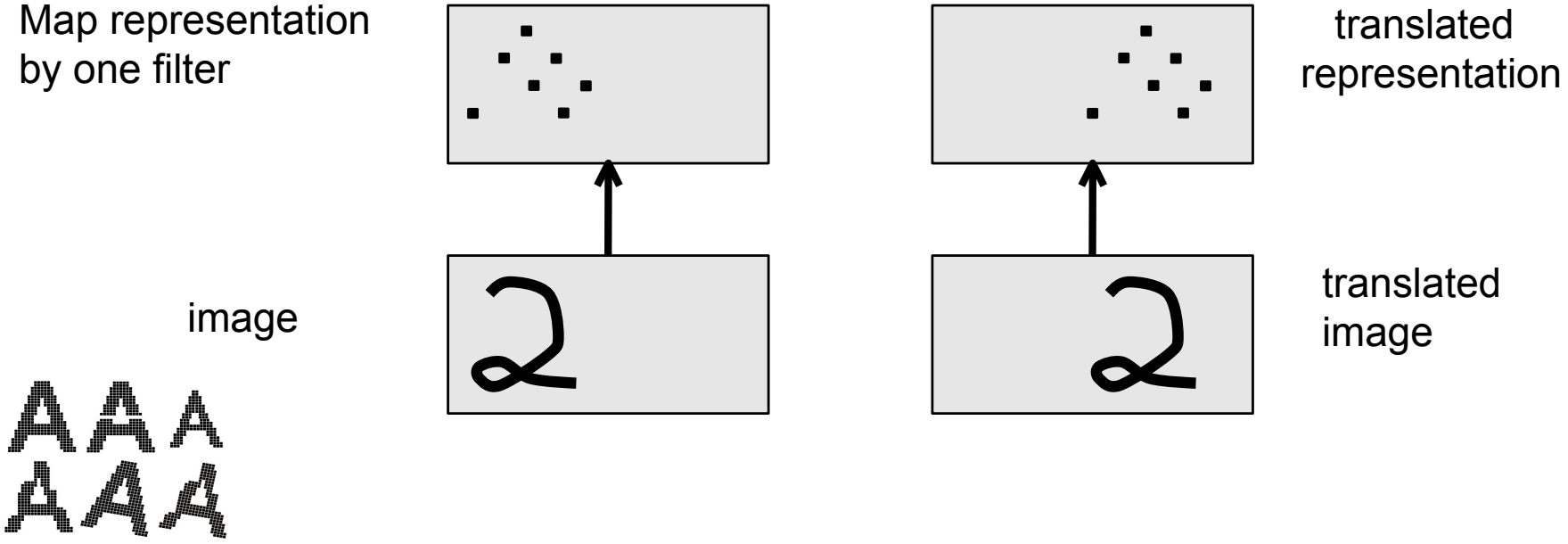
M filters => M feature maps



Rq: not many weights but many neurons! => memory issues will appear

What does replicating the feature detectors achieve?

- Equivariant activities (Hinton Ex): Replicated features do not make the neural activities invariant to translation. The activities are equivariant.

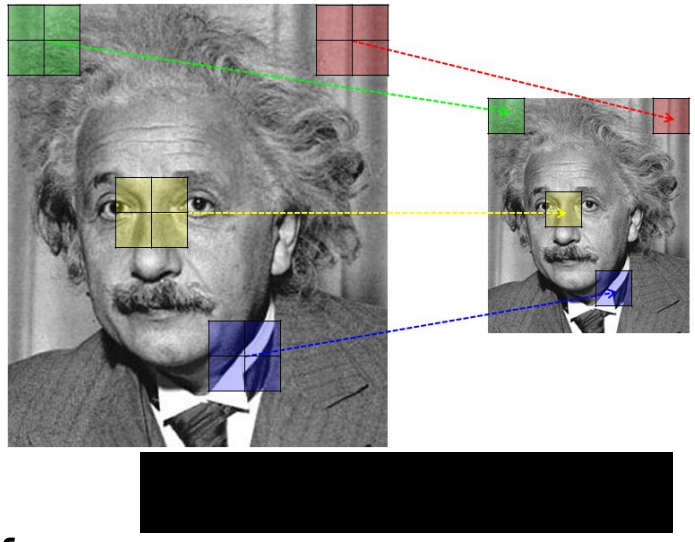


How to get invariance to 2D spatial transformation of the input?

Getting (more) local Invariance

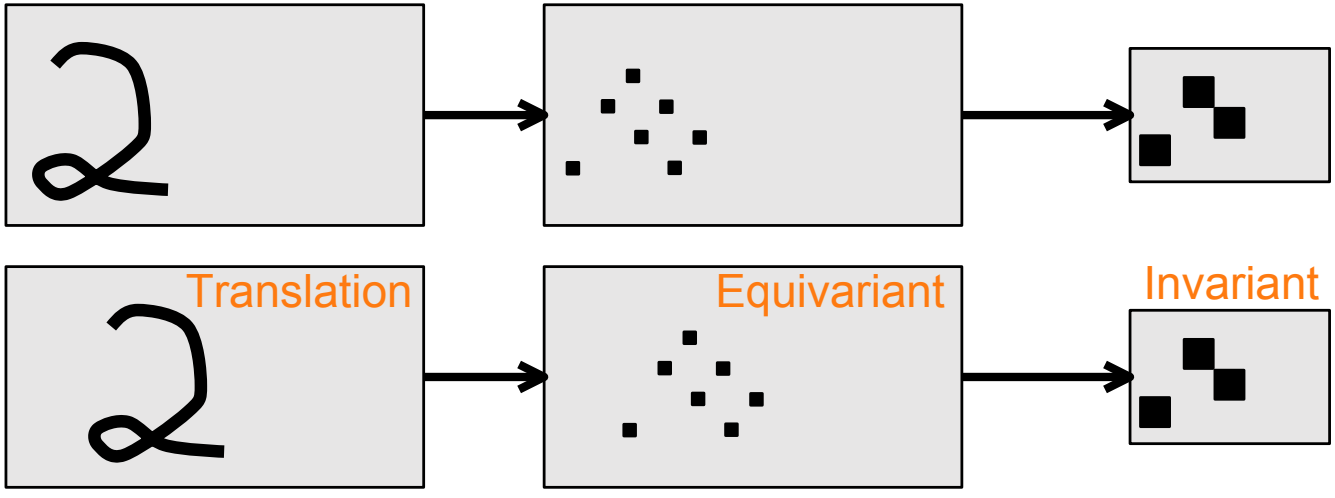
(local) spatial **POOLING** of the outputs of replicated feature detectors:

- Averaging neighboring replicated detectors to give a single output to the next level
- Max pooling: Taking the maximum in a neighboring



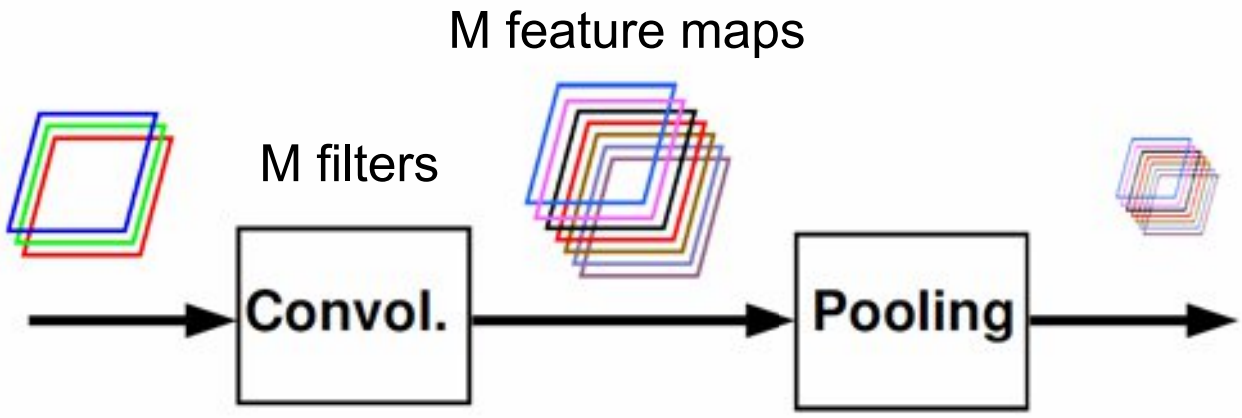
Get a small amount of translational invariance at each level

Reducing the number of inputs to the next layer of feature extraction



=> Stability OK (at least for local shift) for Convolutional Net!

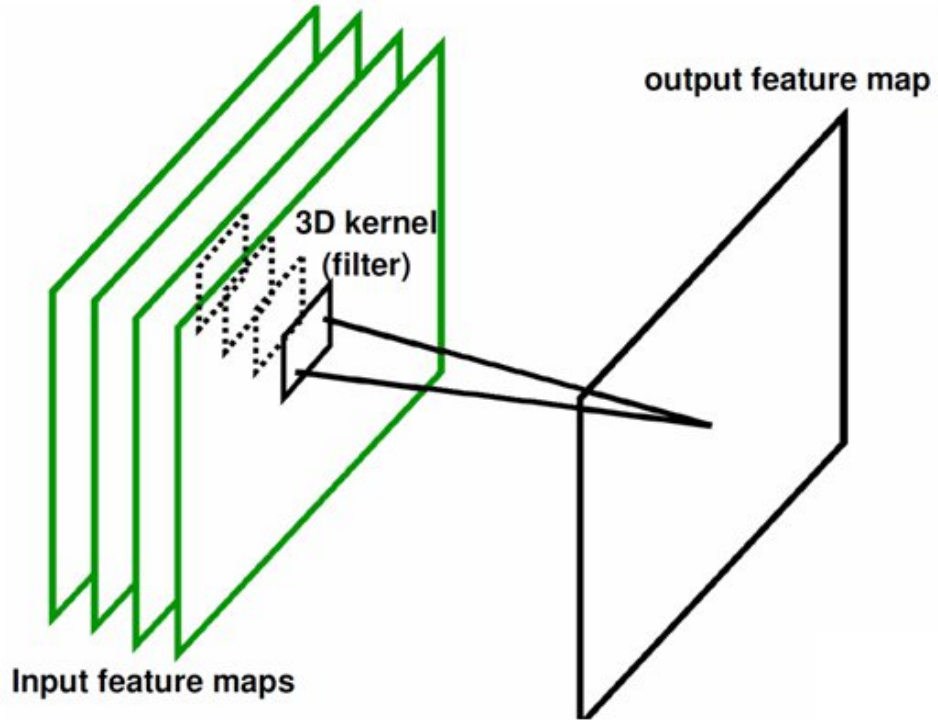
To sum up:

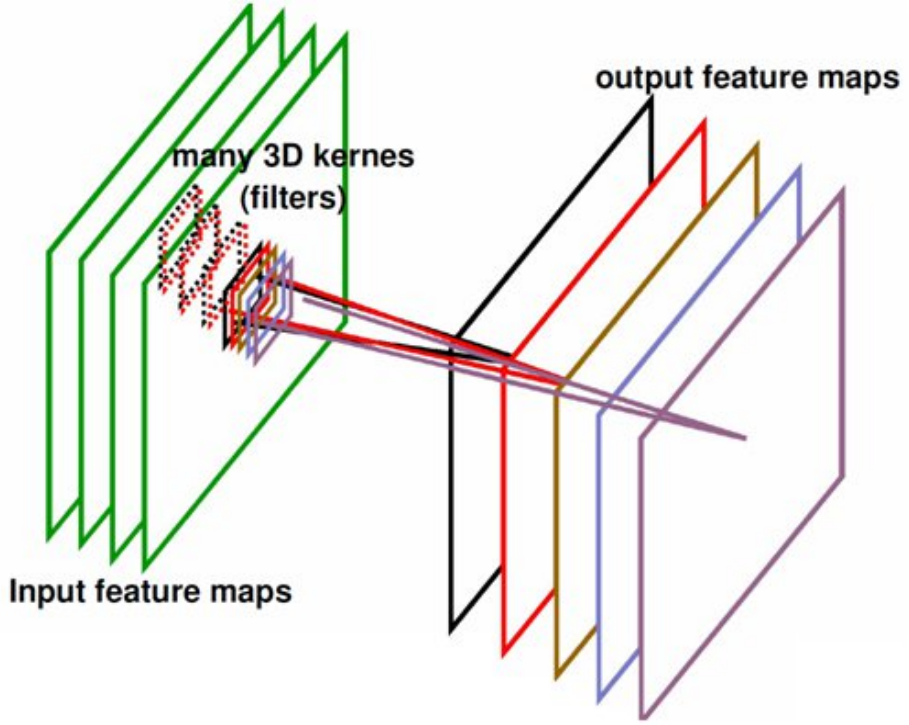


Color images: 3D kernels for filtering

mxnxd parameters per filter

Idem for any layer i to layer i+1

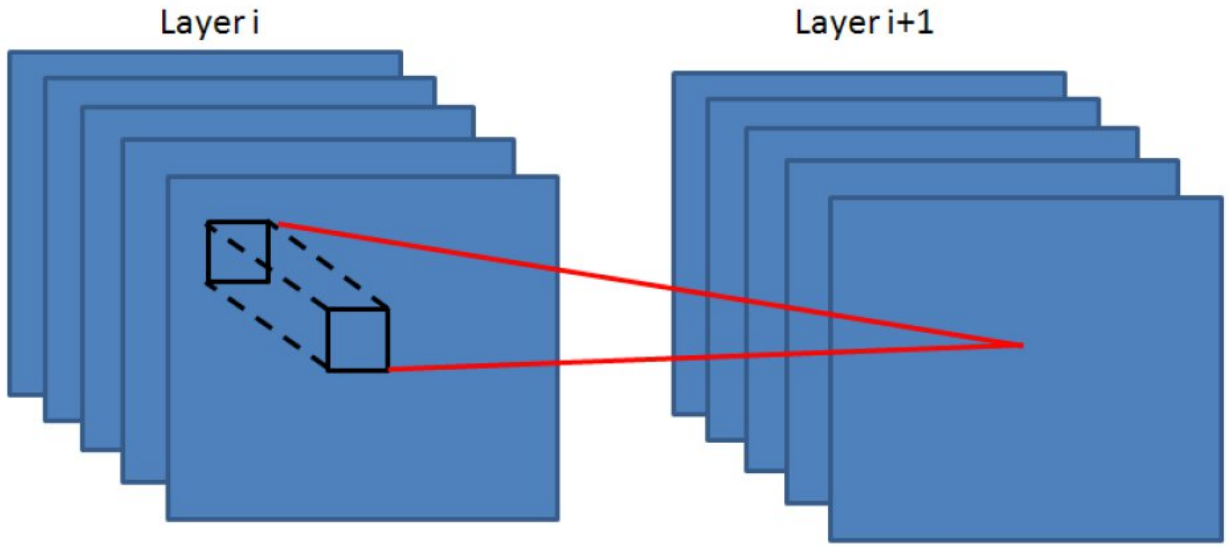




LCN: Local Contrast Normalization

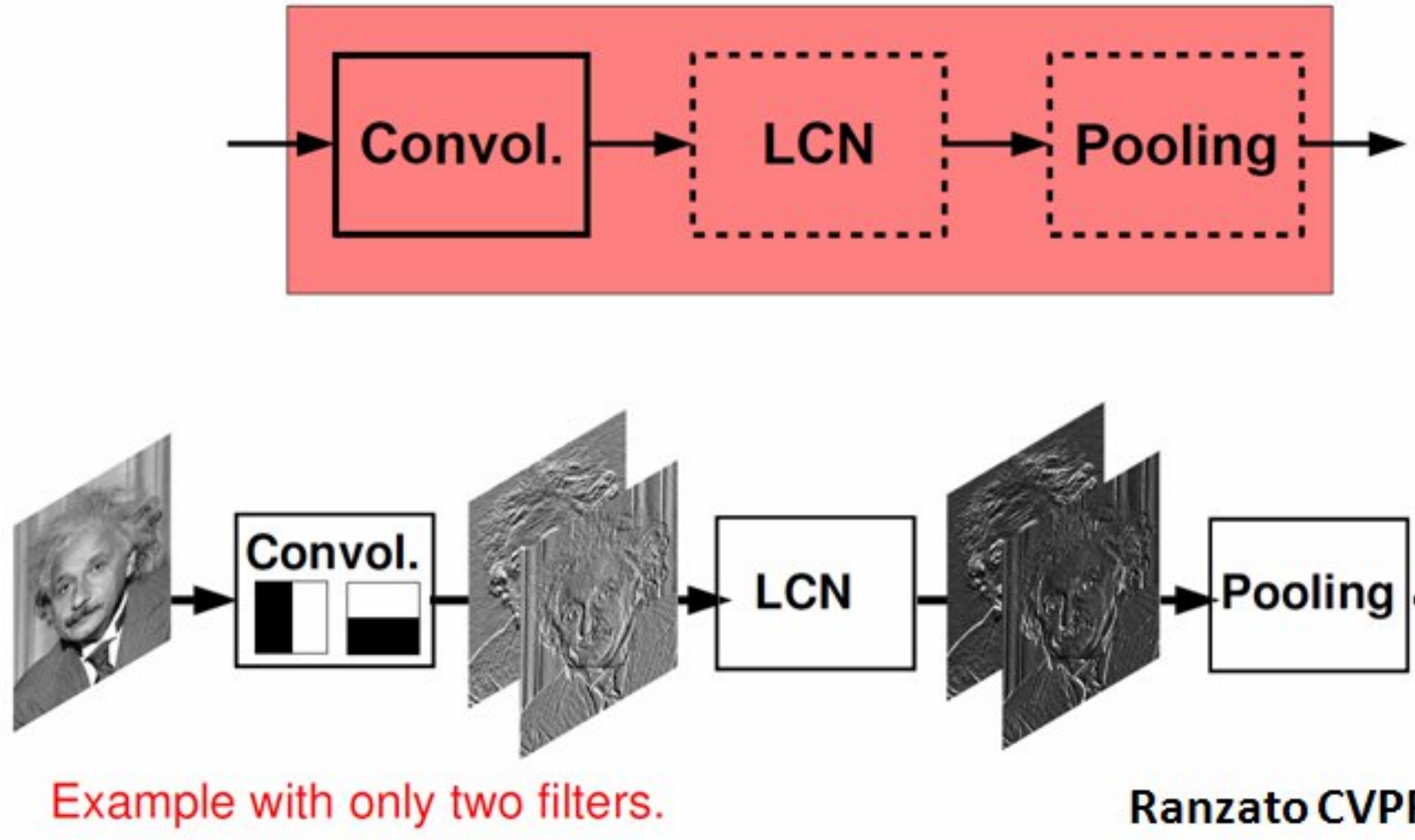
Normalization within a neighborhood along both spatial and feature dimensions

$$h_{i+1,x,y,k} = \frac{h_{i,x,y,k} - m_{i,N(x,y,k)}}{\sigma_{i,N(x,y,k)}}$$

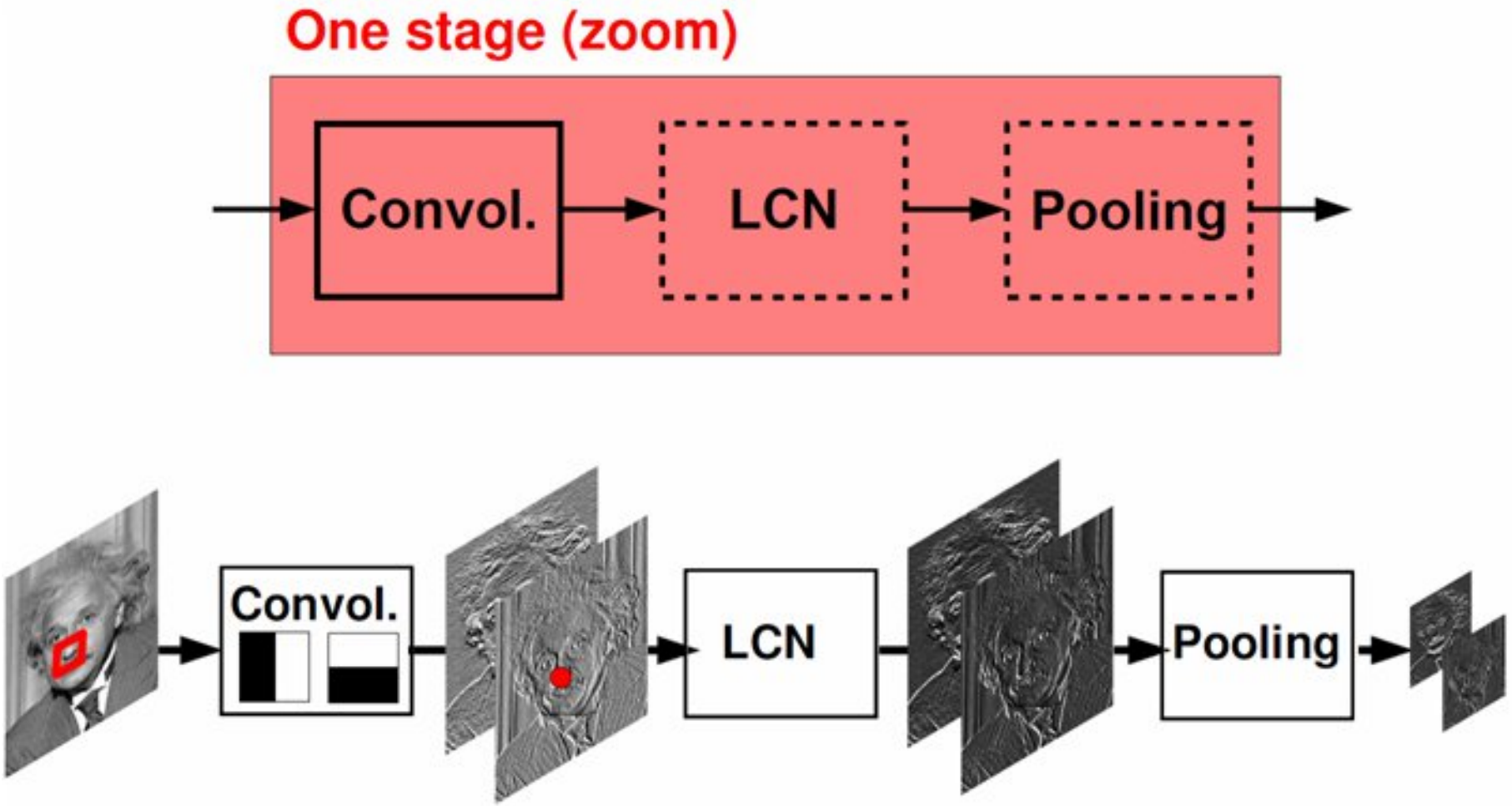


=> Very important for training large nets to carefully consider normalization within mini-batchs [S. Ioffe, C. Szegedy 2015]

1stage of convolutional neural networks



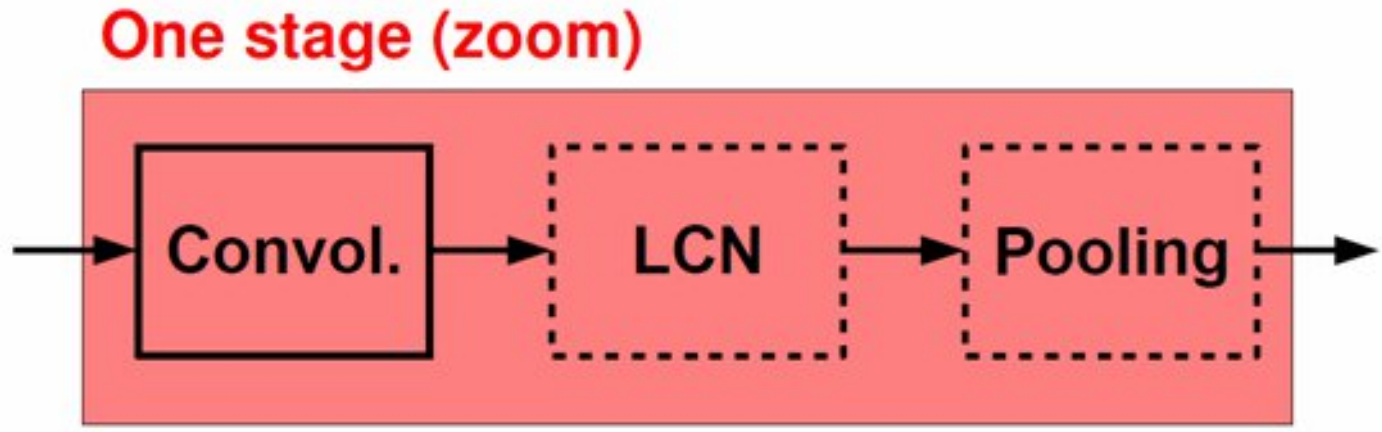
1stage of convolutional neural networks

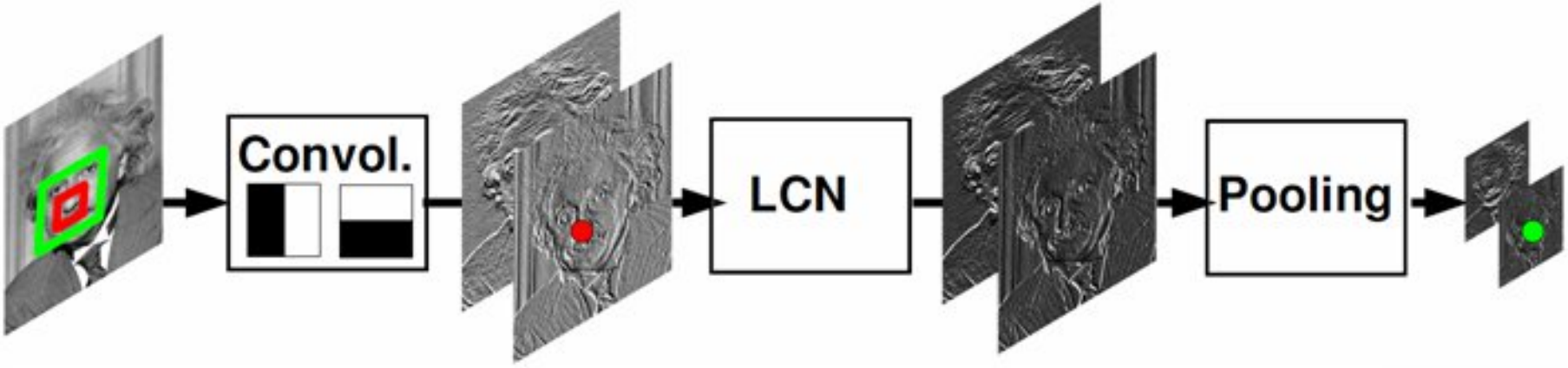


A hidden unit in the first hidden layer is influenced by a small neighborhood (equal to size of filter).

Ranzato CVPR'13

1stage of convolutional neural networks

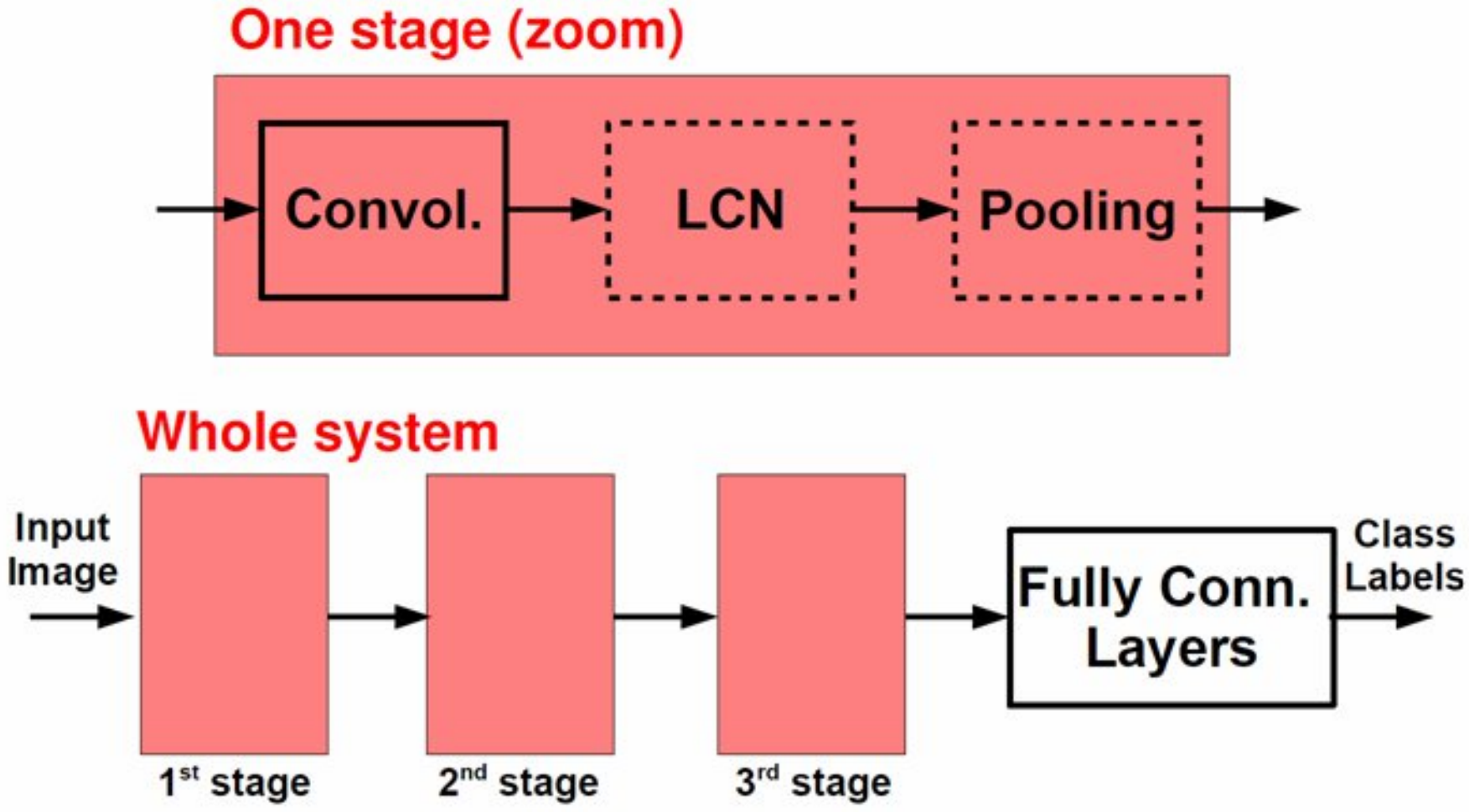




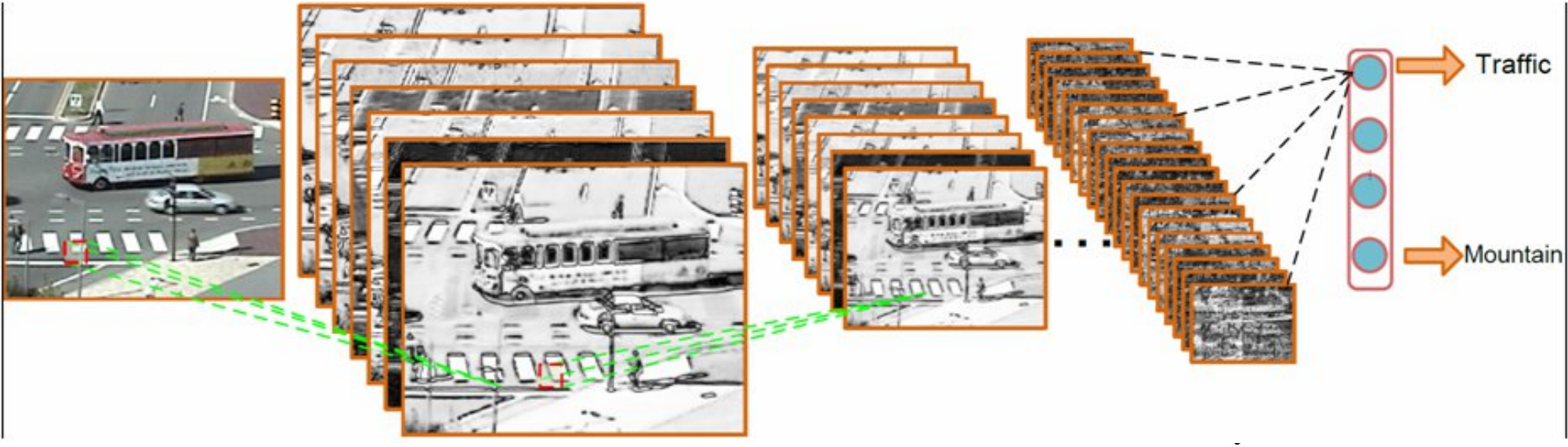
A hidden unit after the pooling layer is influenced by a larger neighborhood (it depends on filter sizes and the sizes of pooling regions)

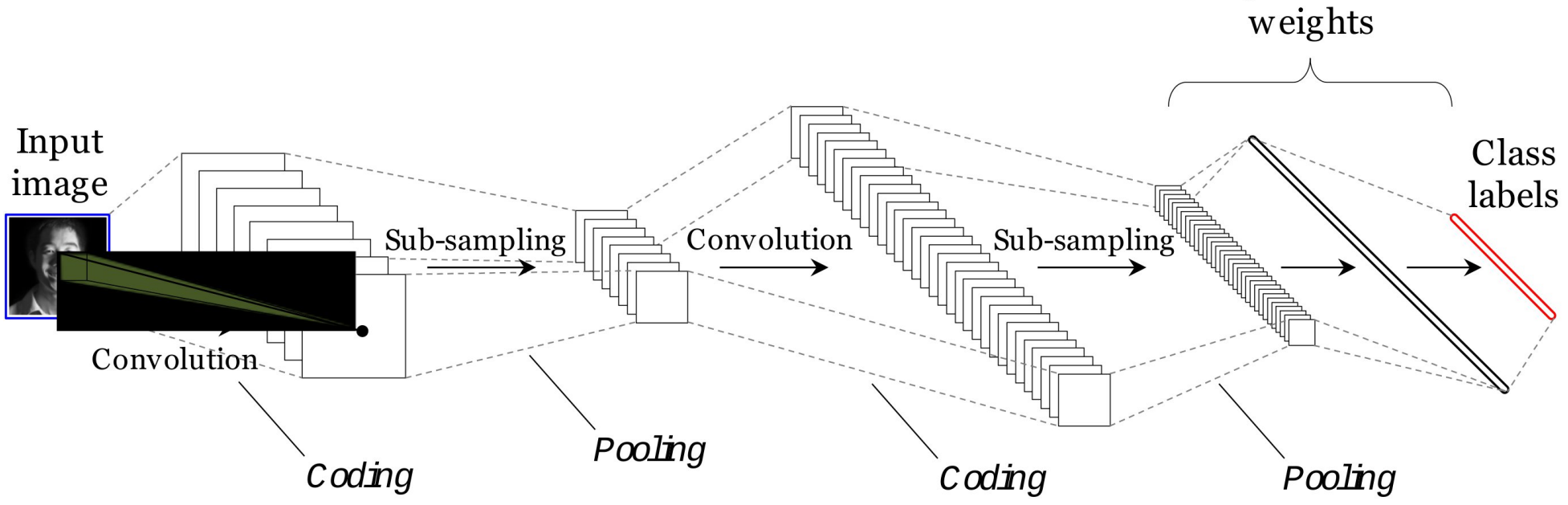
Ranzato CVPR'13

Full ConvNet architecture



To sum up: Full ConvNet architecture





To sum up: Full ConvNet architecture

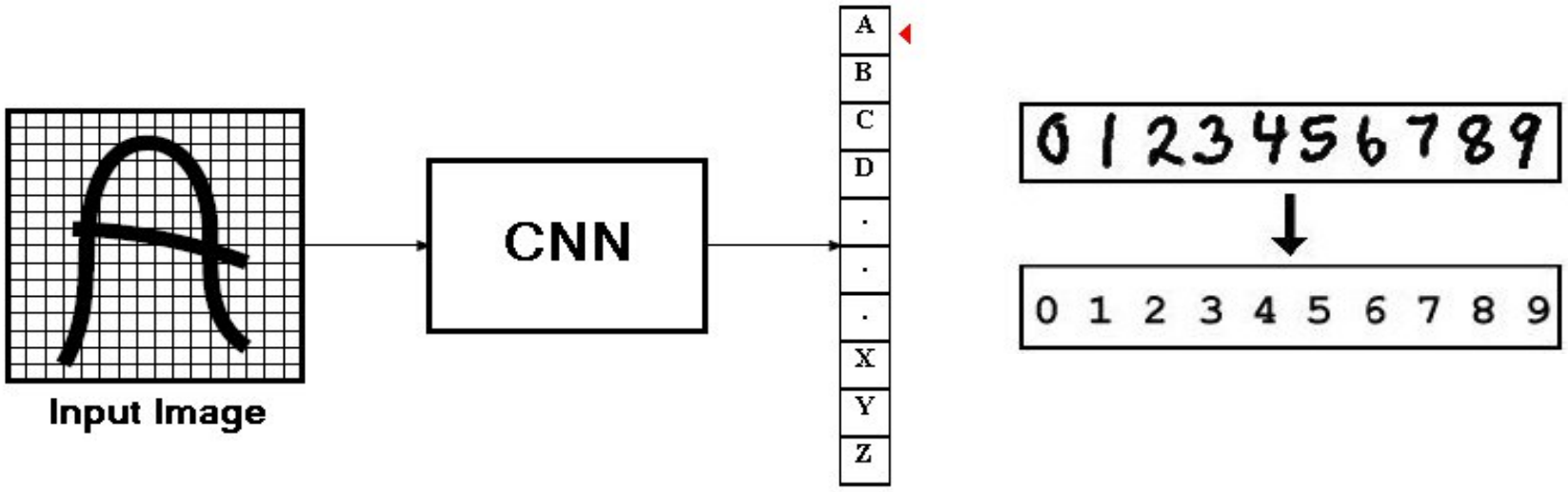
ConvNet (CNN): feed-forward network with

-- ability to extract topological properties from image

-- designed to recognize visual patterns

Working directly from pixel images with (no/minimal) preprocessing

Trained with back-propagation



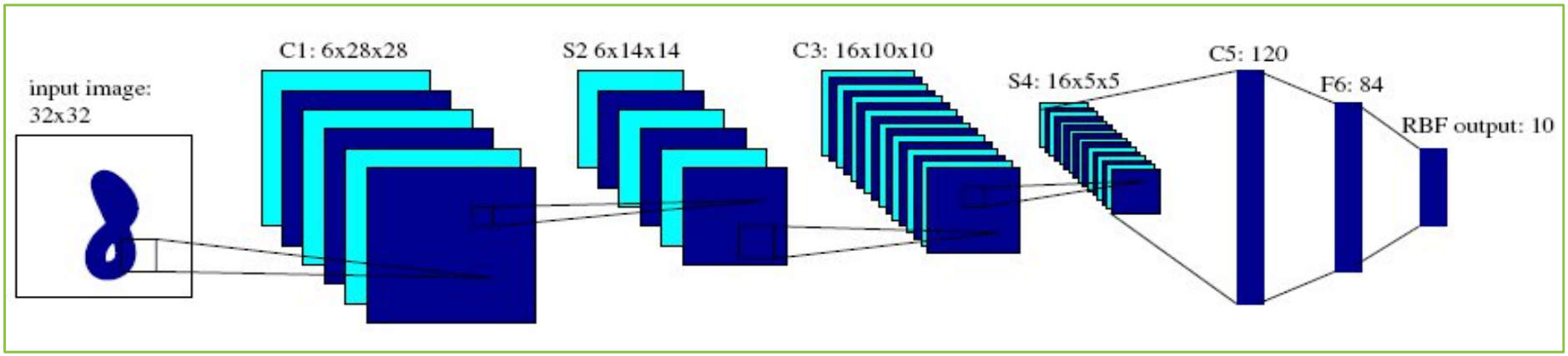
Outline
Convolutional Nets for visual classification

1. Recap MLP
2. Convolutional Neural Networks
3. **Examples: LeNet5, AlexNet**

Example: LeNet5

Introduced by Y. LeCun

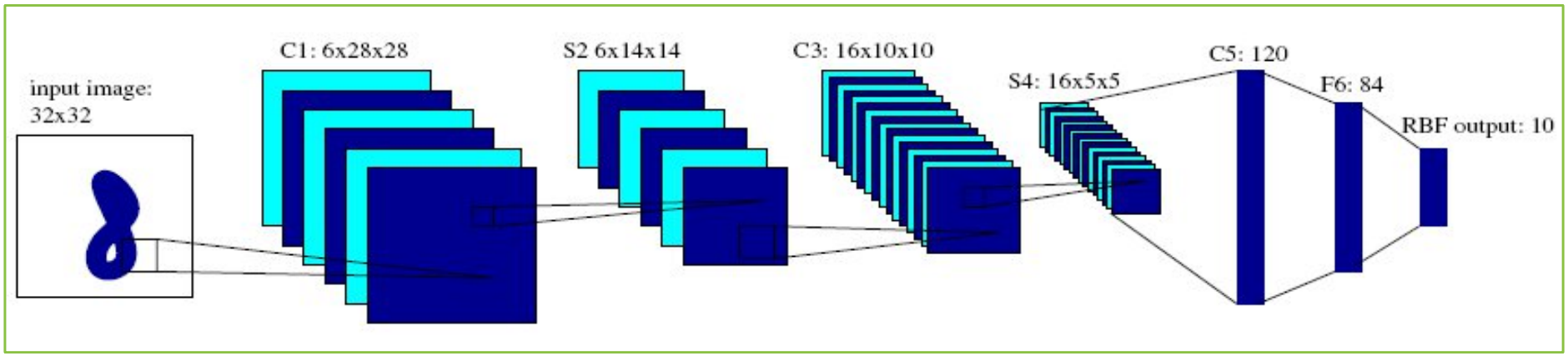
Raw image of 32 × 32 pixels as input



Example: LeNet5

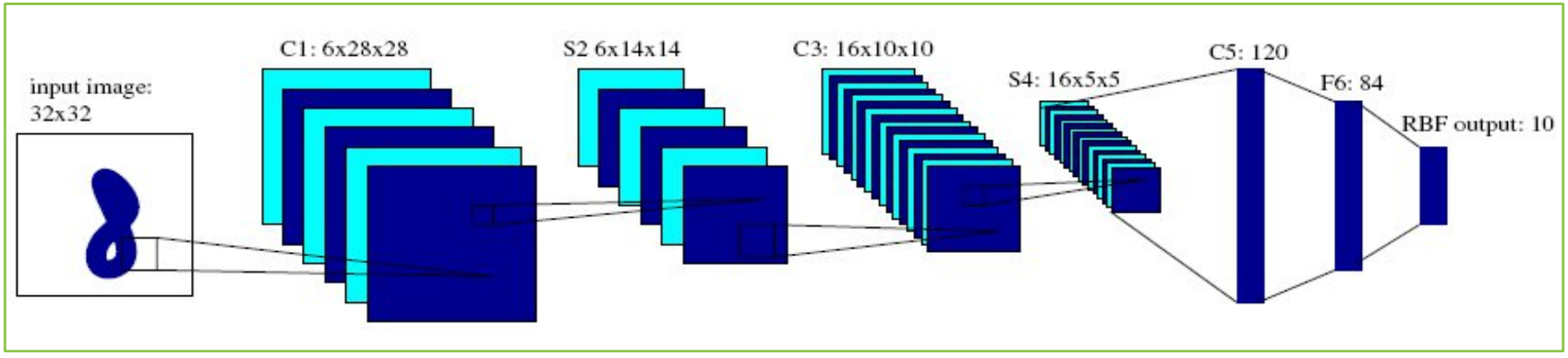
- C1,C3,C5 : Convolutional layer
- 5 × 5 Convolution matrix
- S2 , S4 : Subsampling layer = Pooling+stride s=2

  => Subsampling by factor 2
- F6 : Fully connected layer

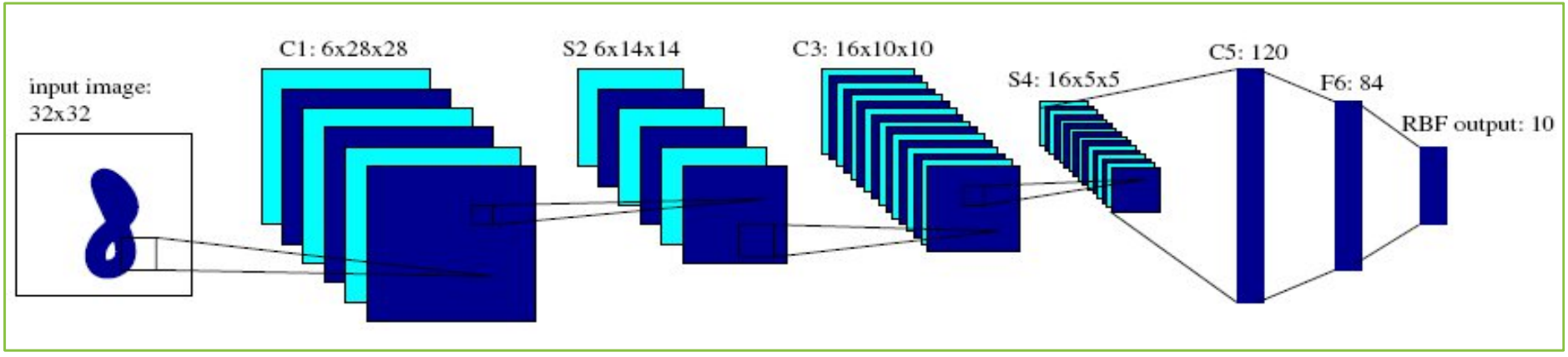


LeNet5

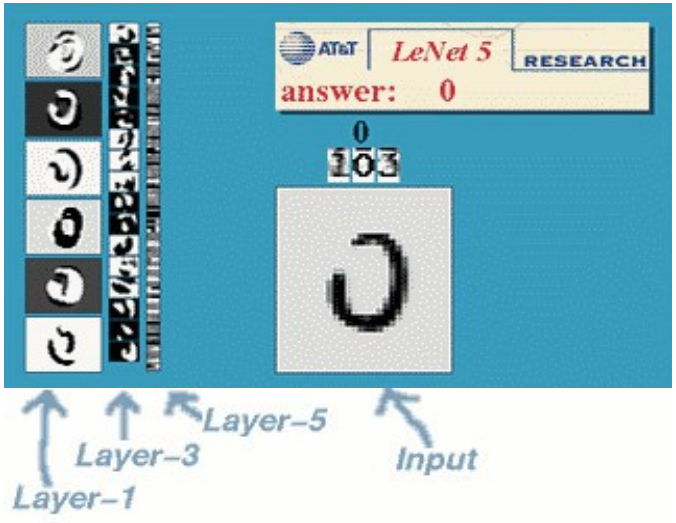
All the units of the layers up to F6 have a sigmoidal activation function



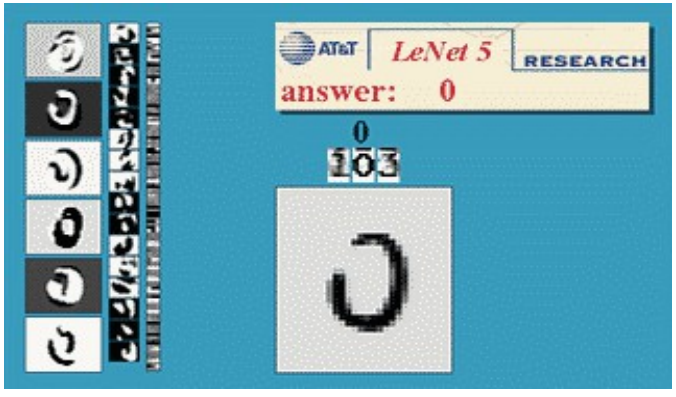
LeNet5

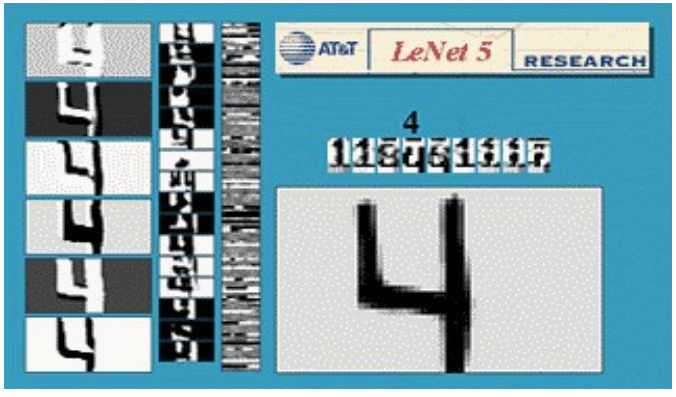


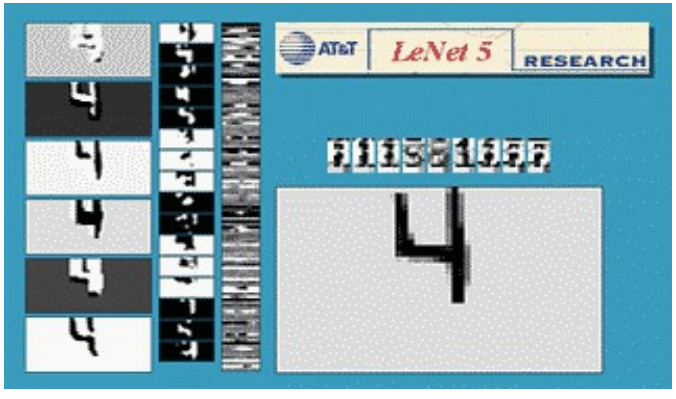
About 187,000 connections
About 14,000 trainable weights

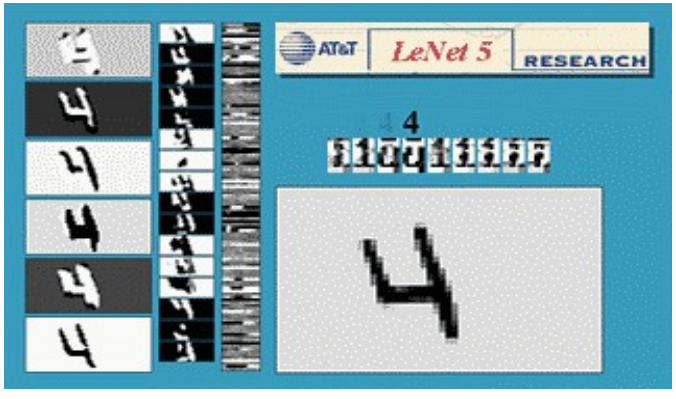


LeNet5 (@LeCun)

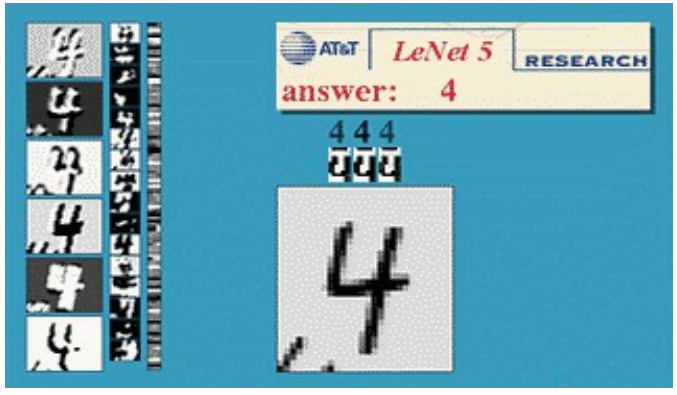


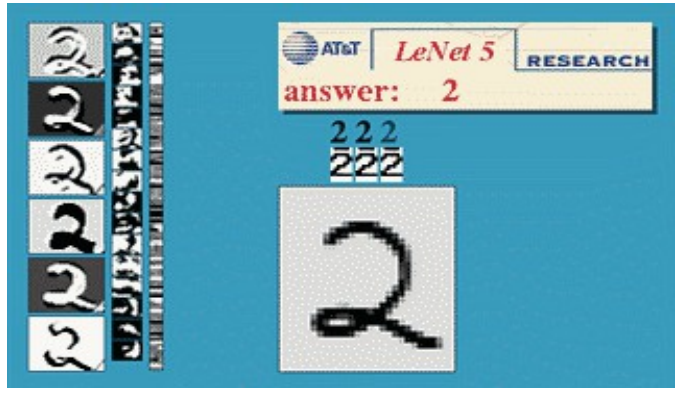


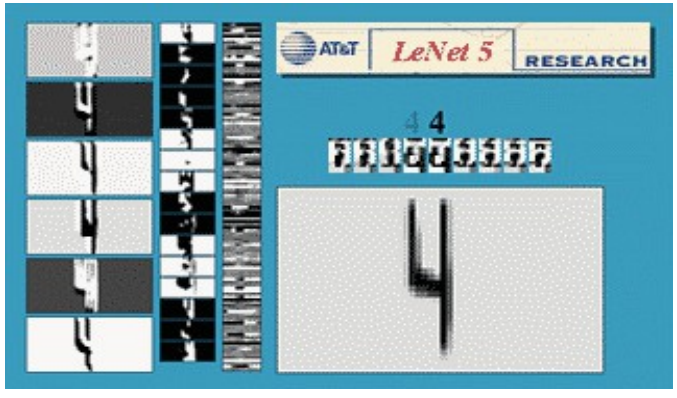


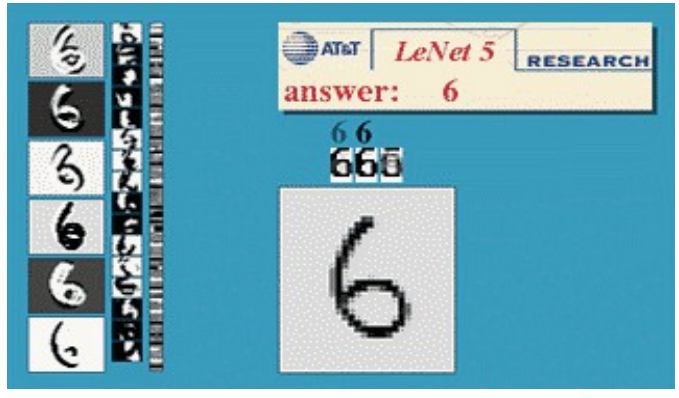


LeNet5 (@LeCun)



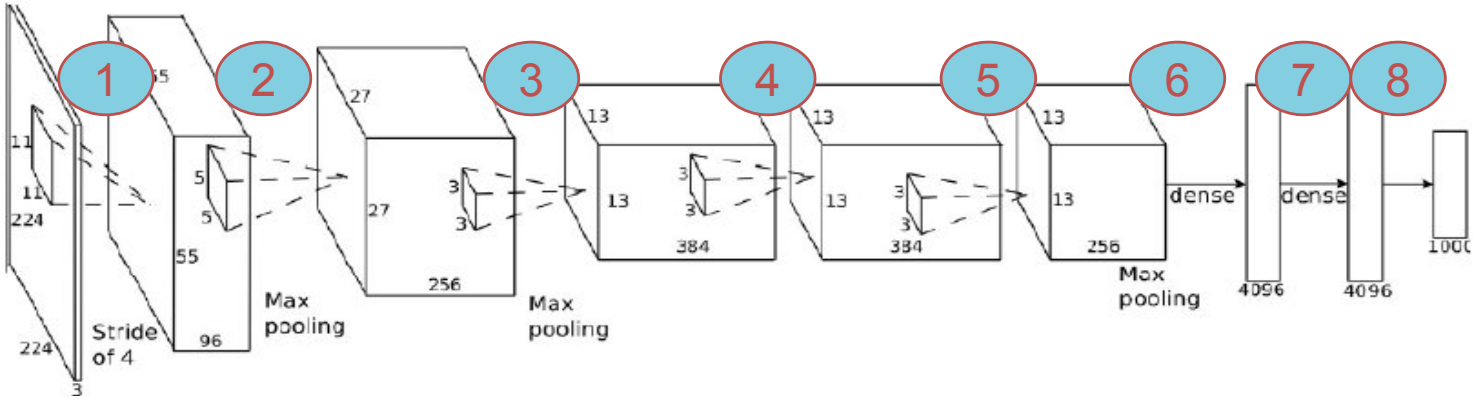




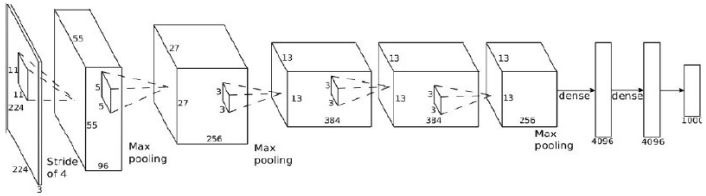


AlexNet 2012

- Same model as LeCun'98 but:
  - Bigger model (8 layers)
  - More data (10^6 vs 10^3 images)
  - GPU implementation (50x speedup over CPU)
  - Better regularization (DropOut)



AlexNet 2012

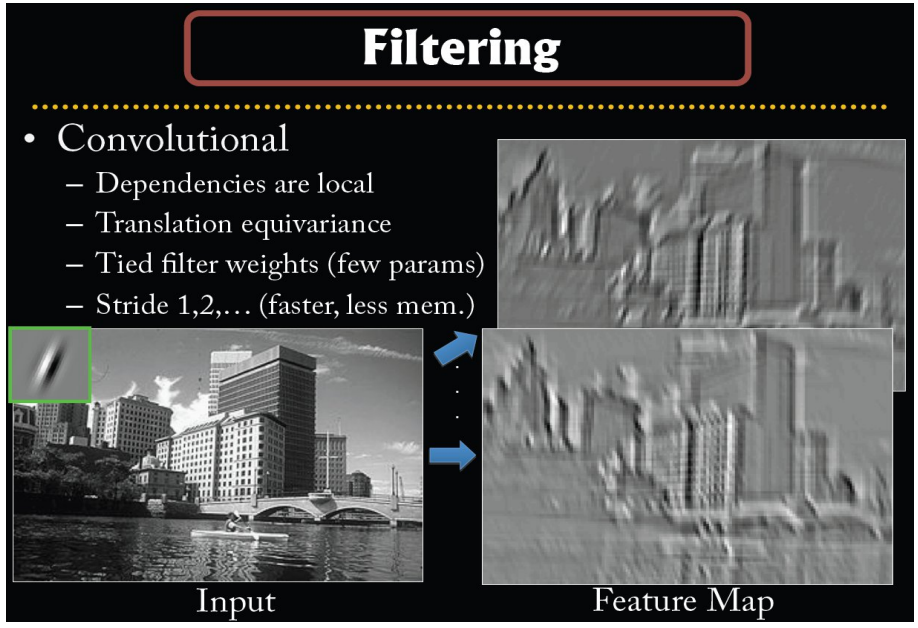


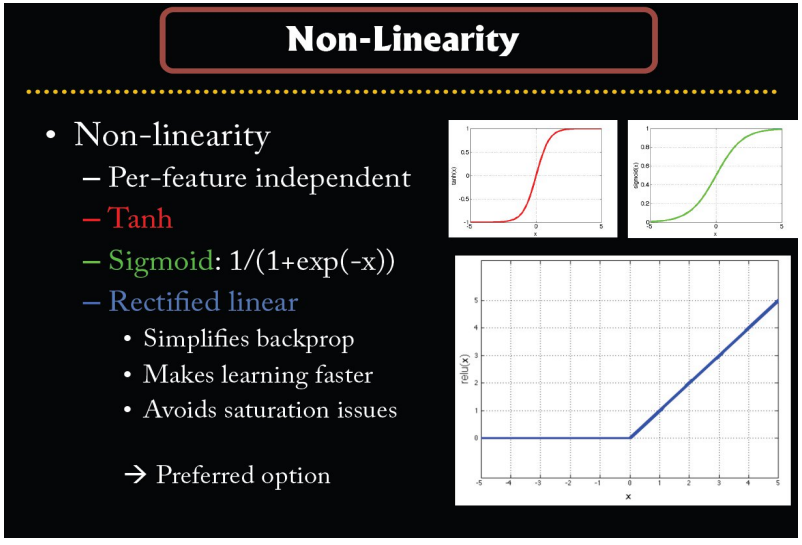
Same type of convnet with

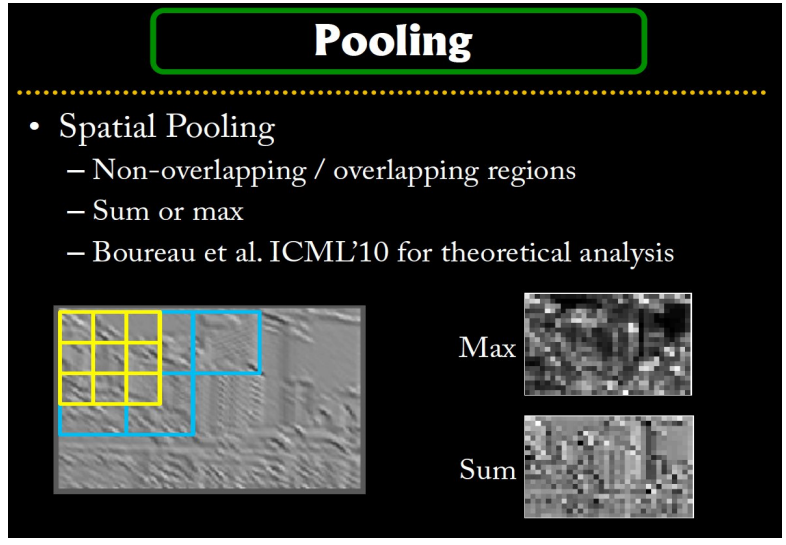
- Filtering (convolution)
- Non-Linearity
- Pooling

8 layers but 224x224 input images => much biger model:

- 650,000 neurons
- 60,000,000 weights!



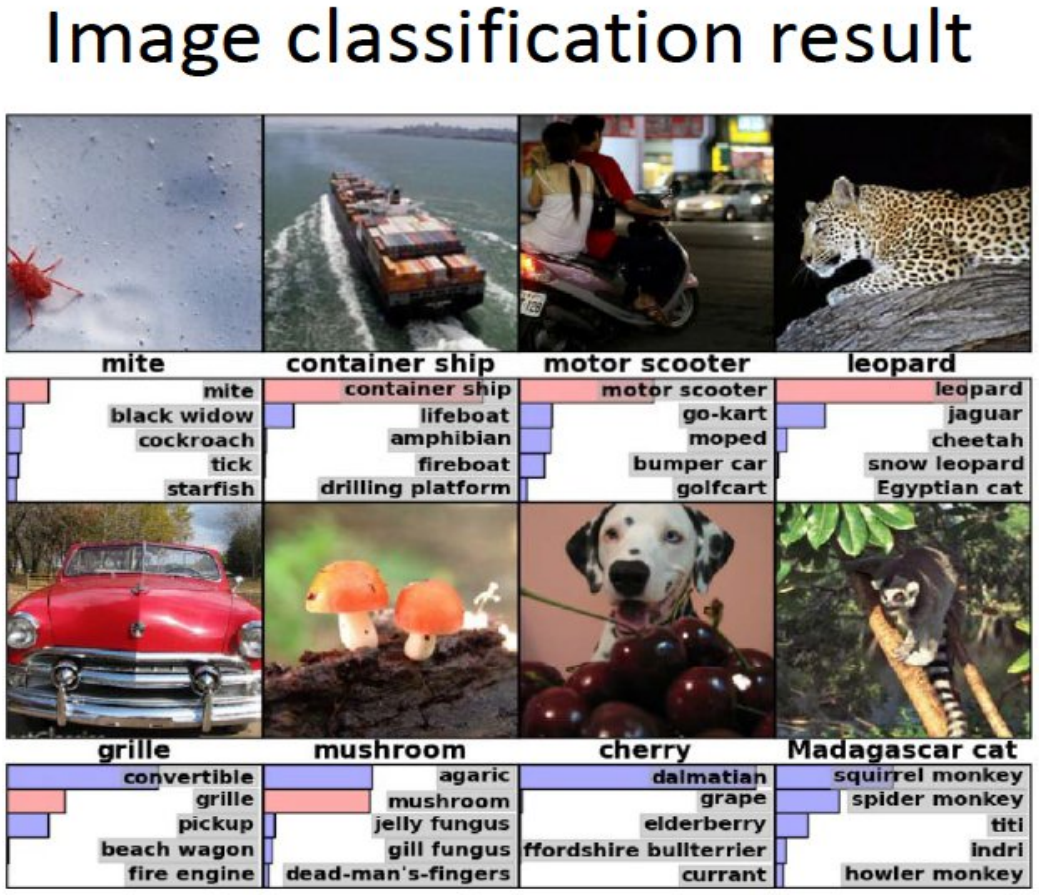




More data for supervised training

ImageNet 2012: the (deep) revolution

- 1.2 million labeled images
- 1000 classes
- Mono-class
- TOP5



Learning the AlexNet

- Basics:
  - SGD, Backprop
  - Cross Validation
  - Grid search
- “New”
  - Huge computational resources (GPU)
  - Huge training set (1 million images)
  - Data augmentation - Pre-processing
  - Dropout
  - ReLu
  - *Contrast normalization*

Data Augmentation

lots of jittering, mirroring, and color perturbation of the original images generated on the fly to increase the size of the training set

Crop, flip,.. in train / in test

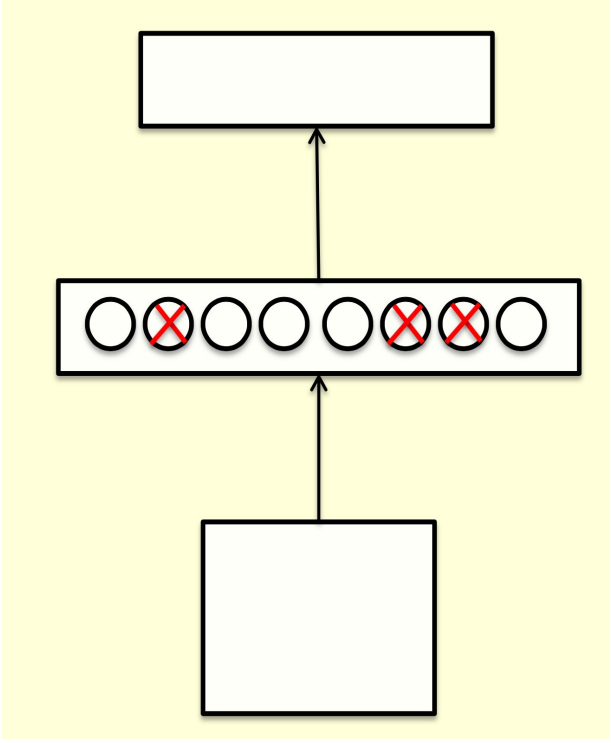


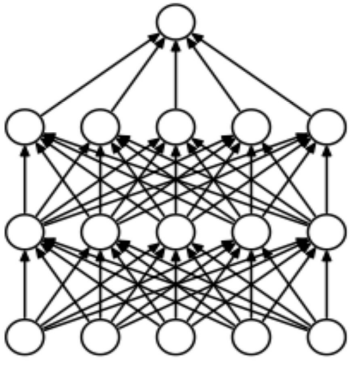
Dropout: an efficient way to average many large neural nets

For each training example, randomly omit each hidden unit with probability 0.5

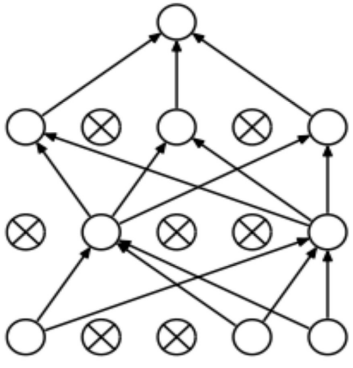
Due to sharing of weights, model strongly regularized

- Pulls the weights towards what other models want.
- Better than L2 and L1 regularization that pull weights towards zero





Standard Neural Net



After applying dropout.

@Hinton, NIPS 2012

Dropout: what do we do at test time?

Option 1:

Sample many different architectures and take the geometric mean of their output distributions

Option 2: (Faster way)

Use all the hidden units

but after **halving their outgoing weights**

Rq: In case of single hidden layer, this is equivalent to the geometric mean of the predictions of all models

For multiple layers, it's a pretty good approximation and its fast

How well does dropout work?

Improving generalization:

- For very deep nets, or at least when there are huge fully connected layers (eg. AlexNet first FC layer, VGG next, …)
- Less useful for fully convolutional nets

Useful to prevent feature co-adaptation (feature only helpful when other specific features present)

Later in course

Þ **Dropout as a Bayesian Approximation**

Þ **Representing Model Uncertainty in Deep Learning**

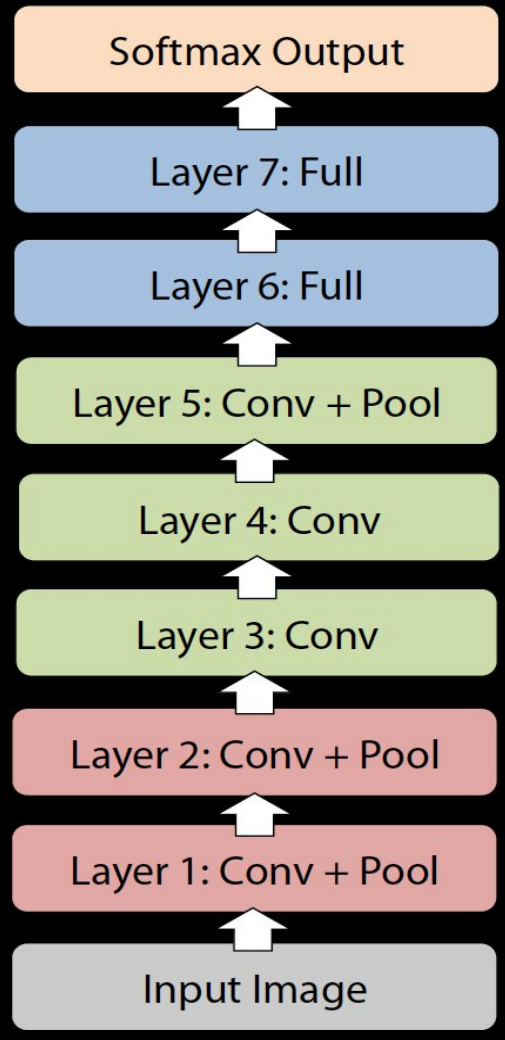
AlexNet 2012

Ablation study

1. **Number of layers**
2. ***Tapping off features at each layer***
3. **Transfo Robustness vs layers**

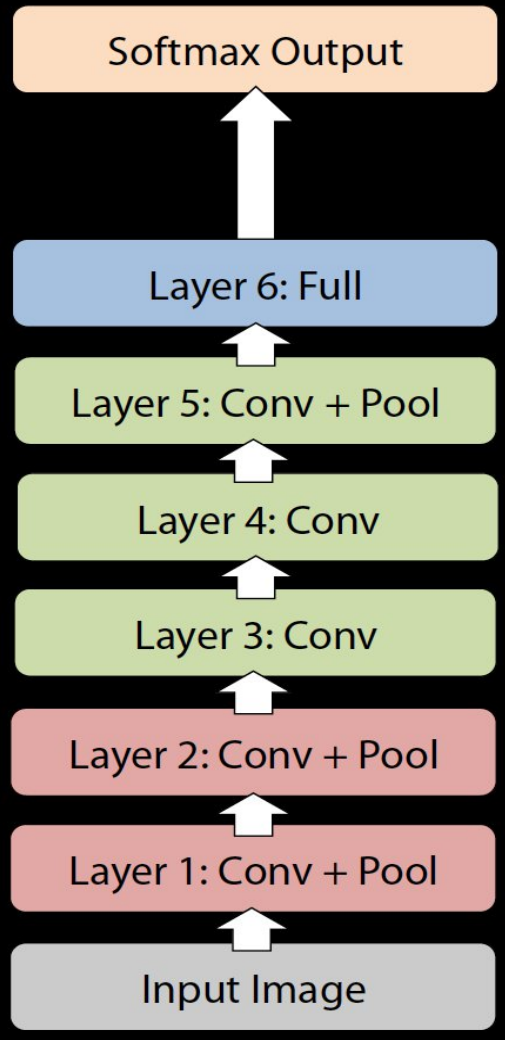
Architecture of Krizhevsky et al.

- 8 layers total
- Trained on Imagenet dataset [Deng et al. CVPR'09]
- 18.2% top-5 error
- Our reimplementation:
  18.1% top-5 error



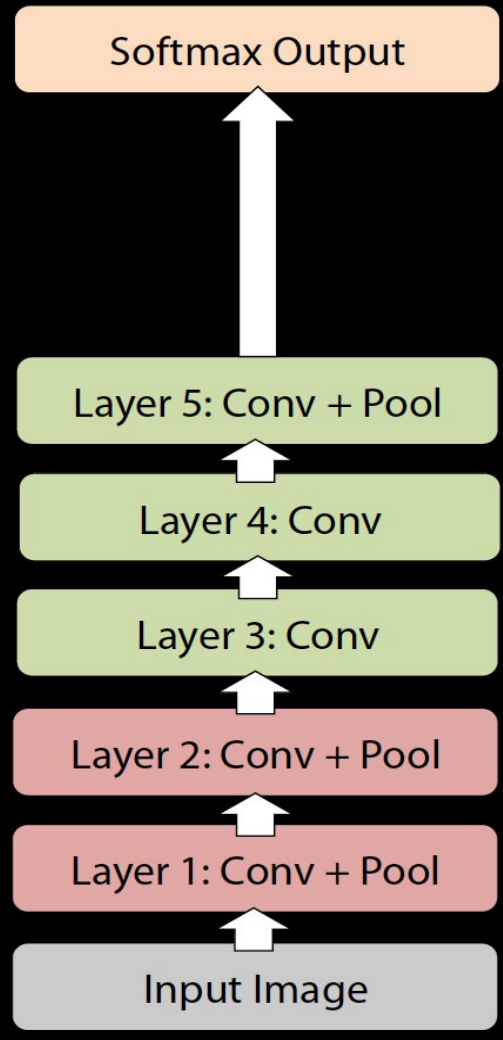
Architecture of Krizhevsky et al.

- Remove top fully connected layer
  - Layer 7
- Drop 16 million parameters
- Only 1.1% drop in performance!



Architecture of Krizhevsky et al.

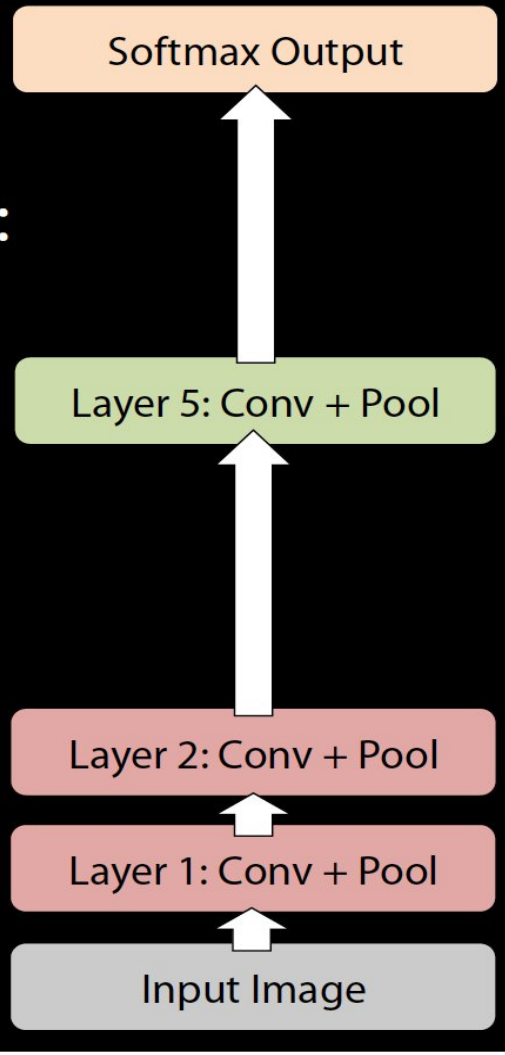
- Remove both fully connected layers
  - Layer 6 & 7
- Drop ~50 million parameters
- 5.7% drop in performance



Architecture of Krizhevsky et al.

- Now try removing upper feature extractor layers & fully connected:
  - Layers 3, 4, 6 ,7
- Now only 4 layers
- 33.5% drop in performance

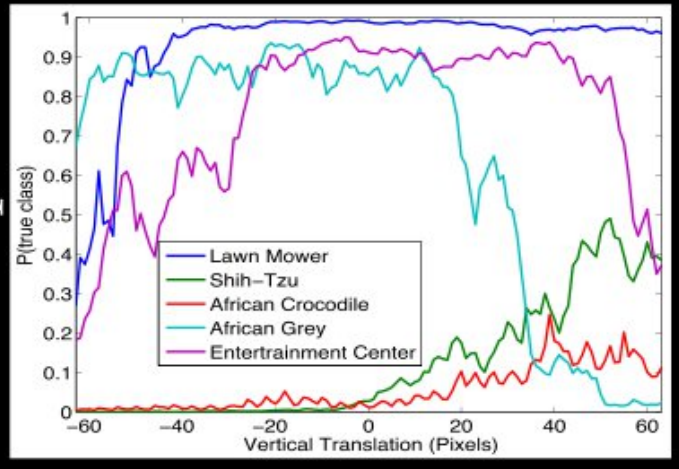
→Depth of network is key



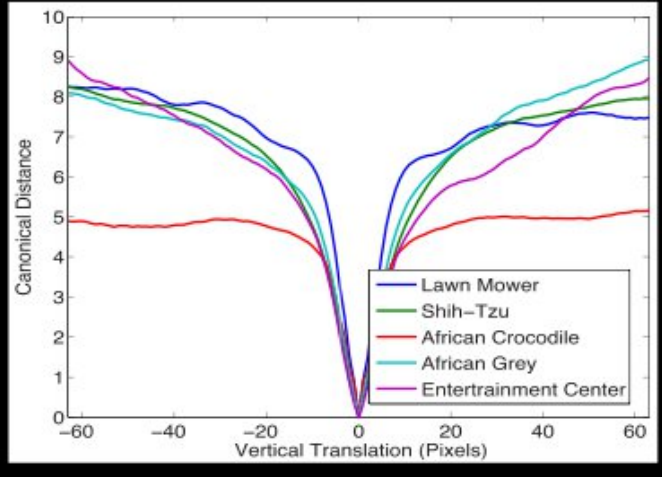
Translation (Vertical)



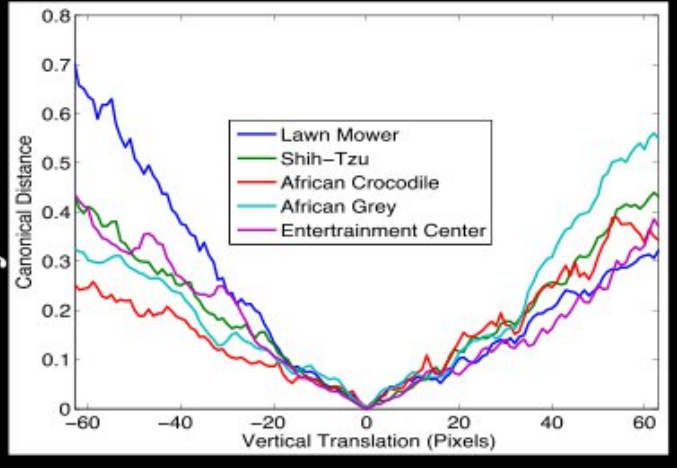
Output



Layer 1



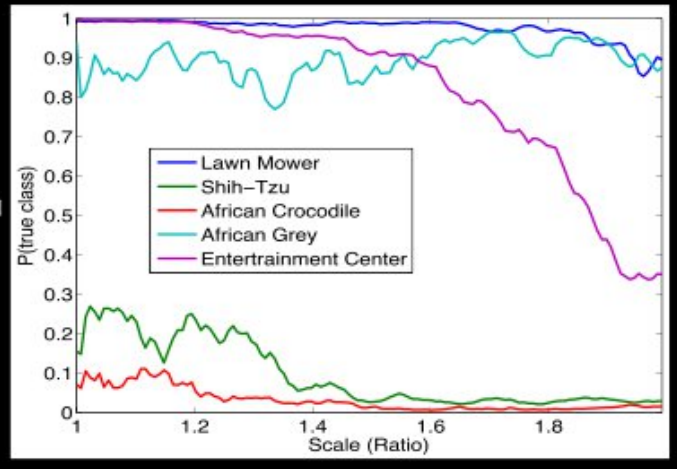
Layer 7



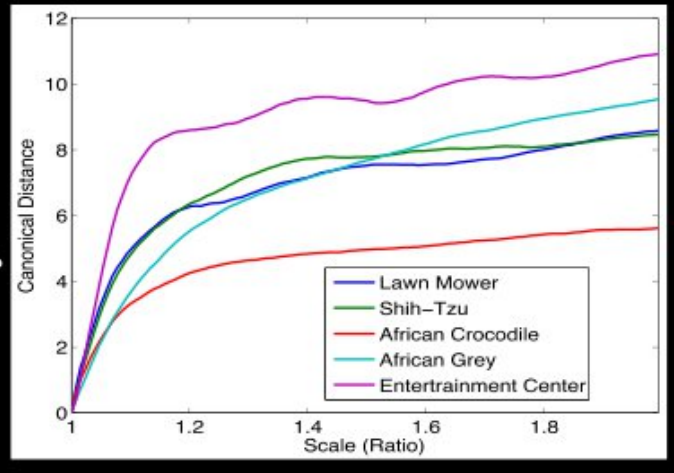
Scale Invariance



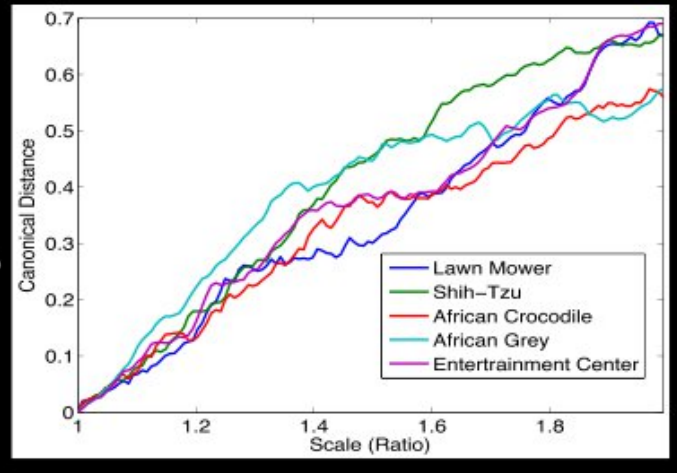
Output



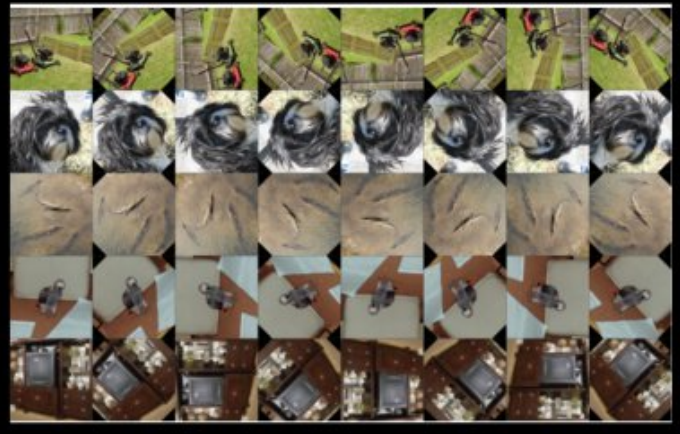
Layer 1



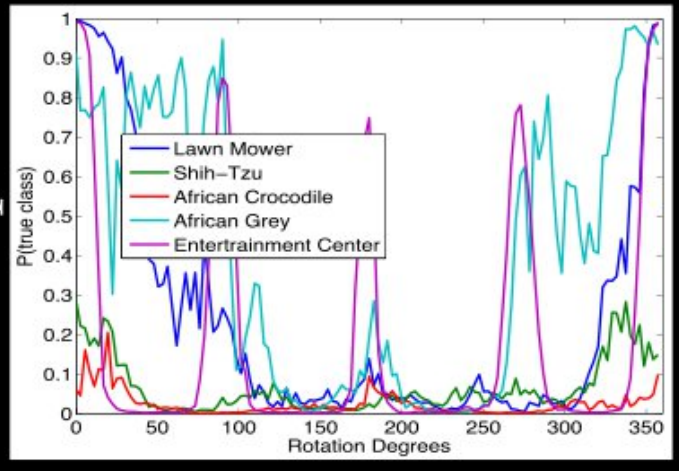
Layer 7



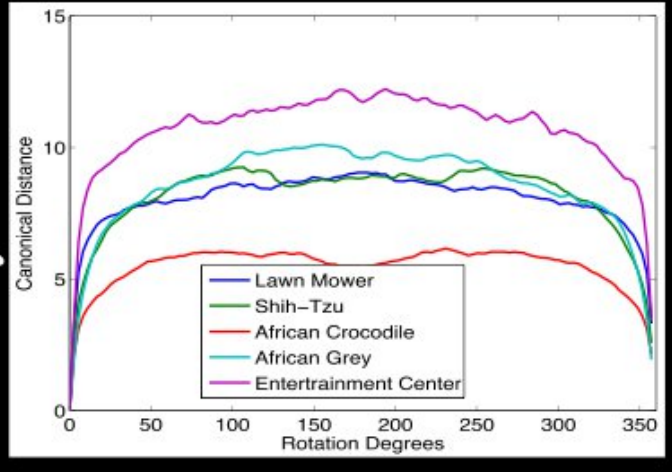
Rotation Invariance



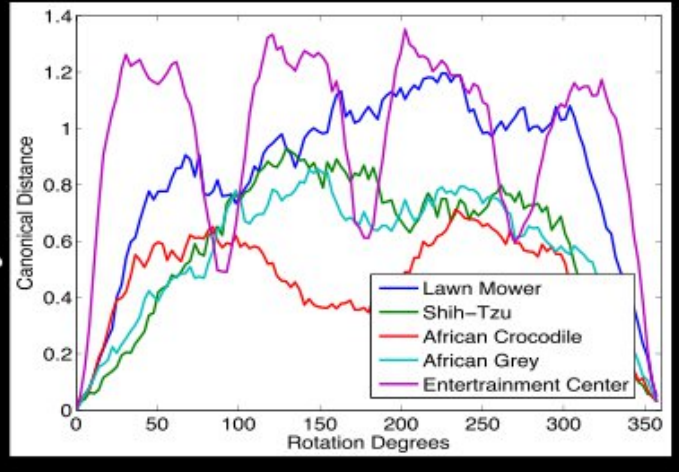
Output



Layer 1

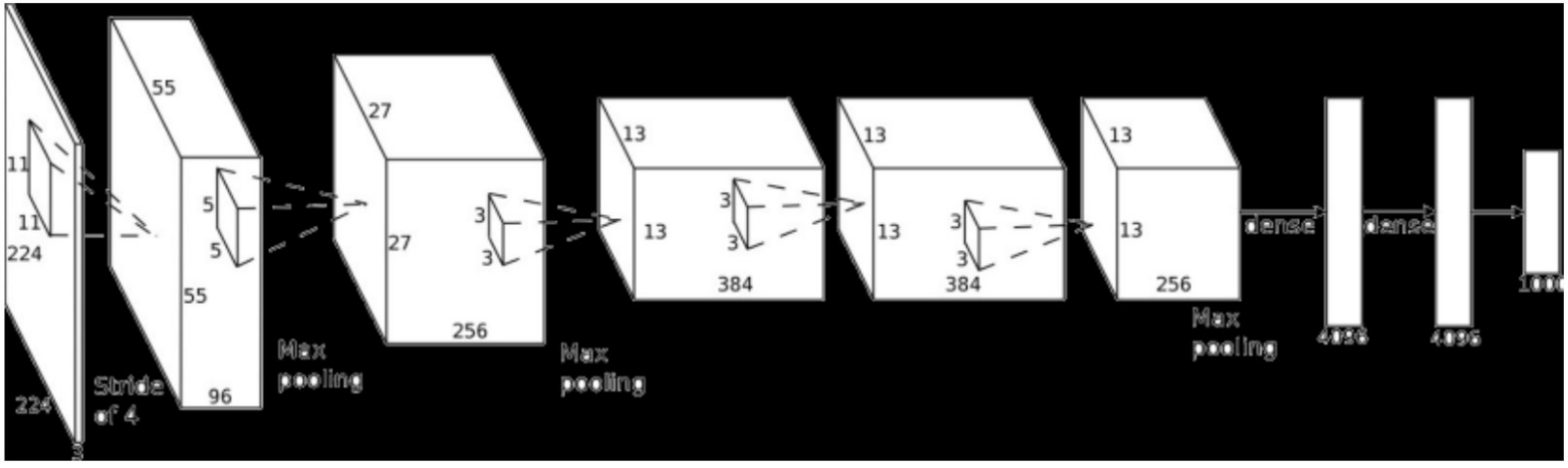


Layer 7

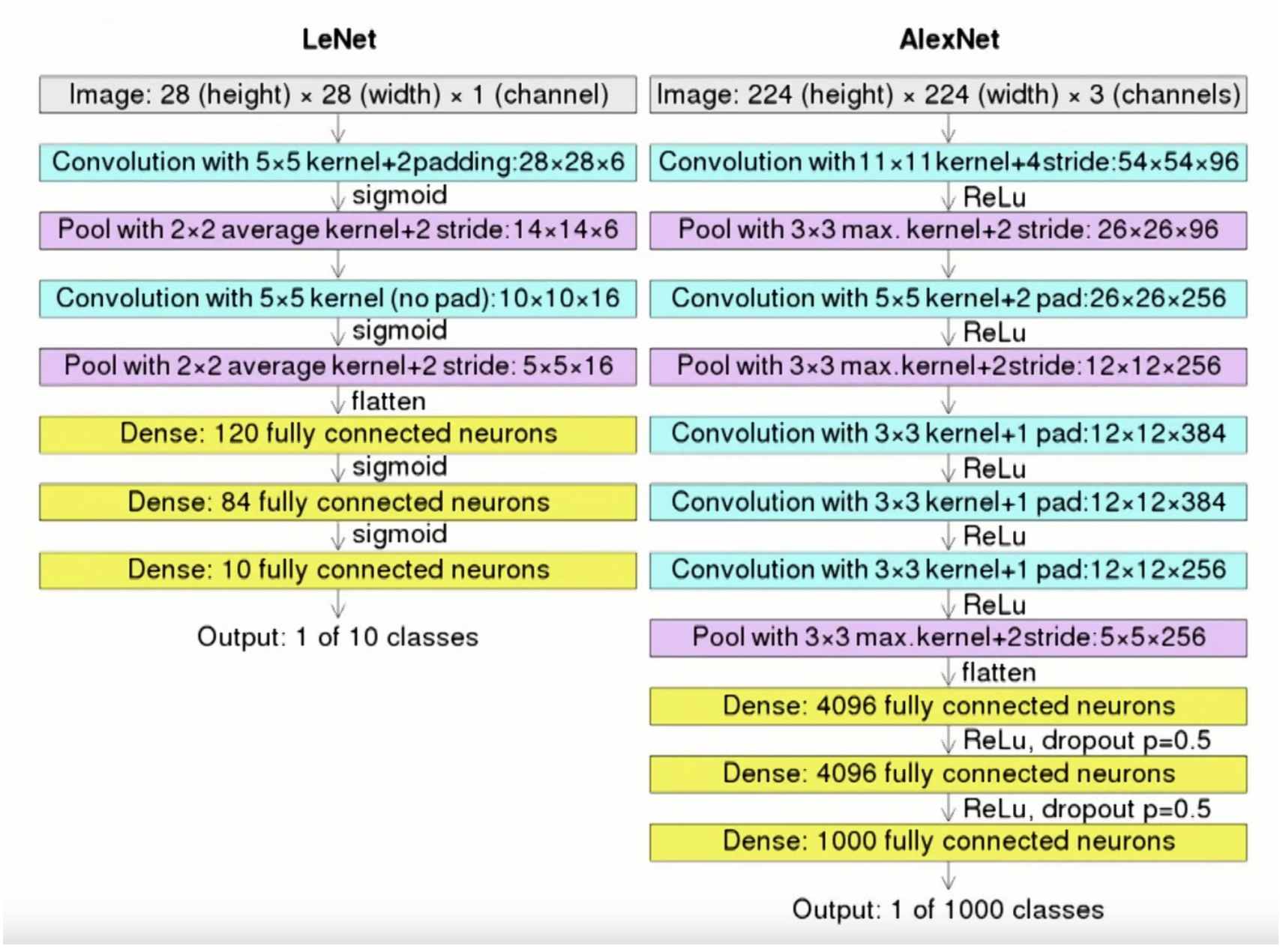


Deep ConvNets for image classification

- AlexNet 8 layers, 62M parameters



Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton
ImageNet Classification with Deep Convolutional Neural Networks.
In *NIPS*, 2012.


LeNet
Image: 28 (height) × 28 (width) × 1 (channel)
Convolution with 5×5 kernel+2padding:28×28×6
sigmoid
Pool with 2×2 average kernel+2 stride:14×14×6
Convolution with 5×5 kernel (no pad):10×10×16
sigmoid
Pool with 2×2 average kernel+2 stride: 5×5×16
flatten
Dense: 120 fully connected neurons
sigmoid
Dense: 84 fully connected neurons
sigmoid
Dense: 10 fully connected neurons
Output: 1 of 10 classes
AlexNet
Image: 224 (height) × 224 (width) × 3 (channels)
Convolution with 11×11 kernel+4 stride:54×54×96
ReLu
Pool with 3×3 max. kernel+2 stride: 26×26×96
Convolution with 5×5 kernel+2 pad:26×26×256
ReLu
Pool with 3×3 max.kernel+2stride:12×12×256
Convolution with 3×3 kernel+1 pad:12×12×384
ReLu
Convolution with 3×3 kernel+1 pad:12×12×384
ReLu
Convolution with 3×3 kernel+1 pad:12×12×256
ReLu
Pool with 3×3 max.kernel+2stride:5×5×256
flatten
Dense: 4096 fully connected neurons
ReLu, dropout p=0.5
Dense: 4096 fully connected neurons
ReLu, dropout p=0.5
Dense: 1000 fully connected neurons
Output: 1 of 1000 classes

Outline

1 Convolutional Neural Networks

2 Modern Deep Architectures

3 Implementation

4 Transfer Learning

Challenges for Training Deep Learning Models

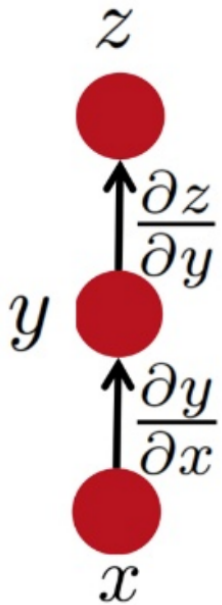
- Training deep ConvNets : Gradient descent on loss function $\mathcal{L}$:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla \mathcal{L}\left(\mathbf{w}^t\right)$$

- Error-Backpropagation : way to compute $\nabla \mathcal{L}\left(\mathbf{w}^t\right)$ in neural networks
  - Analytical expression of the gradient straightforward : **chain rule**
  - **BUT :** efficient evaluation of gradient can be very tricky

$\Rightarrow$ **Efficient backprop implementation far from trivial**

1. **Numerical Optimization Issues**
2. **Automatic Differentiation**

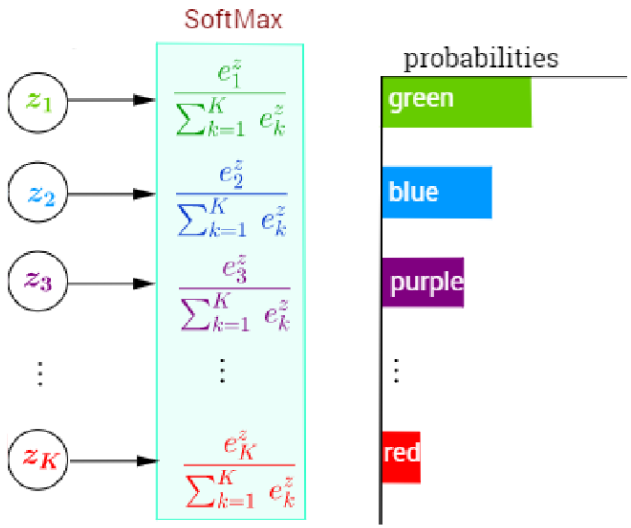


Numerical Optimization Issues

- Accumulating approximations (rounding) can be problematic
- Underflow : $x \approx 0$ or $x = 0$: different behaviors
  - Division by 0 $\Rightarrow$ Nan
- Overflow : large > 0 or < 0 numbers $\Rightarrow$ Nan
- Ex : softmax on $\mathbf{x} = \{x_1, x_2, ..., x_K\}$:

$$SM(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$$

- What if $x_i = C \ \forall i$, and :
  - $C \to +\infty$?
  - $C \to -\infty$?



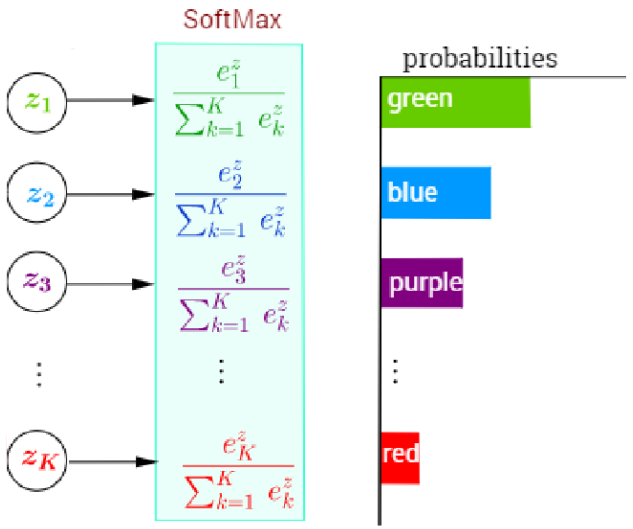
Numerical Optimization Issues

$$SM(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$$

- If $x_i = C \ \forall i \ SM(\mathbf{x})_i = \frac{1}{K} \ \forall i$ expected
  - $C \to -\infty \Rightarrow e^C \to 0$: division by 0, Nan ! (underflow)
  - $C \to +\infty \Rightarrow e^C \to +\infty$: Nan ! (overflow)
- Numerical stabilization for denominator :

$$\mathbf{z} = \mathbf{x} - max_i(x_i) \Rightarrow SM(\mathbf{z}) = SM(\mathbf{x})$$

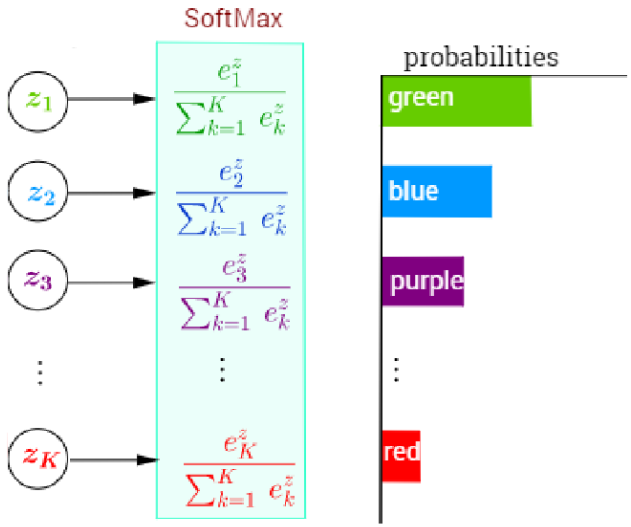
  - $max_i(e^{z_i}) = 1 \Rightarrow$ no overflow
  - $max_i(e^{z_i}) = 1 \Rightarrow$ no underflow



Numerical Optimization Issues

- $SM(\mathbf{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$, $\mathbf{z} = \mathbf{x} - max_i(x_i) \Rightarrow SM(\mathbf{z}) = SM(\mathbf{x})$
- If we compute $log\left[SM(\mathbf{z})\right]$, eg cross-entrpoy loss ?
  - $SM(\mathbf{z}) = 0$ (underflow) $\Rightarrow log\left[SM(\mathbf{z})\right] \rightarrow -\infty$: Nan !
- Solution : stabilize $log\left[SM(\mathbf{x})_i\right] = x_i - log\left[\sum_{j=1}^{K} e^{x_j}\right]$
  - Same solution : $\mathbf{z} = \mathbf{x} - max_i(x_i)$

  $\Rightarrow log\left[SM(\mathbf{z})_i\right] = log\left[SM(\mathbf{x})_i\right]$?
  $\Rightarrow$ Underflow, overflow ?



Computing Derivatives

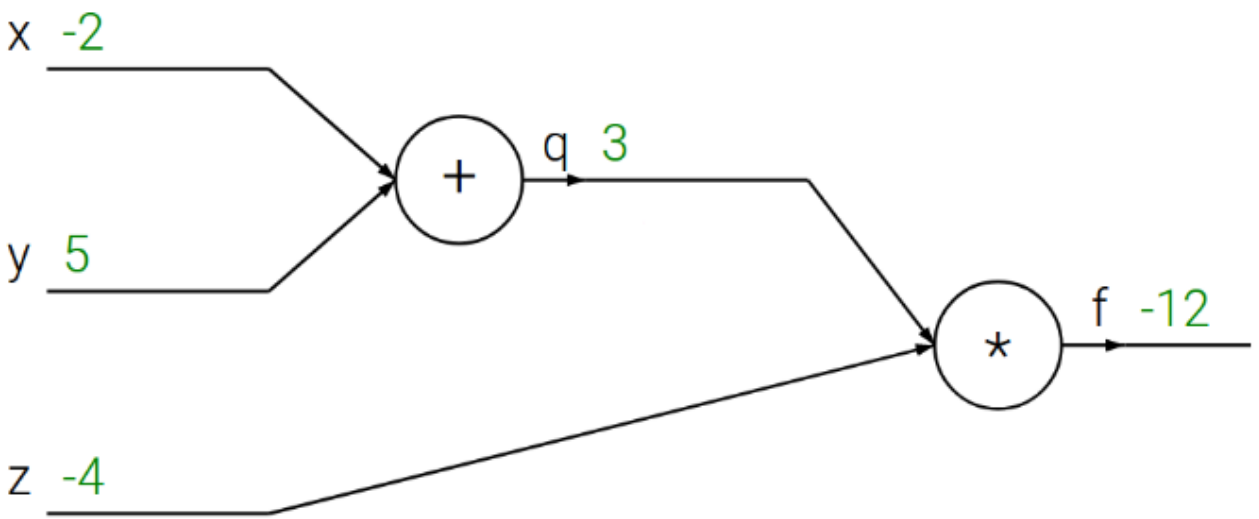
- Numerical approximation : $f'(x) \approx \frac{f(x+h)*f(x)}{h}$?
  - $\ominus$ Approximate, numerical issues (underflow/overflow)
- Symbolic differentiation ?
  - $\ominus$ Rapidly : huge expressions, many duplicated terms $\Rightarrow$ lack of efficiency

$f(x)$	$f'(x)$	$f'(x)$
$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2)+64(1-x)(1-2x)^2(1-8x+8x^2)^2-64x(1-2x)^2(1-8x+8x^2)^2-256x(1-x)(1-2x)(1-8x+8x^2)^2$	$64(1-42x+504x^2-2640x^3+7040x^4-9984x^5+7168x^6-2048x^7)$

- **Automatic differentiation**
  - **Interleave symbolic differentiation and simplification steps**
  - **Symbolic differentiation at the elementary operation level keep intermediate numerical results**

Automatic Differentiation (AD)

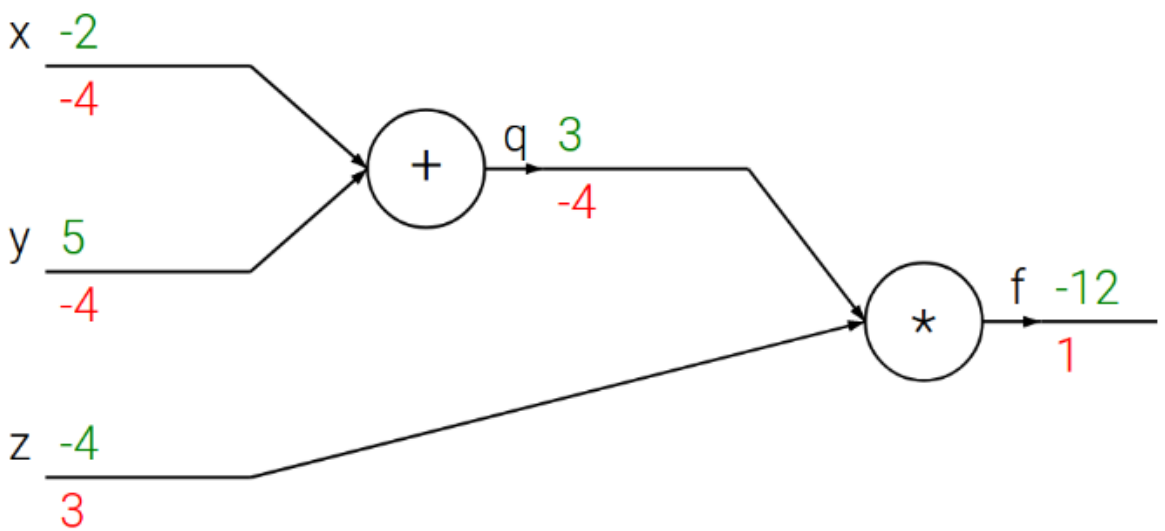
- Computation graph : core abstraction for computing gradient with backprop
- Ex[a] : $f(x, y, z) = (x + y) * z$
- Ultimate goal : compute numerical values $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$
- $x = -2$, $y = 5$, $z = -4$, forward prop $\Rightarrow$ $q = 3$, $f = -12$



a. From Stanford course : http://cs231n.github.io/optimization-2/

Computation Graph & backprop

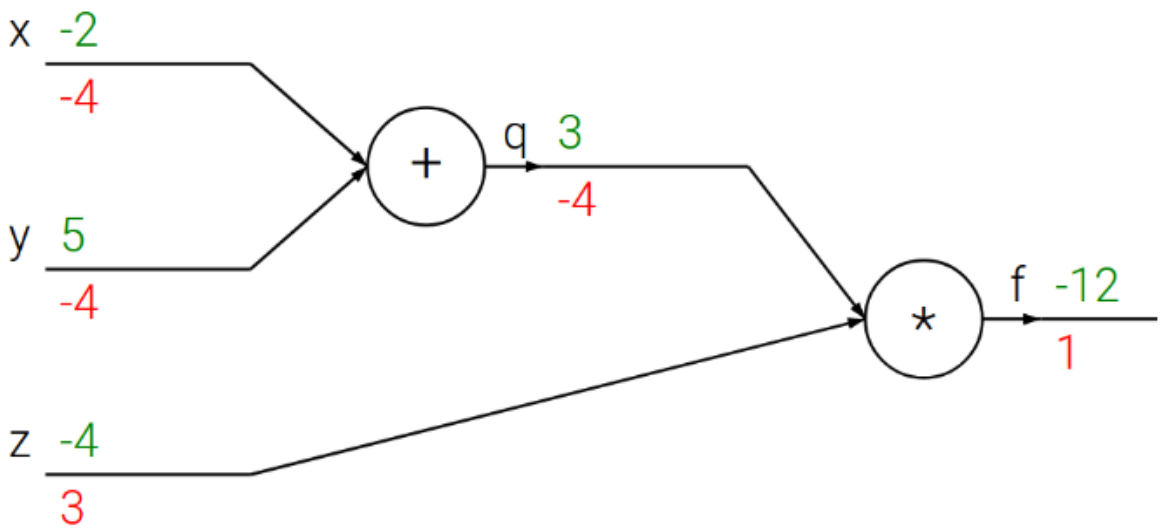
- $\frac{\partial f}{\partial f} = 1$, back-prop $\Rightarrow \frac{\partial f}{\partial q}, \frac{\partial f}{\partial z}$
- $f = q \star z \Rightarrow \frac{\partial f}{\partial q} = z = -4, \boxed{\frac{\partial f}{\partial z} = q = 3}$, back-prop $\Rightarrow \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}$
- $q = x + y \Rightarrow \boxed{\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x} = -4 \star 1 = -4} \quad \boxed{\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y} = -4 \star 1 = -4}$



Computation Graph (CG) & backprop

- Automatic differentiation with CG : Forward + backward pass
  - Backward pass : recursively compute derivatives from top → bottom
  - **Dynamic programming**, big speed-up wrt naive forward-mode differentiation
  - ex : Forward forward-mode differentiation from x : $\frac{\partial x}{\partial x} = 1$, $\frac{\partial q}{\partial x} = 1$, $\frac{\partial f}{\partial x} = -4$

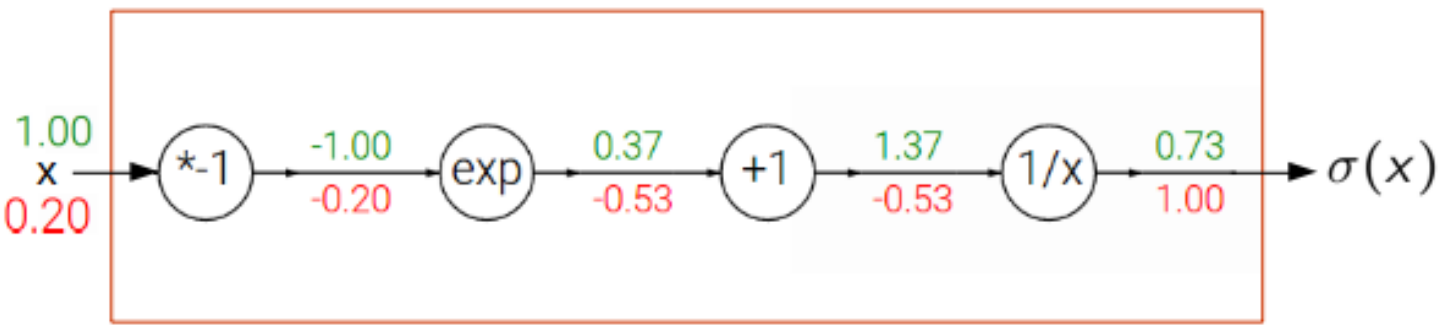
$\Rightarrow$ **N nodes : N forward passes vs 1 forward + 1 backward for backprop**



Computation Graph (CG) & backprop

- Symbolic differentiation only at atomic operation level, *e.g.* binary arithmetic operators [a], exp, log, trigonometric functions
- Include block with known derivative which is well numerically behaved
- Ex : sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$, $\sigma'(x) = \sigma(x)\left[1 - \sigma(x)\right]$

$x = 1.0 \Rightarrow \sigma(x) = 0.73$, $\sigma'(x) = 0.2$



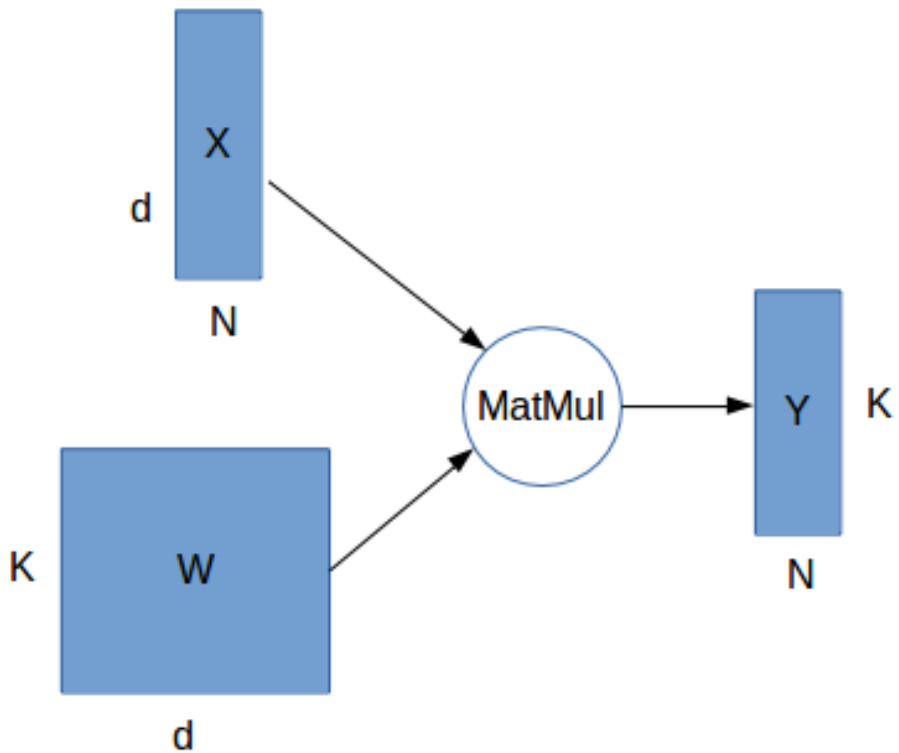
a. multiplication, addition, subtraction, division, *etc*

Deep Learning resources for the community

- Deep learning softwares / libraries features :
  - Build big computational graphs on tensors
  - Compute gradients automatically (**autodiff**)
  - GPU / TPU processing, compilation of the graph (XLA, `torch.compile`)
- **Historical** libraries (now mostly discontinued) :
  - MatConvNet (Oxford), Caffe (UC Berkeley), Torch (Lua, NYU), Theano (U Montreal)
- **Current** libraries :
  - **PyTorch** (Meta → PyTorch Foundation) : eager, object-oriented
    ⇒ de facto standard in research & industry
  - **JAX** (Google) : NumPy API + composable transformations (`grad`, `jit`, `vmap`)
    ⇒ functional, very fast on GPU / TPU, ecosystem : Flax, Optax
  - TensorFlow (Google) : still used in production, declining in research
  - Hubs of pre-trained models : `torchvision`, `timm`, Hugging Face

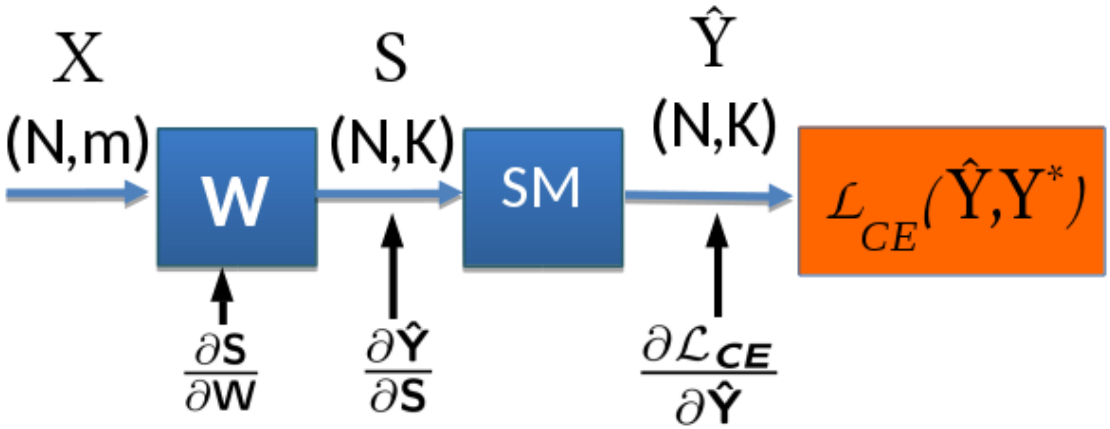
Back-prop on Tensors

- Simply compute derivative wrt flattened tensor
- **BUT :** direct chain rule application
  $\Rightarrow$ big memory & computation issues !
  Problem already with last linear layers



Back-prop on Tensors

- **Example : batch training for last linear layer**
- Data matrix $\mathbf{X}$ $(N \times m)$, label matrix $\hat{\mathbf{Y}}$, $\mathbf{Y}^*$ $(N \times K)$
- Cross-entropy loss : $\mathcal{L}_{CE}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^{N} log(\hat{y}_{c^*,i})$



$\Rightarrow \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = ?$ - Ex : $m = 4096$, $N = 100$, $K = 1000$ (ImageNet)

- $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}}$: size $K \cdot d \approx 4M$ params $\approx 32MB$ memory OK

Back-prop on Tensors

$$\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = ? \text{ - Ex : } m = 4096,\ N = 100,\ K = 1000 \text{ (ImageNet)}$$

- **Chain rule :** $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{S}} \frac{\partial \mathbf{S}}{\partial \mathbf{W}}$
  - $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{S}} = \hat{\mathbf{Y}} - \mathbf{Y}^* = \Delta$: size $K \cdot N = 100K$ params $\approx 800KB$ small
  - **BUT :** $\frac{\partial \mathbf{S}}{\partial \mathbf{W}}$ size $(K \cdot N) \cdot (K \cdot d) = 1000 \cdot 100 \cdot 1000 \cdot 4000$ $= 400G$ params $\approx 3.2TB$ huge ! !
  - $\frac{\partial \mathbf{S}}{\partial \mathbf{W}}$: **far too large to fit into memory, not explicitly computable**

$$\frac{\partial \mathbf{S}}{\partial \mathbf{W}} = \begin{pmatrix} \mathbf{x}_1 & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{x}_1 & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{x}_1 \\ \dots & \dots & \dots & \dots \\ \mathbf{x}_N & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{x}_N & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{x}_N \end{pmatrix}$$

Back-prop on Tensors

- $\frac{\partial \mathbf{S}}{\partial \mathbf{W}}$: **far too large to fit into memory, not explicitly computable**
- However, computing $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{S}} \frac{\partial \mathbf{S}}{\partial \mathbf{W}}$ is tractable !
  - For a single example : $\frac{\partial \mathbf{s}}{\partial \mathbf{W}} = \begin{pmatrix} \mathbf{x} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{x} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{x} \end{pmatrix}$, $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{s}} = \boldsymbol{\delta} = \begin{pmatrix} \delta_1 & \dots & \delta_K \end{pmatrix}$

$$\Rightarrow \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \begin{pmatrix} \delta_1 & \dots & \delta_K \end{pmatrix} \begin{pmatrix} \mathbf{x} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{x} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{x} \end{pmatrix} = \begin{pmatrix} \delta_1 \mathbf{x} & \dots & \delta_K \mathbf{x} \end{pmatrix}$$

  - With matrix $m \times K$ reshaping :

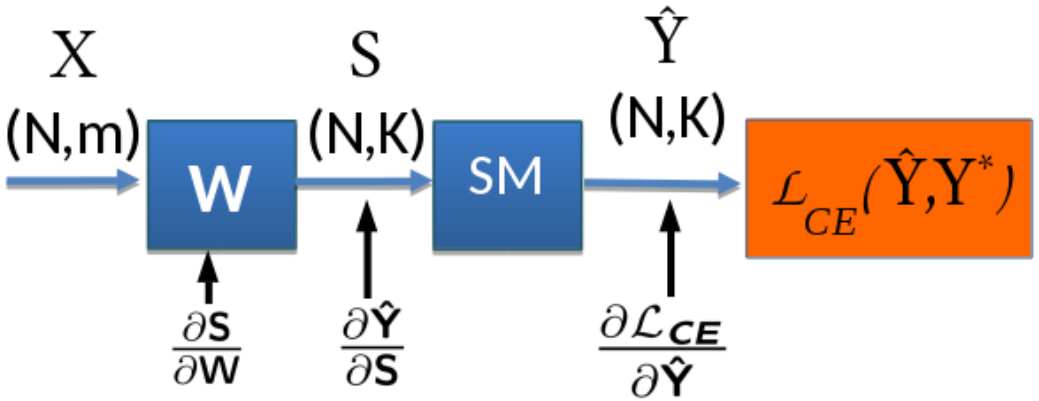
$$\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \begin{pmatrix} x_1 \\ x_m \\ \dots \\ x_m \end{pmatrix} \begin{pmatrix} \delta_1 & \delta_2 & \dots & \delta_K \end{pmatrix} = \mathbf{x}^T \delta$$

Back-prop on Tensors

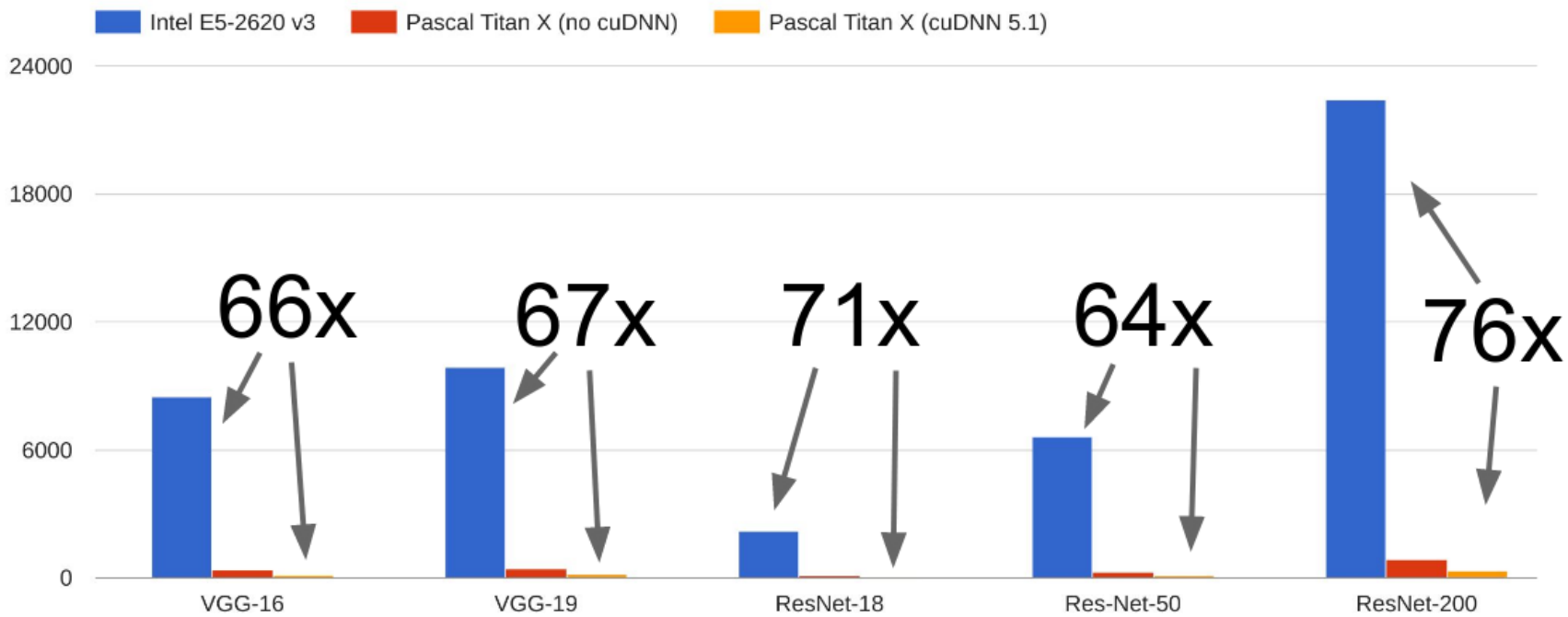
- $\frac{\partial \mathbf{s}}{\partial \mathbf{W}}$ intractable but $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{s}} \frac{\partial \mathbf{s}}{\partial \mathbf{W}}$ tractable
  - **Solution :** first project $\mathbf{s}$ on δ :
    - $s_p = \mathbf{s}\ \delta^T = \sum_{k=1}^{K} s_k \delta_k = \sum_{k=1}^{K} \sum_{j=1}^{m} w_{jk} x_k \delta_k$
    - Compute gradient on $s_p \in \mathbb{R}$:

$$\frac{\partial s_p}{\partial \mathbf{W}} = \begin{pmatrix} \delta_1 \mathbf{x} & \delta_2 \mathbf{x} & \dots & \delta_K \mathbf{x} \end{pmatrix} = \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}}$$

  - **Each layer should be able to compute** $\frac{\partial \mathbf{S}_p}{\partial \mathbf{W}}$
    - $\frac{\partial \mathbf{S}_p}{\partial \mathbf{W}} = \frac{\partial \mathbf{S} \Delta^T}{\partial \mathbf{W}}$ tractable $(\neq \frac{\partial \mathbf{S}}{\partial \mathbf{W}})$



Computation on GPU



- Training deep ConvNets : huge speed-up with Graphical Processing Units (GPU) [1]
  - Especially convolution

1. data from https://github.com/jcjohnson/cnn-benchmarks

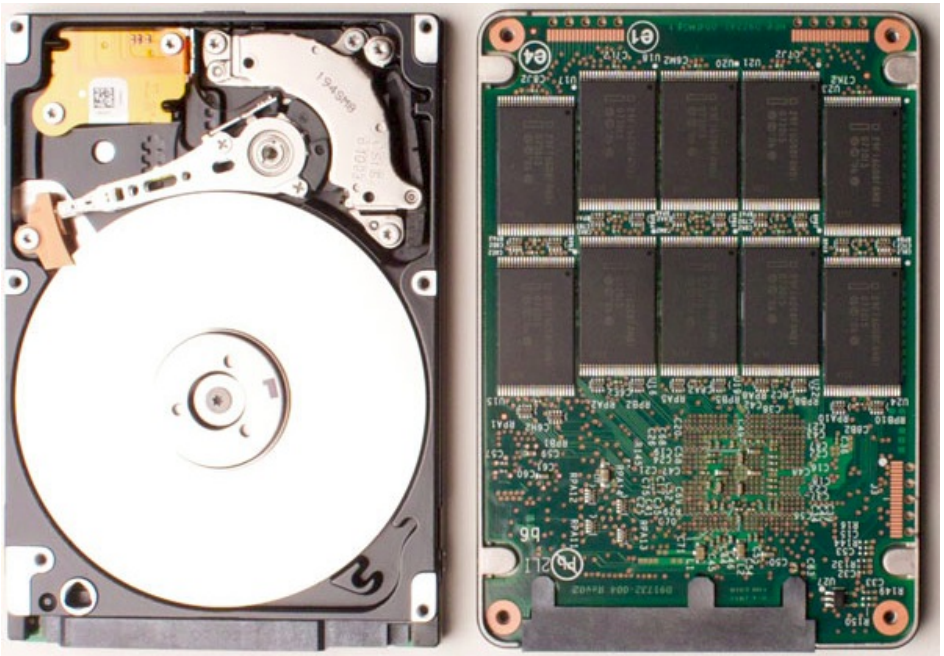
Computation on GPU

- Solution for GPU programming :
  - CUDA (NVIDIA) :
    - C-code
    - Higher-level APIs : cuBLAS, cuFFT, cuDNN, etc
  - ROCm / HIP (AMD), Metal / MPS (Apple), TPU (Google, via XLA)
- **Deep Learning libraries : transparent GPU computing**



Computation on GPU

- Big data : transfer from disk to memory $\Rightarrow$ bottleneck
- Solution : use SSD instead of HDD

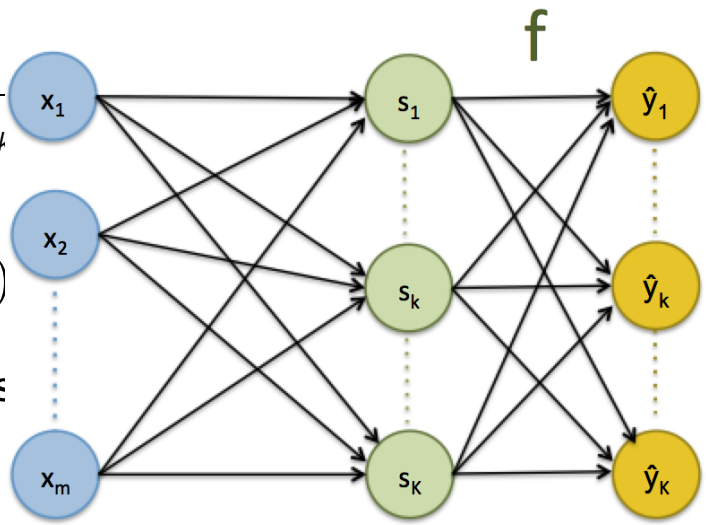


Deep Learning resources : PyTorch & JAX

- Very efficient implementation for computation graphs on multi-dimensional tensors
- Automatic differentiation, GPU support
- **Do not re-invent the wheel : use them !**
- **Libraries used in this course : PyTorch and JAX**
  - PyTorch : `pip install torch torchvision` – `https://pytorch.org/`
  - JAX : `pip install jax flax optax` – `https://docs.jax.dev/`
- Same concepts, two philosophies :
  - PyTorch : **stateful** modules (`nn.Module`), gradients stored in tensors (`.grad`)
  - JAX : **pure functions**, parameters passed explicitly, gradients are functions (`jax.grad`)

Logistic Regression for Classification

- Simple example : Logistic Regression (LR)
- $\mathbf{x}_i$ vector, $\mathbf{s}_i = \mathbf{x}_i\mathbf{W} + \mathbf{b}$
- Soft-Max (SM) : $\hat{\mathbf{y}}_k \sim P(k/\mathbf{x_i}, \mathbf{W}, \mathbf{b}) = \frac{e^{\mathbf{s}_k}}{\sum_{k'=1}^{K} e^{\mathbf{s}_{k'}}}$
- Training with cross-entropy : $\frac{1}{N}\sum_{i=1}^{N} \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*)$
- Example on MNIST database : $K = 10$ classes
- Input : flattened images : $d = 28^2 = 784$
- # parameters : $784 * 10 + 10 = 7850$



PyTorch : Logistic Regression

- Load MNIST data with `torchvision`, batches with a `DataLoader`

```
import torch, torch.nn as nn
from torchvision import datasets, transforms
tf = transforms.Compose([transforms.ToTensor(), transforms.Lambda(torch.flatten)])
train = datasets.MNIST("data", train=True, download=True, transform=tf)
loader = torch.utils.data.DataLoader(train, batch_size=128, shuffle=True)
```

- Define the model (softmax is included in the loss) and the optimizer

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = nn.Linear(784, 10).to(device)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.SGD(model.parameters(), lr=0.5)
```

- Training loop : forward, `backward()` (back-prop), update

```
for epoch in range(20):
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        loss = loss_fn(model(x), y)
        opt.zero_grad()
        loss.backward()
        opt.step()
```

JAX : Logistic Regression

- Parameters are explicit (a *pytree*), the model is a **pure function**

```
import jax, jax.numpy as jnp
key = jax.random.key(0)
params = {"W": 0.01 * jax.random.normal(key, (784, 10)), "b": jnp.zeros(10)}

def loss_fn(params, x, y):
    s = x @ params["W"] + params["b"]
    logp = jax.nn.log_softmax(s)
    return -jnp.mean(logp[jnp.arange(len(y)), y])
```

- `jax.grad` : returns the **gradient function** ; `jax.jit` : compiles with XLA

```
@jax.jit
def sgd_step(params, x, y, lr=0.5):
    grads = jax.grad(loss_fn)(params, x, y)
    return jax.tree.map(lambda p, g: p - lr * g, params, grads)

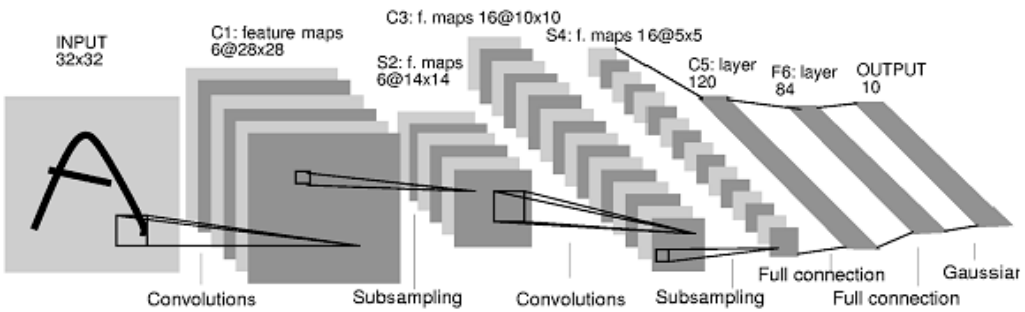
for epoch in range(20):
    for x, y in batches(X_train, y_train, 128):    # numpy arrays
        params = sgd_step(params, x, y)
```

- In practice : **Flax** for layers (`nnx.Linear`, `nnx.Conv`), **Optax** for optimizers

PyTorch : more complex models

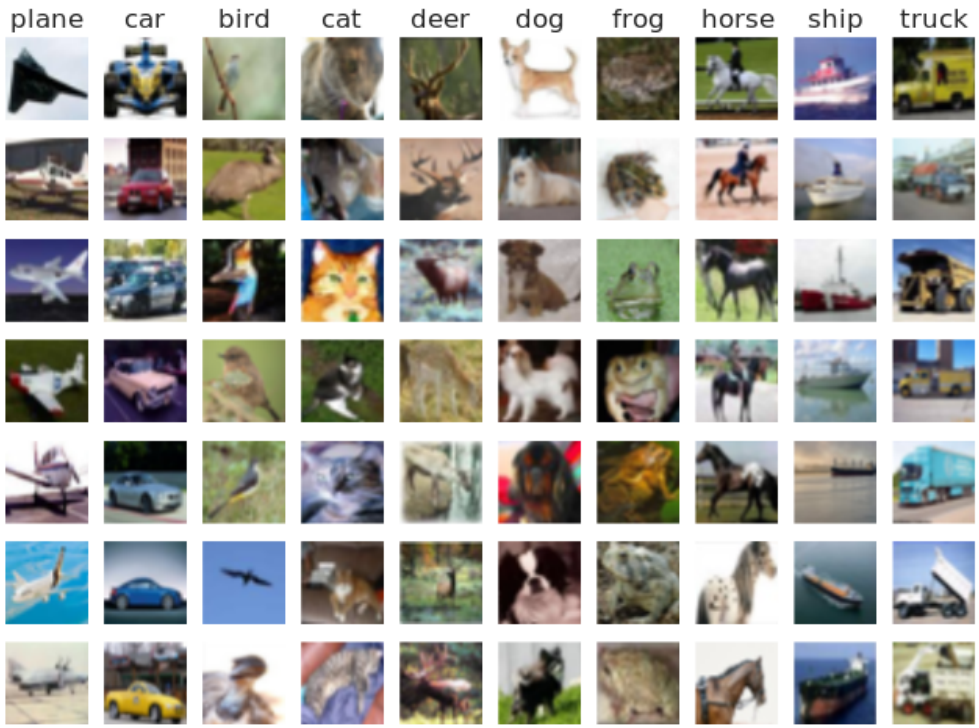
- Design more complex models by stacking layers :
  - Fully connected, convolution, non-linearity, pooling, *etc*
  - Class for convolution on images : `nn.Conv2d` (tensors in $N \times C \times H \times W$)
- Code for training remains unchanged (autodiff does the job)

```
model = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=5), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32, 64, kernel_size=5), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(64 * 4 * 4, 100), nn.ReLU(),
    nn.Linear(100, nb_classes),
)
```



PyTorch & JAX : ecosystem

- Many other layers implemented, *e.g.* dropout, normalization, attention
- Various optimizers implemented, *e.g.* Nesterov, Adam, AdamW (`torch.optim`, `optax`)
- Various loss functions, *e.g.* hinge loss, regression, KL divergence
- Various datasets and pre-trained models, *e.g.* CIFAR, ImageNet (`torchvision`, `timm`, Hugging Face)



PyTorch & JAX : Conclusion

- **PyTorch** : intuitive, Pythonic, eager execution
  - Easy debugging (`print`, `pdb`), huge community and model zoo
  - `torch.compile` for speed, `CUDA_VISIBLE_DEVICES=0` to select a GPU
- **JAX** : functional, close to the maths
  - `grad`, `jit`, `vmap`, `pmap`/`shard_map` : composable transformations
  - Excellent performance on GPU / TPU, popular for large-scale & scientific ML
  - Beware : pure functions, explicit random keys, immutable arrays
- Monitoring : TensorBoard, Weights & Biases
- Try them : `https://pytorch.org/` – `https://docs.jax.dev/`

Outline

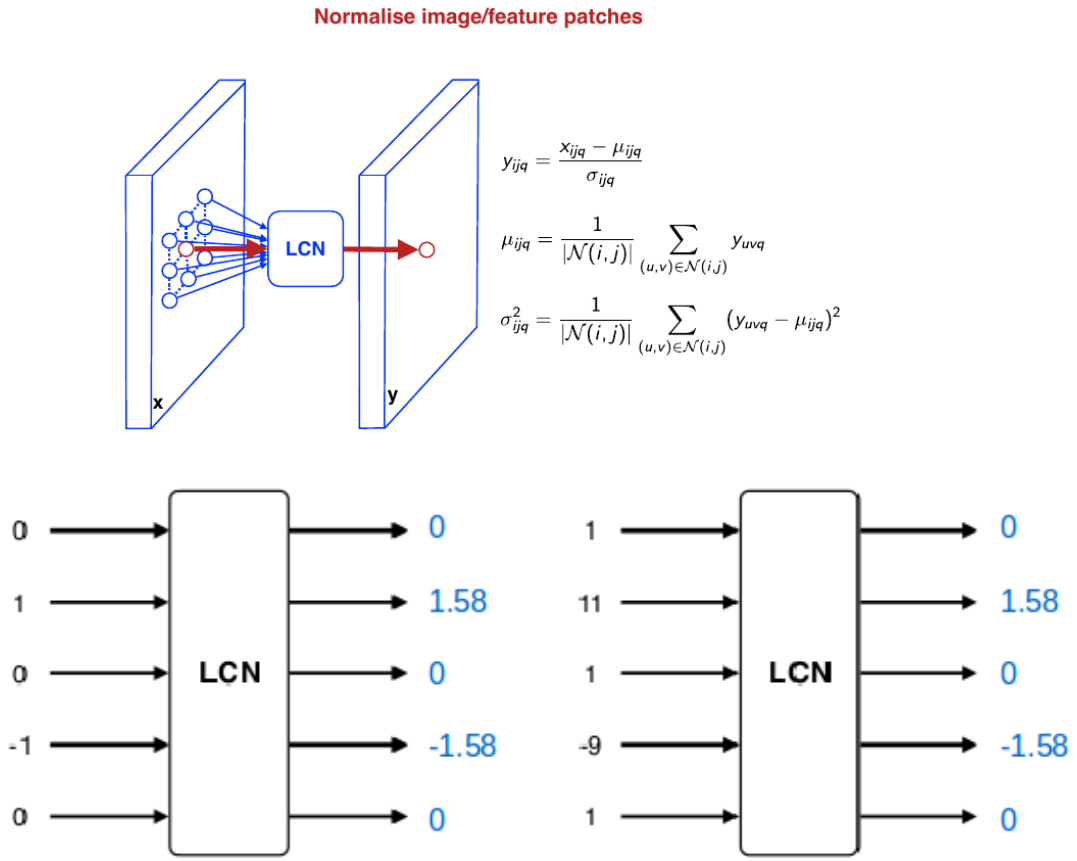
1. Convolutional Neural Networks
2. Modern Deep Architectures
3. Implementation
   - Modern Architectural Components
   - Modern Macro Architectures
4. Transfer Learning

Outline

1. Convolutional Neural Networks
2. Modern Deep Architectures
3. Implementation
   - Modern Architectural Components
   - Modern Macro Architectures
4. Transfer Learning

Local Response/Contrast Normalization

- **Normalize value wrt spatial neighbors** $\mathcal{N}(i,j)$



Credit : A. Vedaldi

- Local equalizing effective
- Helps learning more invariant representations
  $\Rightarrow$ regularization, better generalization

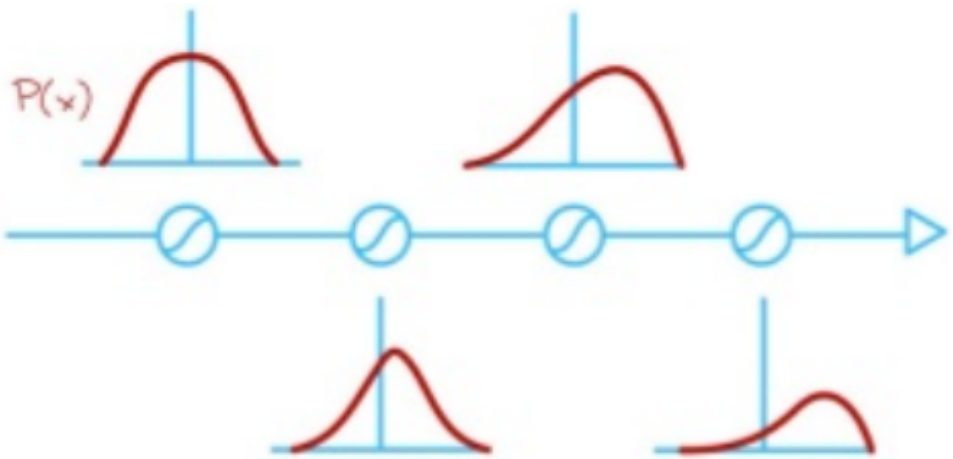
Normalization : Local Feature Normalization

- **Normalize value wrt neighbors in different feature maps**
- Operates at each spatial position independently
- Feature groups : subset of maps (sliding window)
- $\sim$ Lateral inhibition



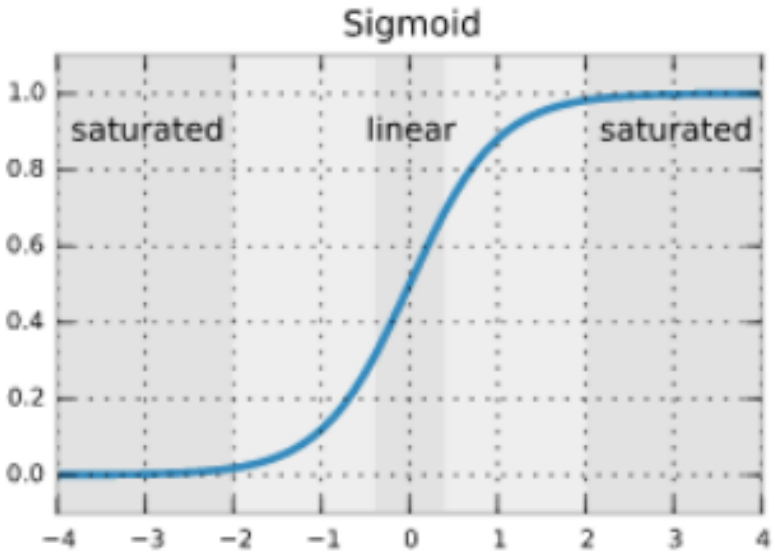
Batch Normalization (BN) [Ioffe and Szegedy, 2015]

- **Recap init : fixed input distribution known to help training**
- Training deep neural networks : distribution of hidden layers unknown, change over training time $\Rightarrow$ **covariate shift**
- Importance of init, *e.g.* Xavier
- **Batch Normalization (BN) :**
  $\downarrow$ importance of init, $\downarrow$ covariate shift



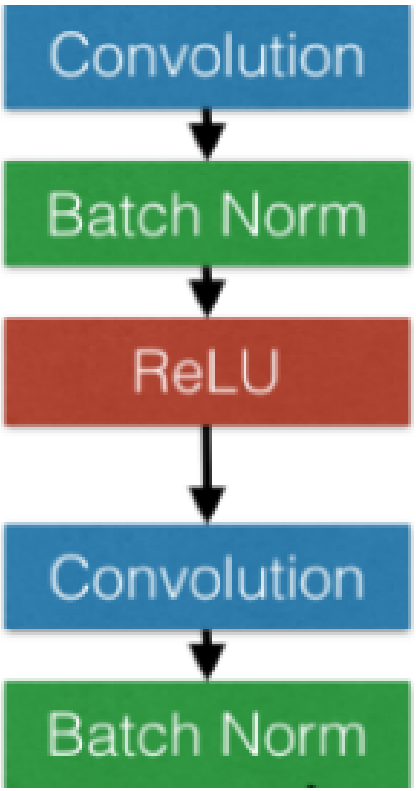
Batch Normalization (BN) [Ioffe and Szegedy, 2015]

- **Normalize input feature distribution** $\sim \mathcal{N}(0,1)$
  - Normalization across each mini-batch :
  - $\mu_B = \frac{1}{N} \sum_{i=1}^{N} x_i$
  - $\sigma_B^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu_B)^2$
  - $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ - ϵ for numerical stability
- **Is input feature distribution** $\sim \mathcal{N}(0,1)$ **good idea ?**
  - Activation may not ever "saturate", *e.g.* sigmoid or tanh
  - Keeping in linear regime : depth useless, $\sim$ global linear model



Batch Normalization (BN) [Ioffe and Szegedy, 2015]

- Scale and shift : $y_i = \gamma \hat{x}_i + \beta$, (γ, β) trained
- Apply after FC / conv and before non-linearity

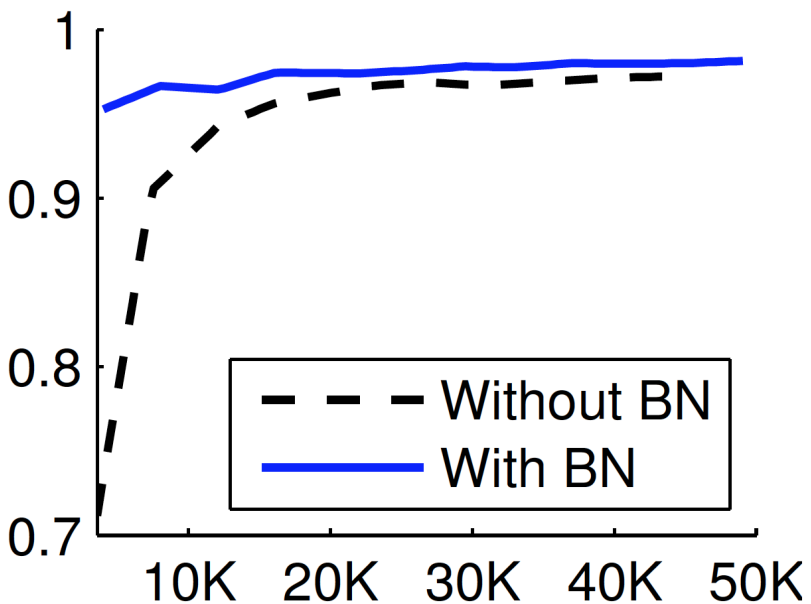


$$\frac{\partial \ell}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i} \cdot \gamma$$
$$\frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} = \sum_{i=1}^{m} \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_{\mathcal{B}}) \cdot \frac{-1}{2} (\sigma_{\mathcal{B}}^2 + \epsilon)^{-3/2}$$
$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \left(\sum_{i=1}^{m} \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{\sum_{i=1}^{m} -2(x_i - \mu_{\mathcal{B}})}{m}$$
$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{m}$$
$$\frac{\partial \ell}{\partial \gamma} = \sum_{i=1}^{m} \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i$$
$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^{m} \frac{\partial \ell}{\partial y_i}$$

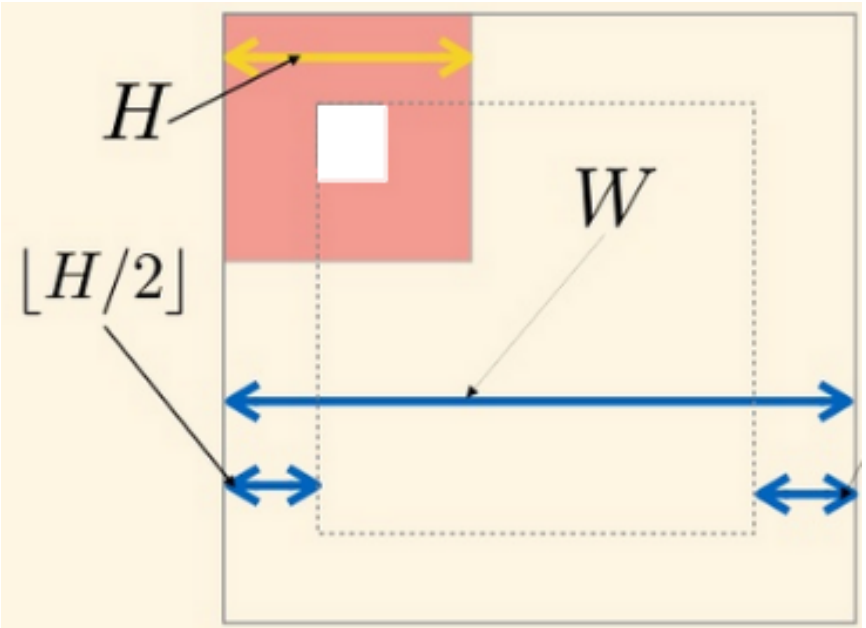
- Batch Normalization differentiable

Batch Normalization (BN) [Ioffe and Szegedy, 2015]

- **Applying BN at test time ?** $\Rightarrow$ Use train set statistics
- **BN Strengths**
  - **Faster training convergence vs covariate shift**
  - **Regularization & generalization**
    $\Rightarrow$ **better performances**
- **BN Conclusion :** especially important for very deep models, *e.g.* ResNet (see next)



Padding Modules

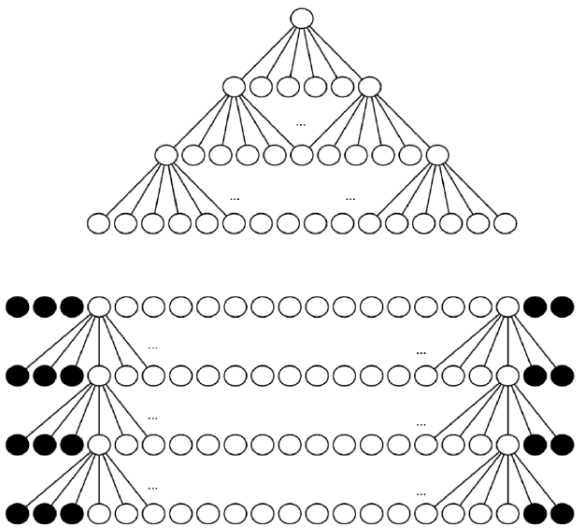


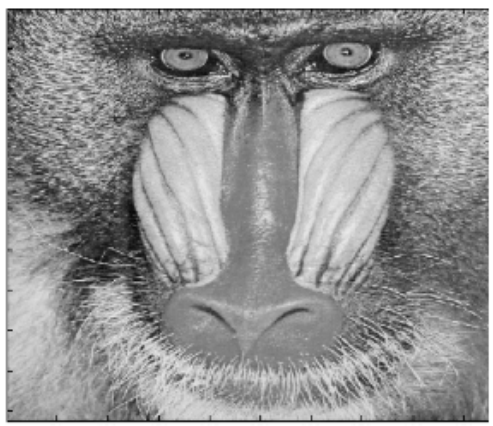
- Filter of size H : problem with the borders $\lfloor \frac{H}{2} \rfloor$
- **Solution 1** : reduce processing to computable area $\Rightarrow$ decreased output size
- **Solution 2** : padding, *i.e.* fill missing info with (arbitrary) values
  - Zero-padding, recopy, mirror, *etc*

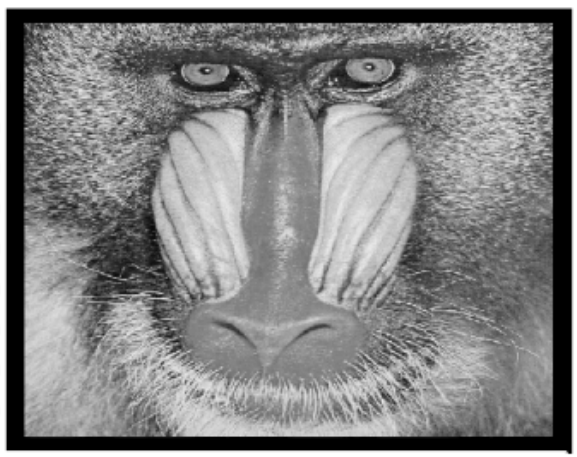
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	2	5	6	3	6	7	3	0	0
0	0	2	3	4	6	7	5	1	8	4	0	0
0	0	8	7	6	5	7	6	3	3	4	0	0
0	0	2	3	5	6	7	8	2	7	3	0	0
0	0	4	5	3	2	1	6	8	7	2	0	0
0	0	1	4	5	3	2	6	7	8	1	0	0
0	0	2	3	4	5	6	8	9	2	1	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0

2	2	2	3	4	6	7	5	1	8	4	4	4
1	1	1	1	2	5	6	3	6	7	3	3	3
1	1	1	1	2	5	6	3	6	7	3	3	7
3	2	2	3	4	6	7	5	1	8	4	4	8
7	8	8	7	6	5	7	6	3	3	4	4	3
3	2	2	3	5	6	7	8	2	7	3	3	7
5	4	4	5	3	2	1	6	8	7	2	2	7
4	1	1	4	5	3	2	6	7	8	1	1	8
3	2	2	3	4	5	6	8	9	2	1	1	2
3	2	2	3	4	5	6	8	9	2	1	1	2
3	1	1	4	5	3	2	6	7	8	1	1	2

Zero-Padding



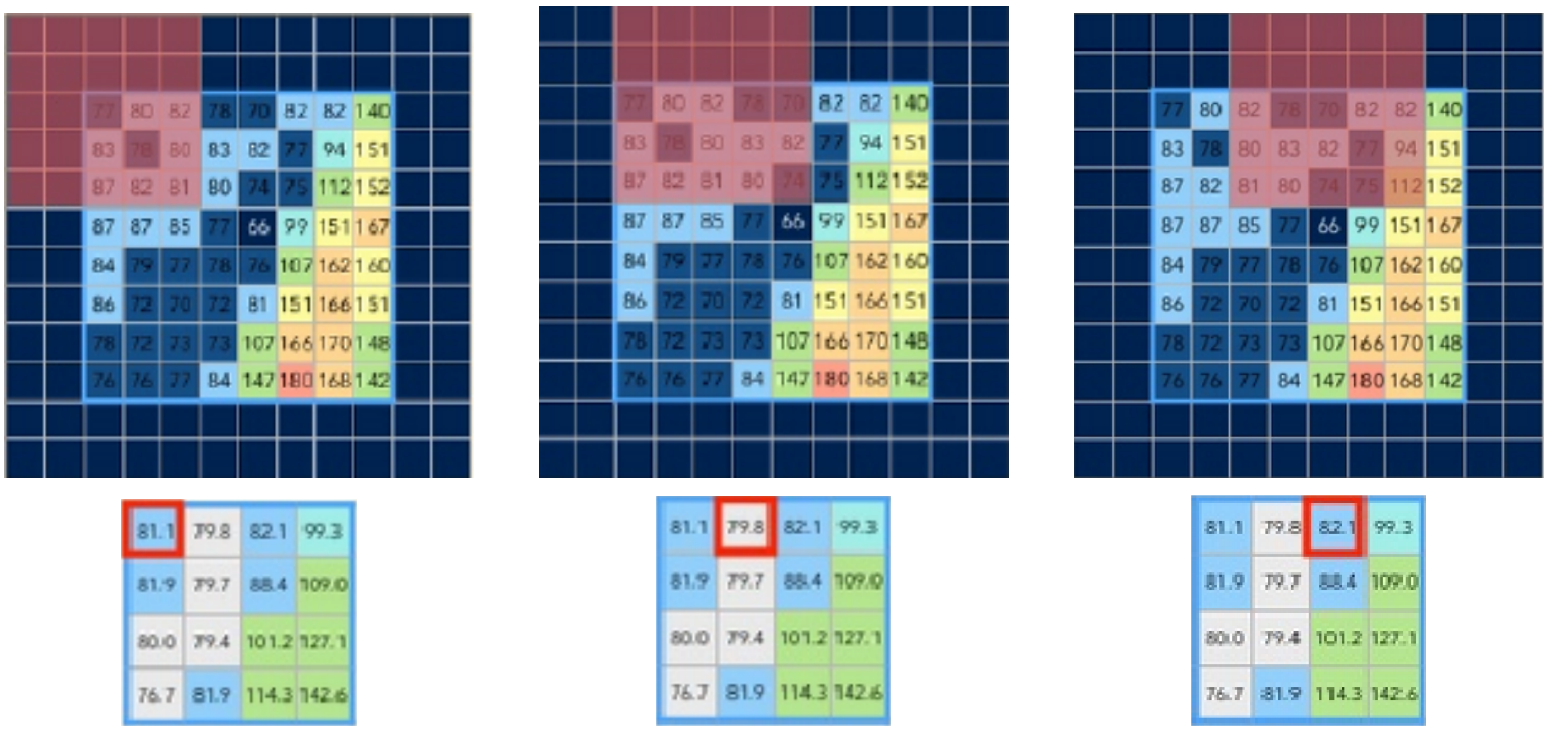




- To avoid shrinking the spatial extent of the network rapidly

Pooling Modules

Overlapping Pooling - Ex : pooling size : 5, stride s=2



Credit : K. Matsui

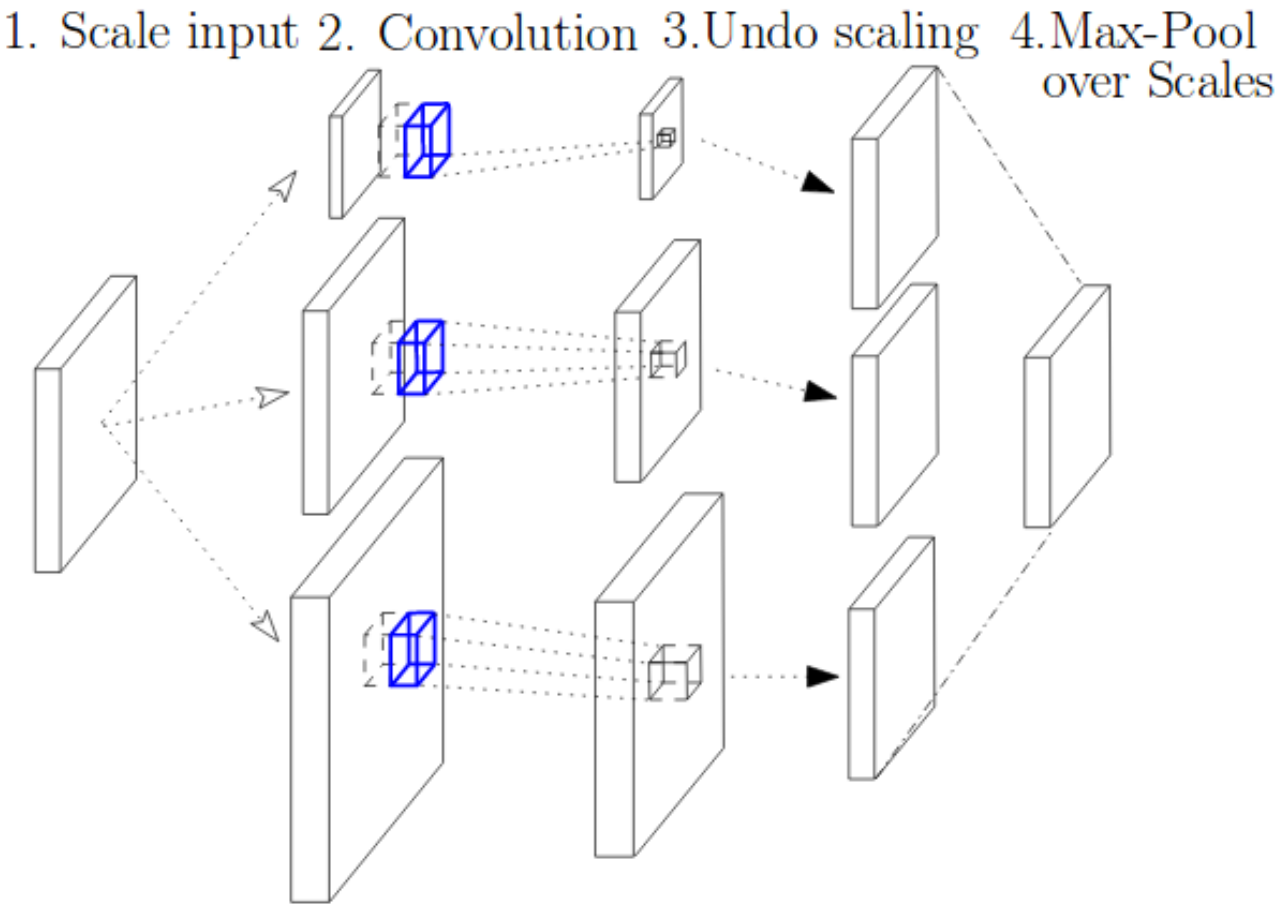
Pooling Modules

Pooling across feature maps

- Aggregation for a given spatial position, between different tensor maps
- Tensor maps (filter output) associated to a given transformation $\Rightarrow$ invariance with max pooling

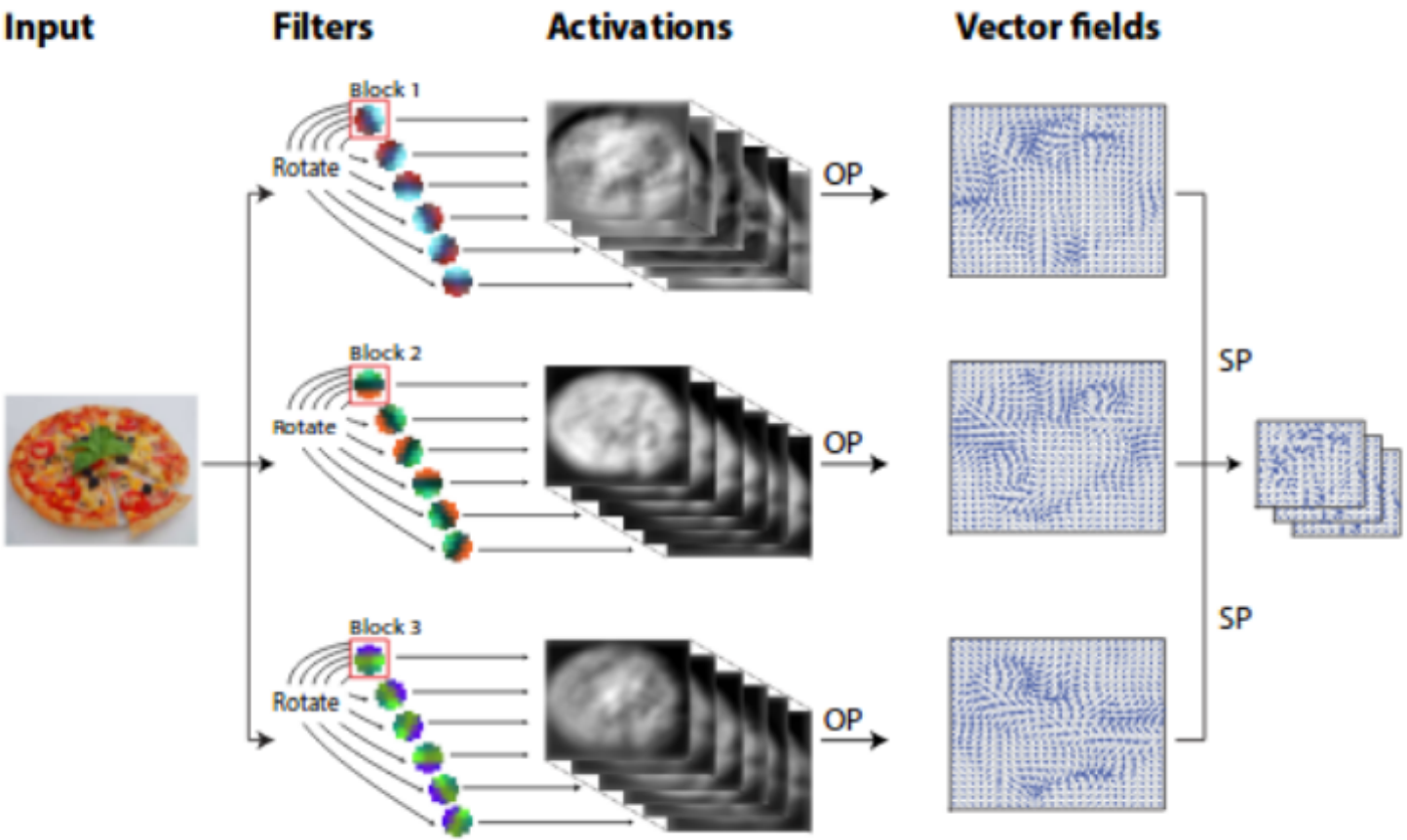
Pooling across feature maps

- Ex : scaling [Kanazawa et al., 2014]

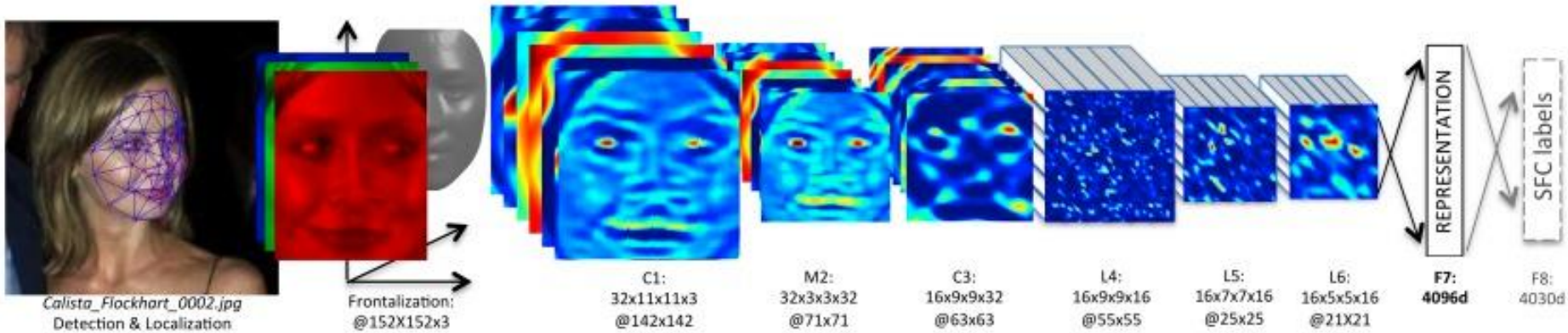


Pooling across feature maps

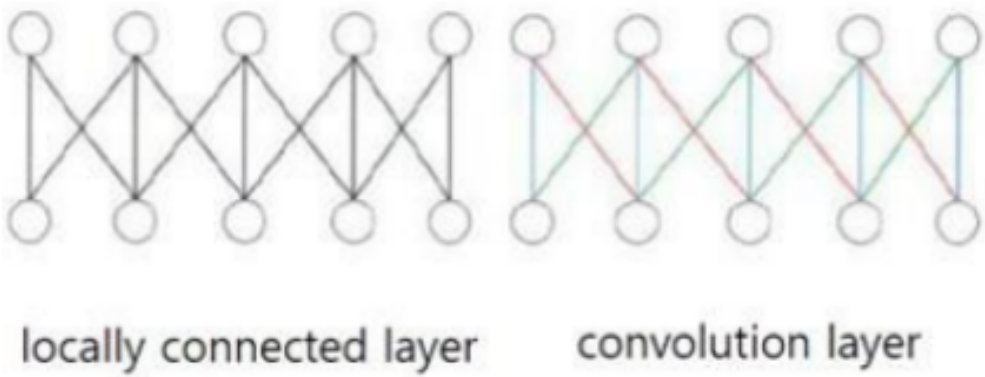
- Ex : rotation [Marcos et al., 2017]



Locally Connected *vs* Convolution Layers



- Locally connected : different features detected across image positions
- Sucessful in specific context, *e.g.* DeepFace [Taigman et al., 2014]

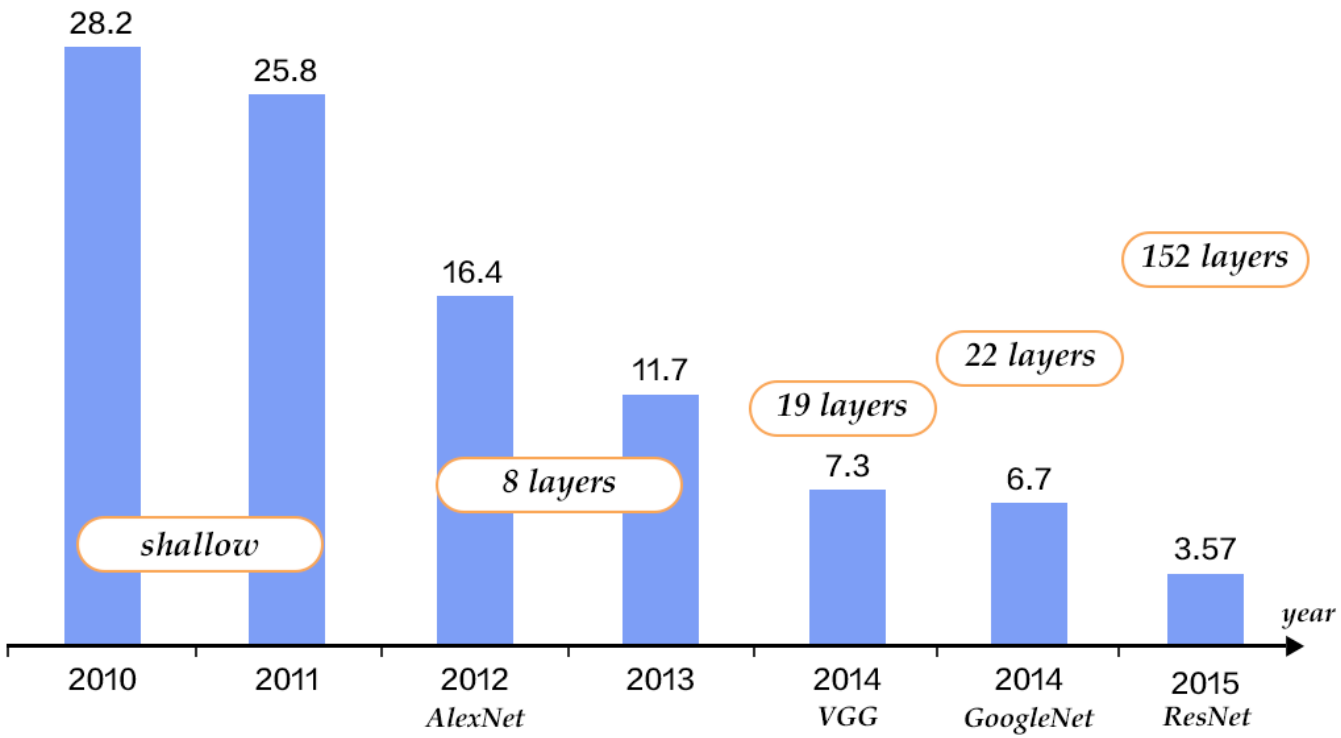


Outline

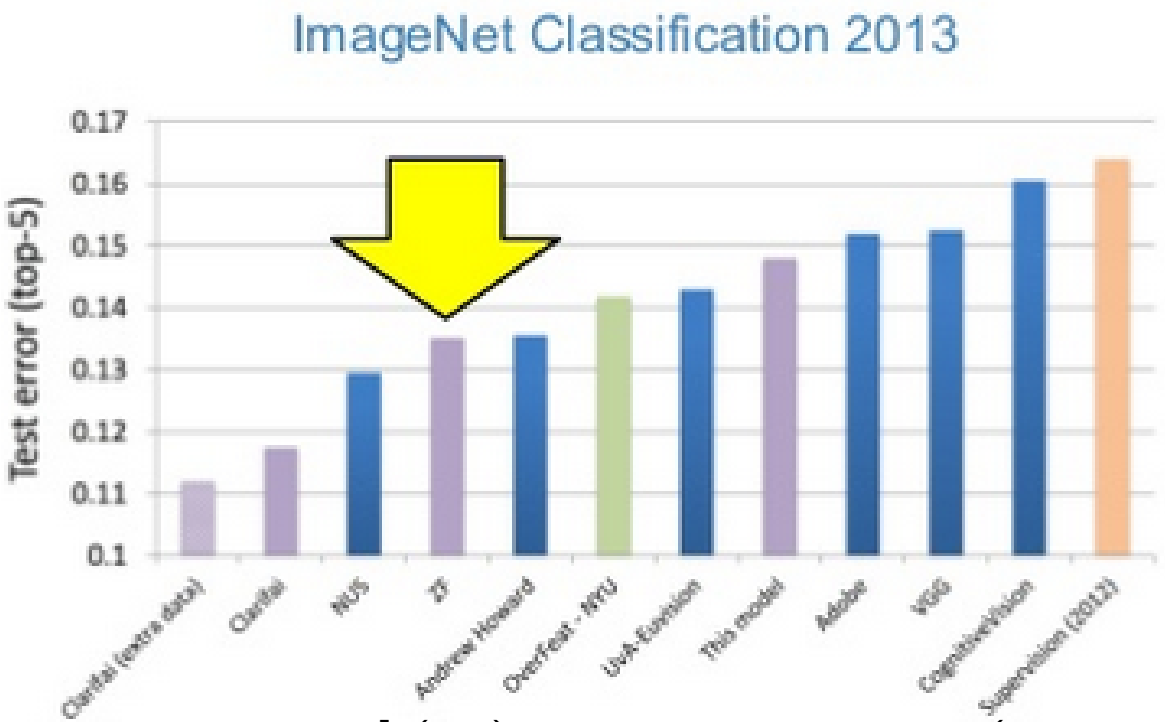
1. Convolutional Neural Networks
2. Modern Deep Architectures
3. Implementation
   - Modern Architectural Components
   - Modern Macro Architectures
4. Transfer Learning

Deep Learning since 2012

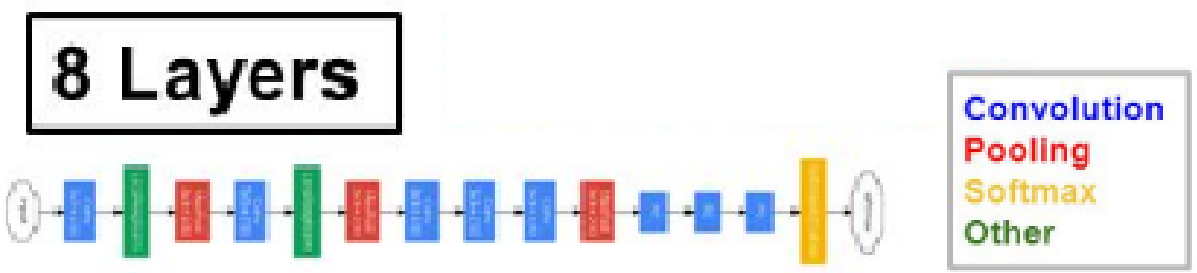
More & more data (Facebook 10^9 images / day)
ILSVRC since 2012 : larger & larger networks



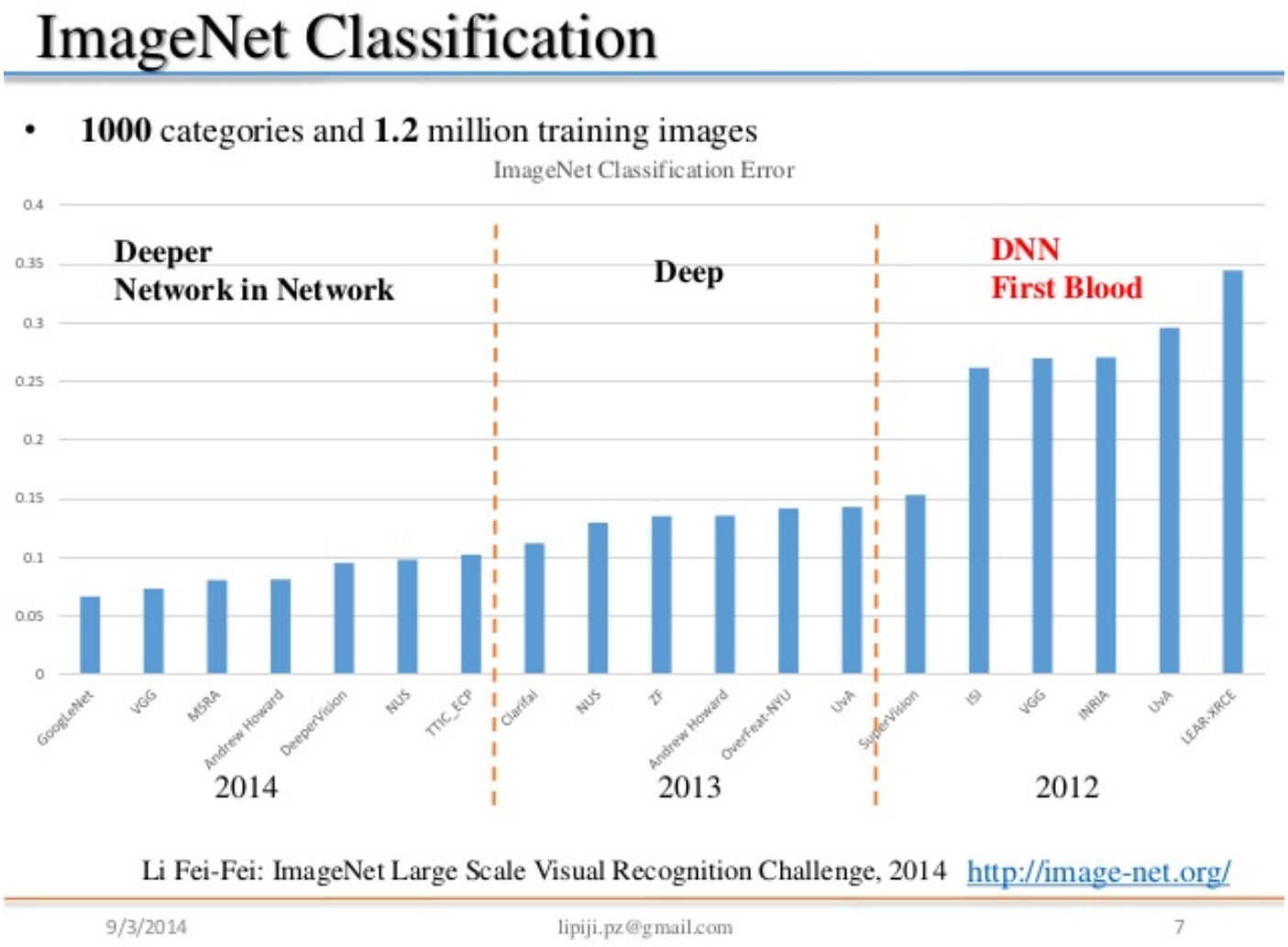
Deep Learning since 2012 : ImageNet'13



- [Zeiler and Fergus, 2014] (ZF) : archi $\sim$ AlexNet'12 (conv+FC)
- With Local Contrast Normalization (LCN) after conv/pool

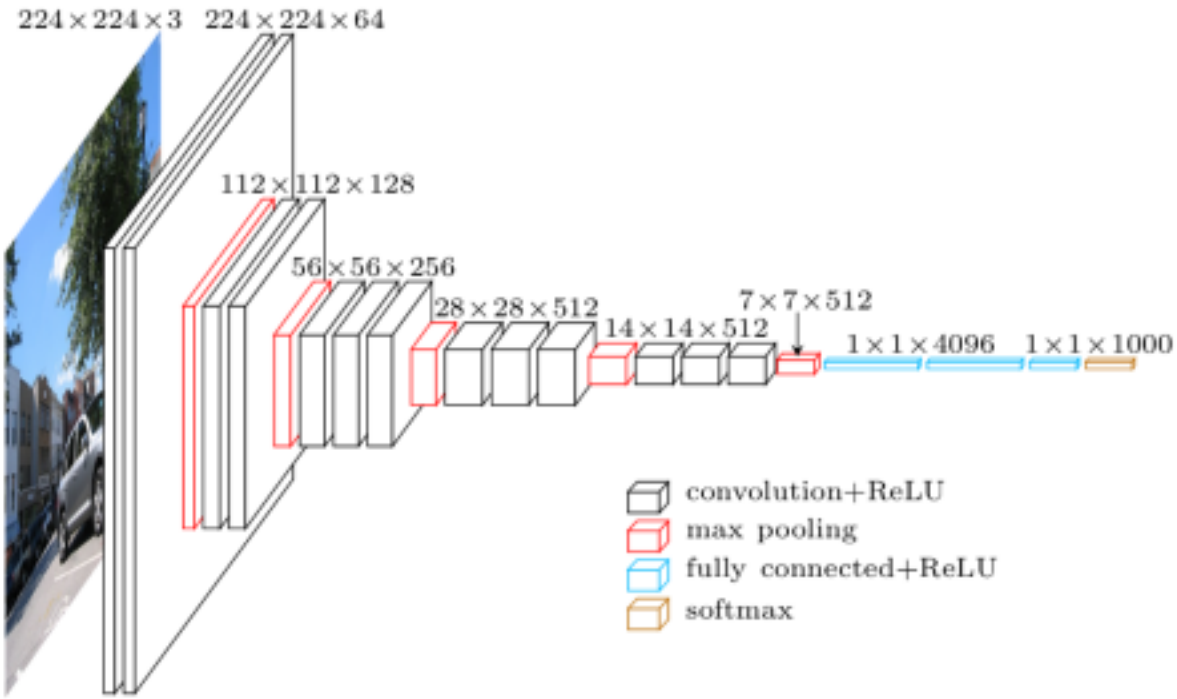


Deep Learning since 2012 : ImageNet'14



- **Two leading nets : VGG & GoogLeNet**

ImageNet'14 : VGG [Simonyan and Zisserman, 2014]



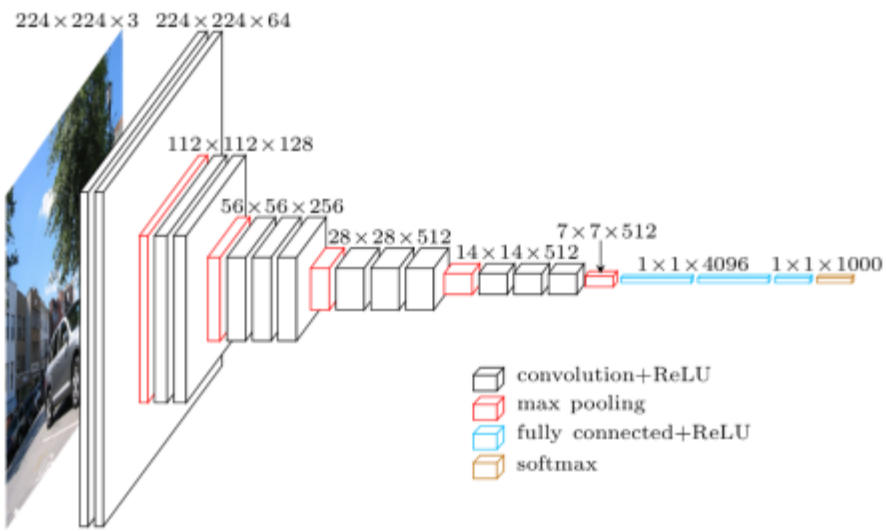


Still Conv + FC, BUT :

- No pooling between some conv layer
- Convolution stride 1

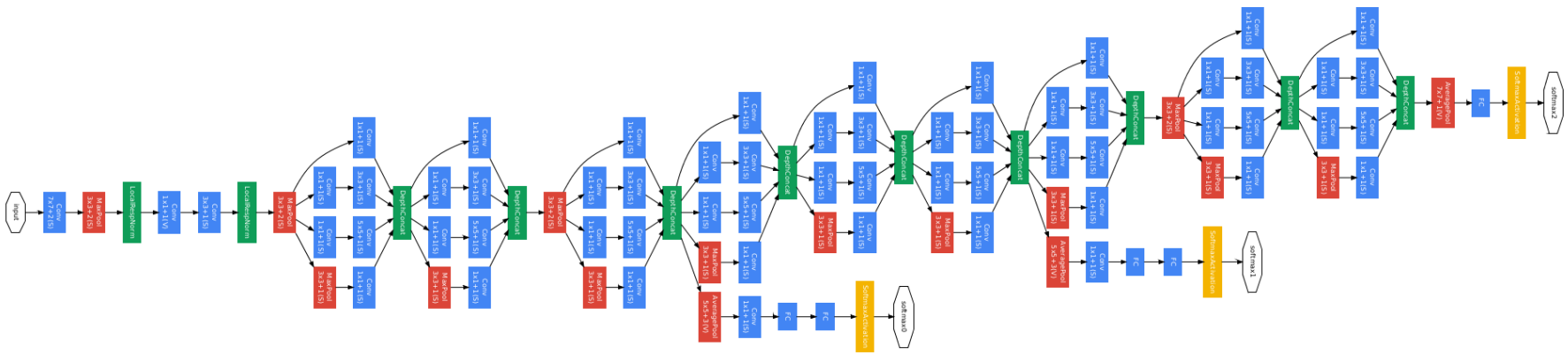
ImageNet'14 : VGG [Simonyan and Zisserman, 2014]





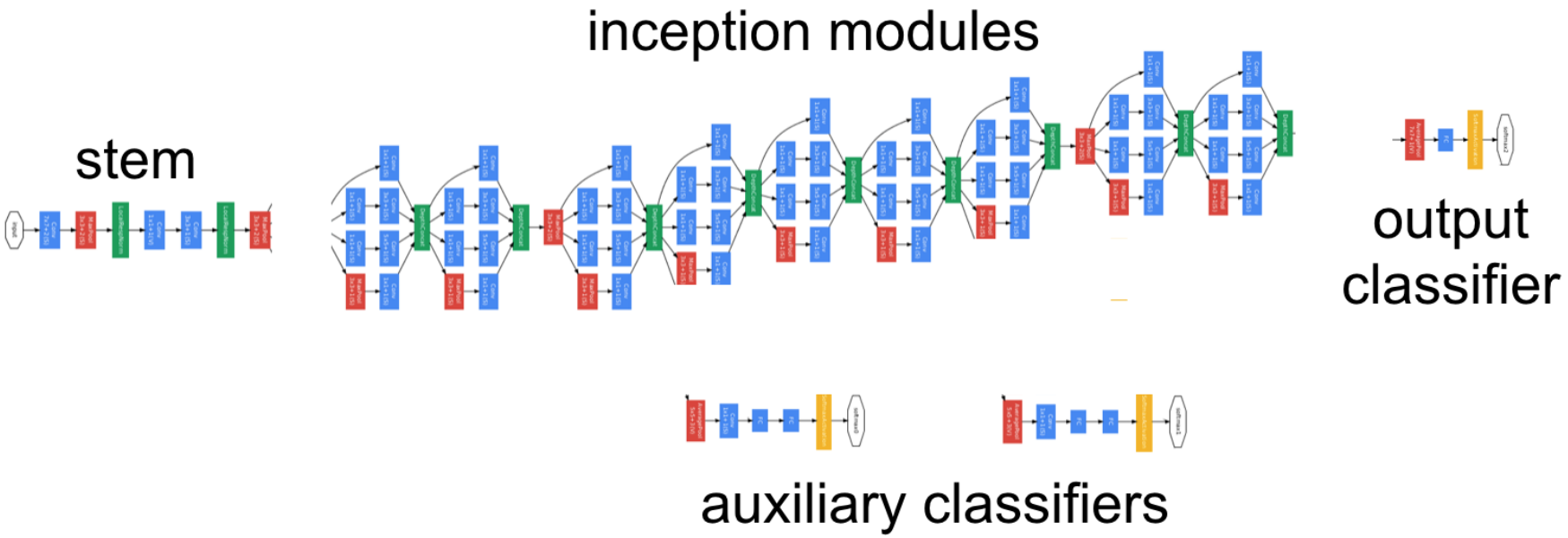
- **3x3 convolulutions :**
  two 3x3 conv $\sim$ one 5x5 conv

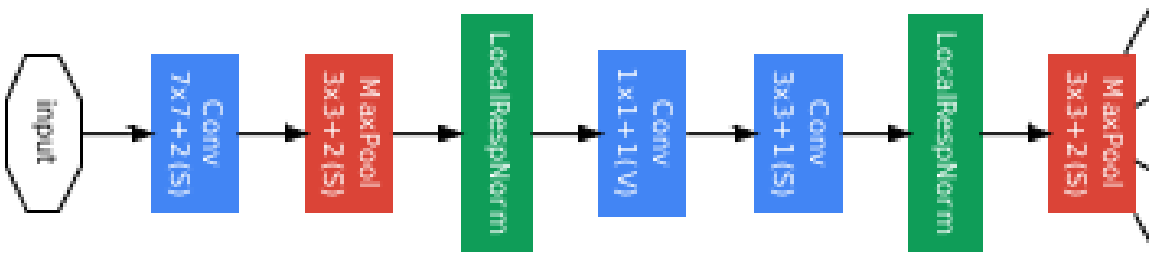
ImageNet'14 : GoogLeNet [Szegedy et al., 2015]



- GoogLeNet global archi : three main components
  1. 'Stem' $\sim$ [Conv-Pool] Layer
  2. **Inception modules : Networks in Networks**
  3. **Auxiliary classifiers**

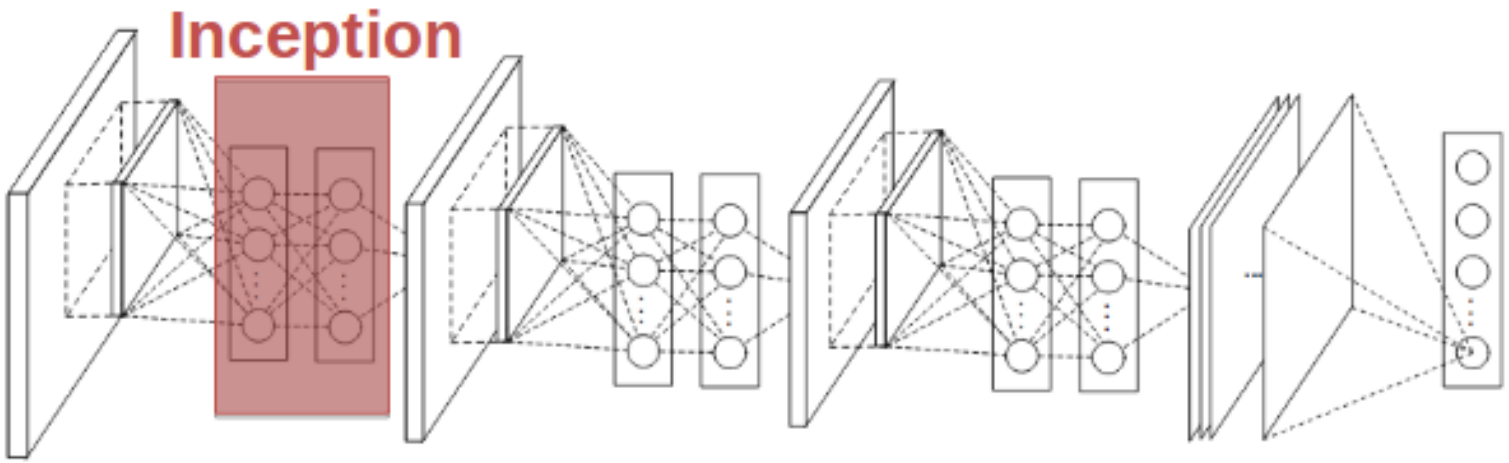
GoogLeNet : 'Stem' ∼ [Conv-Pool] Layers





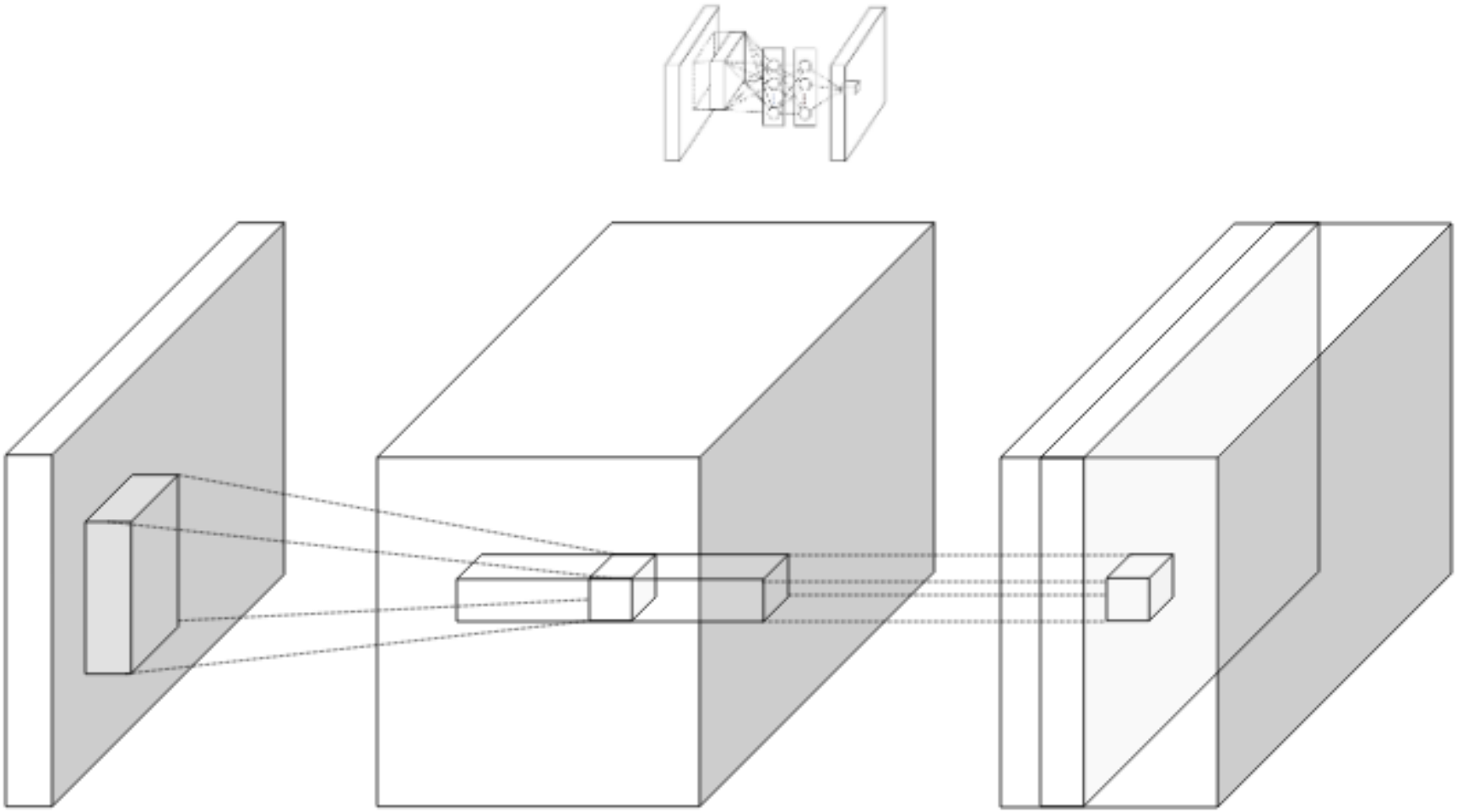
GoogLeNet : Inception Module

- Inspired from Network in Network (NiN) idea [Lin et al., 2013]

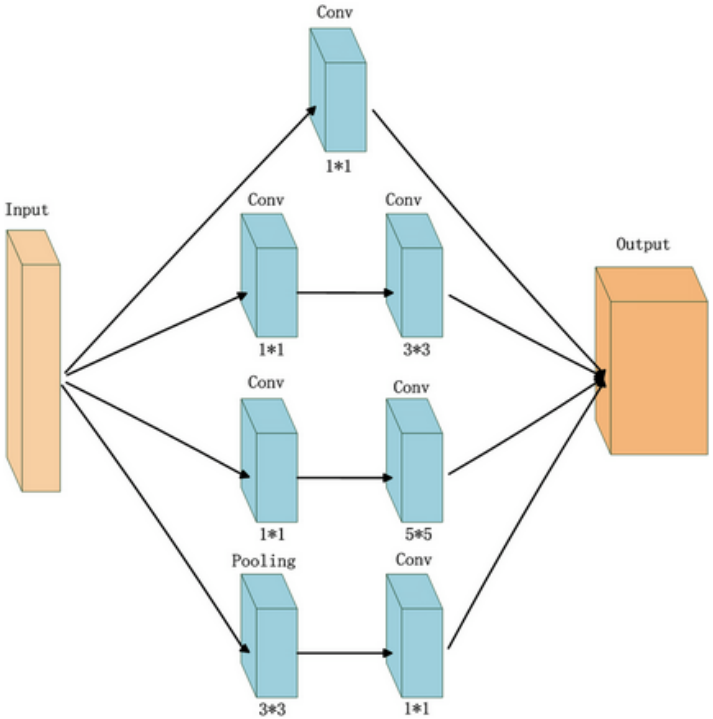


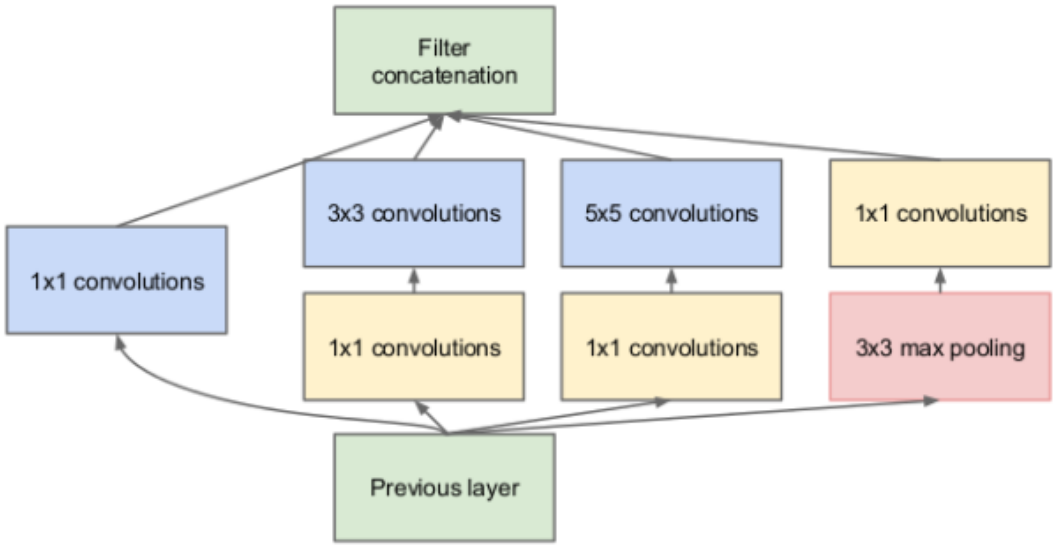
  - Each conv layer : linear + non-linearity
  - NiN block : hierarchy of conv layer
    - 1×1 conv $\sim$ MLP $\Rightarrow$ universal approximator, more expressive power for each patch

GoogLeNet : 1×1 Convolution



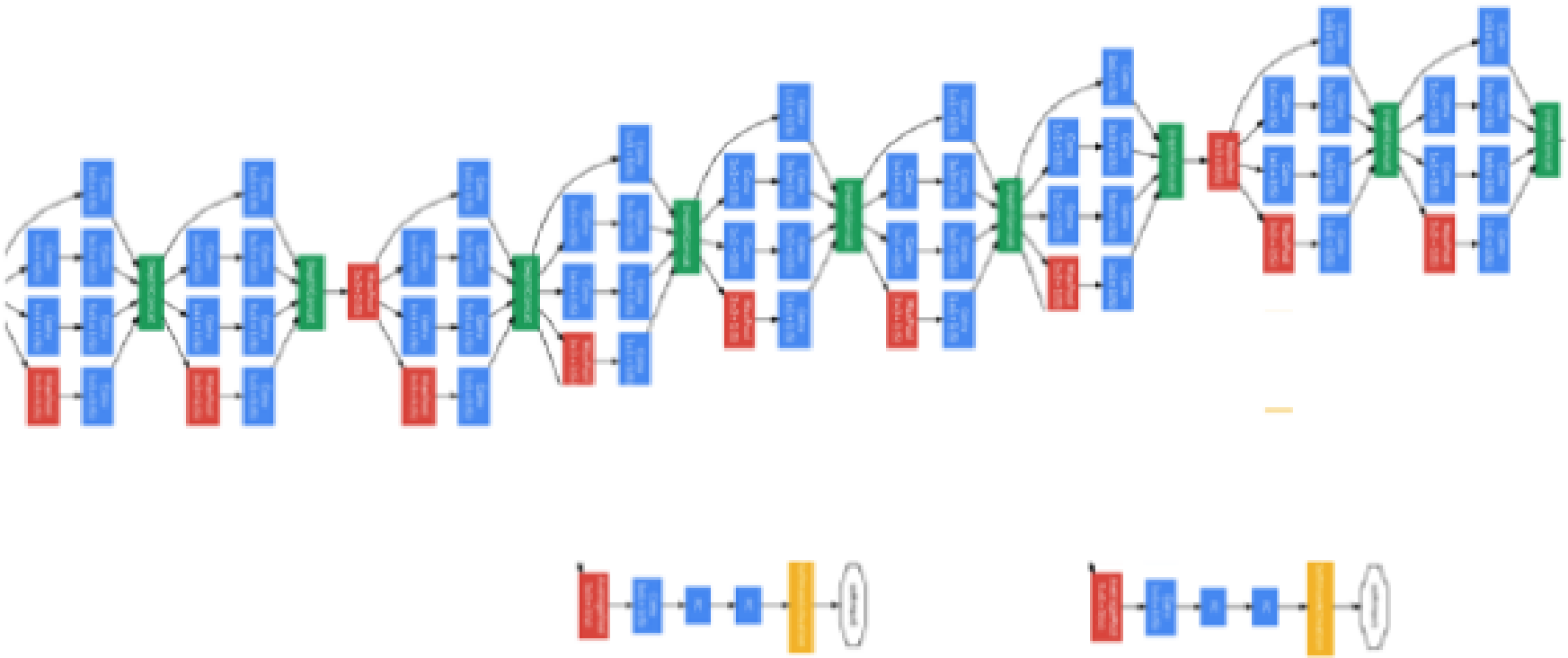
GoogLeNet : Inception Module





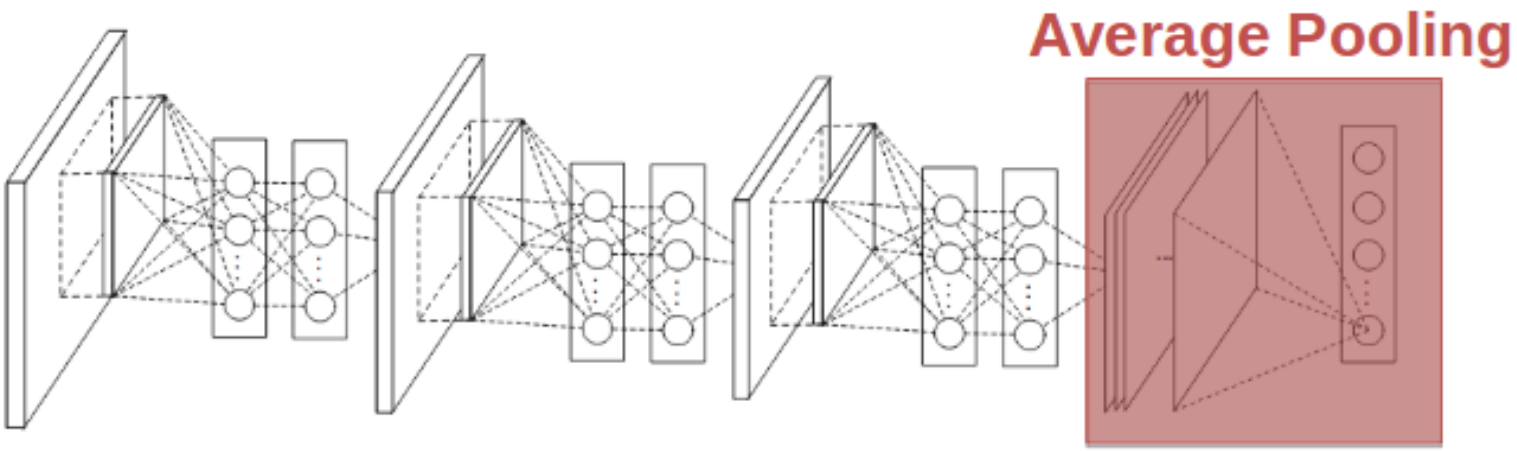
- 1×1, 3×3, 5×5 $\Rightarrow$ multi-scale
- Optimal filter size, pooling or not : learned
- 1×1 : dimensionality reduction
  $\Rightarrow$ reasonable # parameters

ImageNet'14 : GoogLeNet



- **Two auxiliary classifiers**
  - Fights vanishing gradient
  - Regularization during training

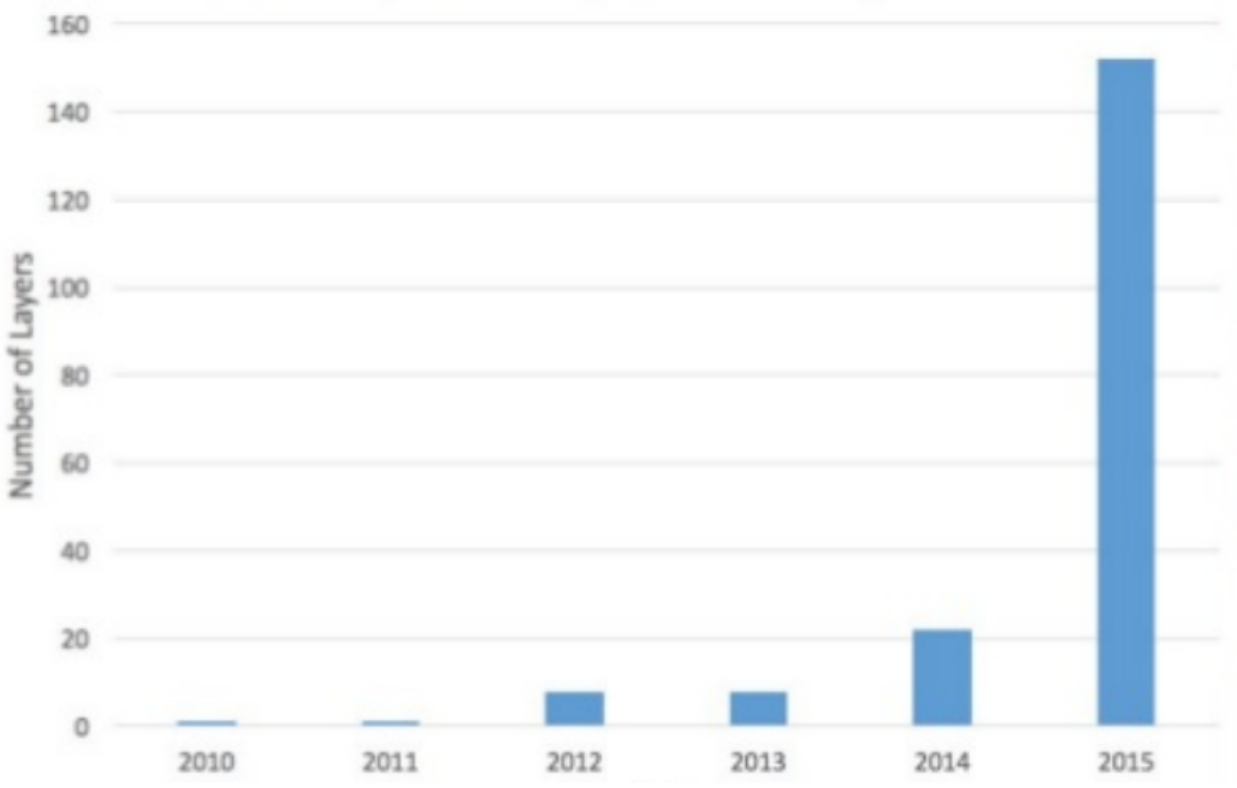
ImageNet'14 : GoogLeNet



- **Output classifier : GLobal Average Pooling (GAP) $\Rightarrow$ no FC !**
  - Drastic $\downarrow$ # params !

Deep Learning since 2012

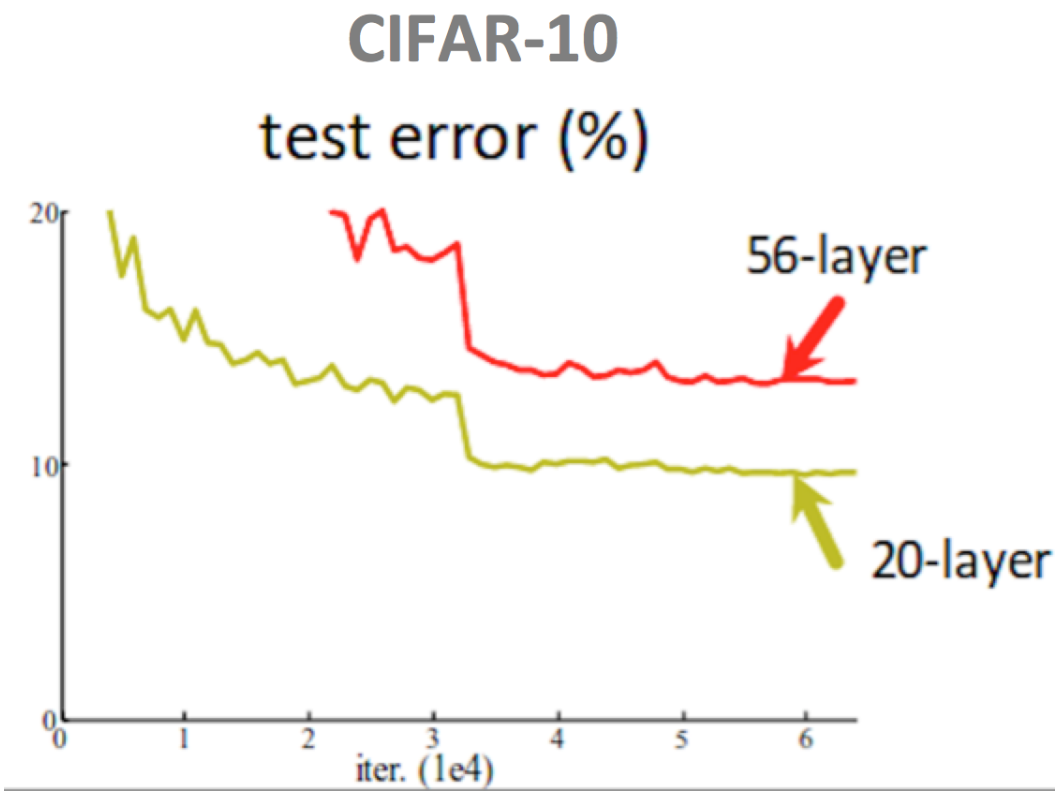
ILSVRC since 2012 : larger & larger networks





- **ImageNet'14 : VGG & GoogLeNet** ~ 20 **layers**

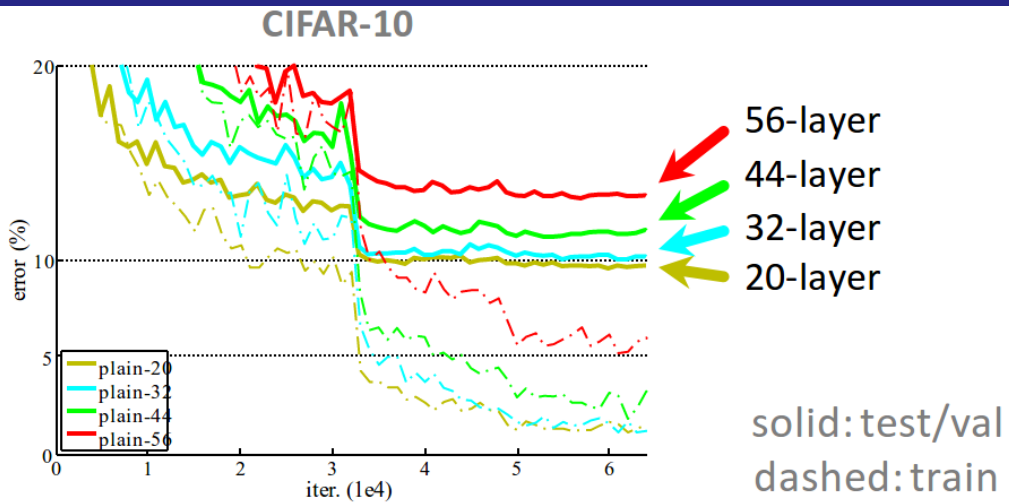
Issue : Training Deeper Networks

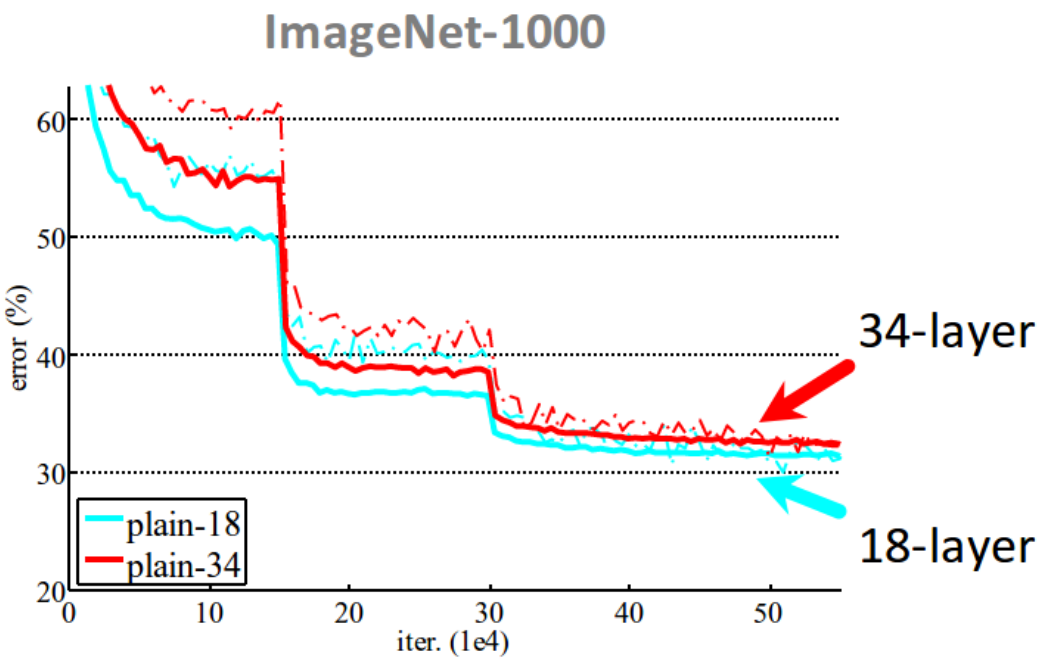


- **<u>BUT :</u> deeper nets have worse performances**
  - Ex : plain nets : stacking 3×3 conv
  - Ex : deeper VGG $\Rightarrow$ VGG56 ¡ VGG20

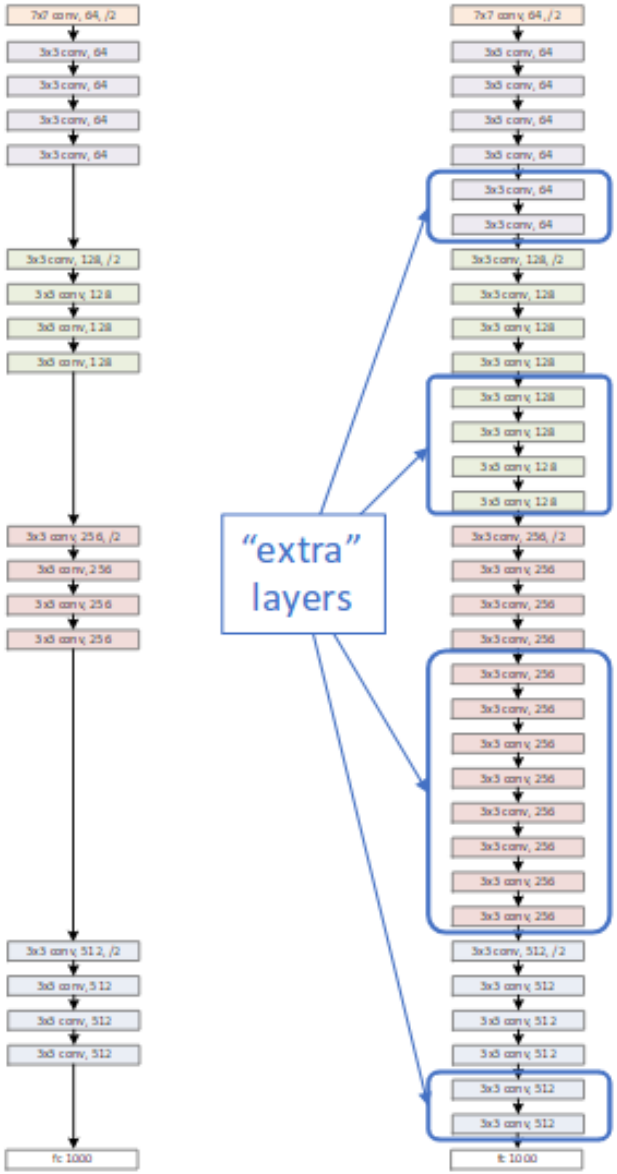
Issue : Training Deeper Networks

- ❦ ❦ **Not a generalization issue, training error also higher !**
- **General phenomenon, observed in many datasets**





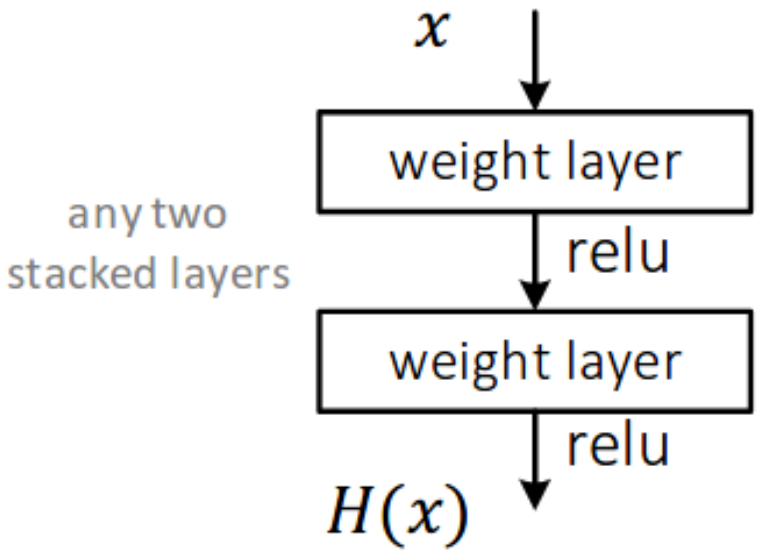
Issue : Training Deeper Networks



- 18 layers (left) *vs* 34 layers (right)
- **Deeper counterpart : richer solution space**
  $\Rightarrow$ should not have higher training error
- Construction : copy from a shallower model
  - Extra layers : set as identity

Identity Mapping with Plain Blocks

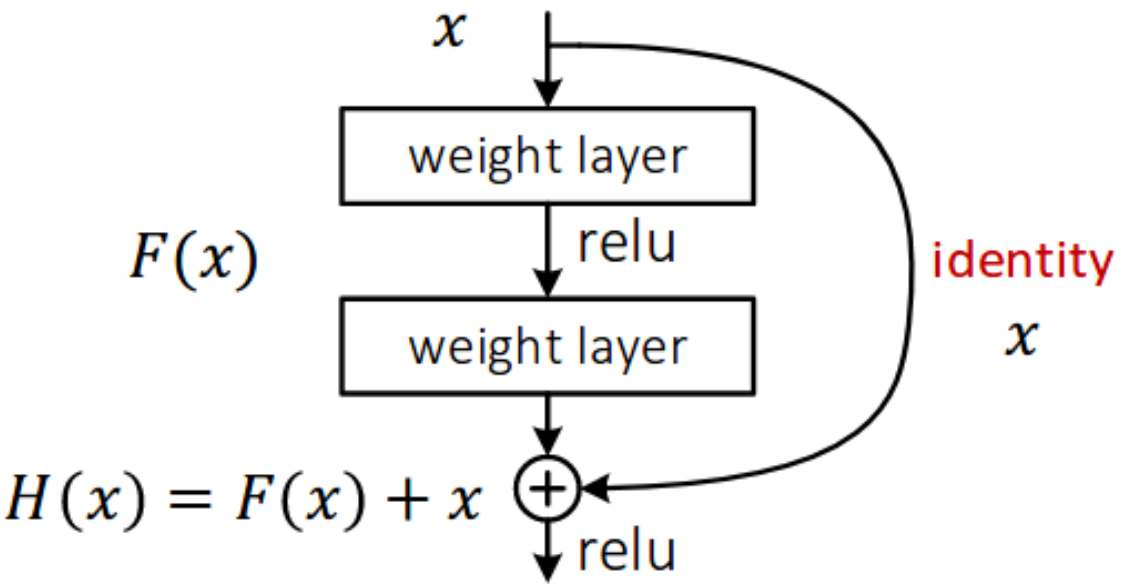
- **BUT : optimization challenge $\Rightarrow$ solvers cannot find this solution when going deeper...**
- $H(x)$ desired mapping to fit with a 2 weight layers
- **Multiple non-linear layers $\Rightarrow$ identity mapping difficult**



Plain Block

Identity Mapping with Residual Blocks [He et al., 2016]

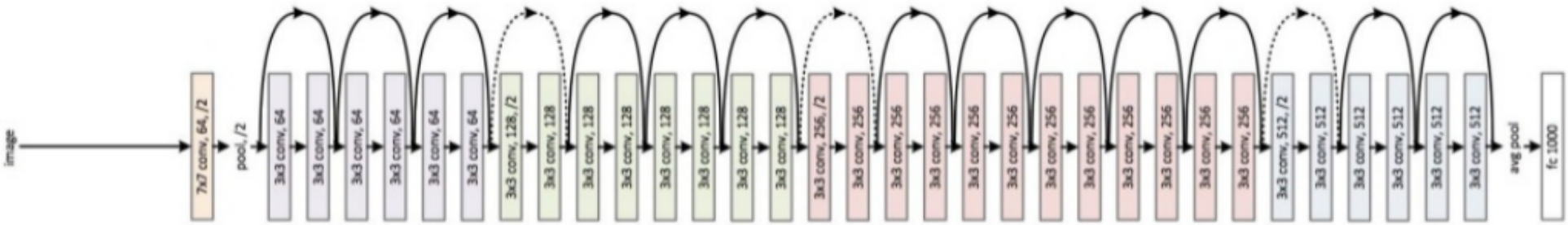
- $H(x)$ desired mapping, 2 layers fit $F(x) \Rightarrow H(x) = F(x) + x$
- **If identity optimal, easy to set weights as 0**
- **If optimal mapping closer to identity, easier to find small fluctuations**



Residual Block

ImageNet'15 : Residual Networks (ResNet) [He et al., 2016]

- Ex for VGG-style : 3x3 conv



ResNet-34

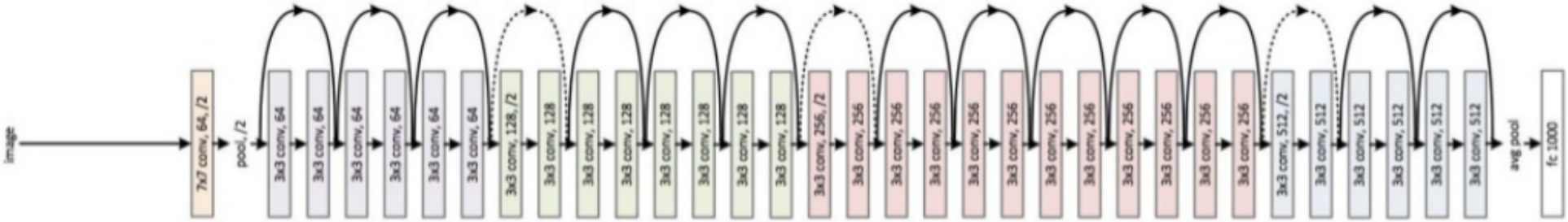
Architecture

- Stride 2 at some layers : spatial size /2, # filters x2
  $\Rightarrow$ same complexity per layer
- When size variations with residual connections : 1x1 conv (dotted)

Training

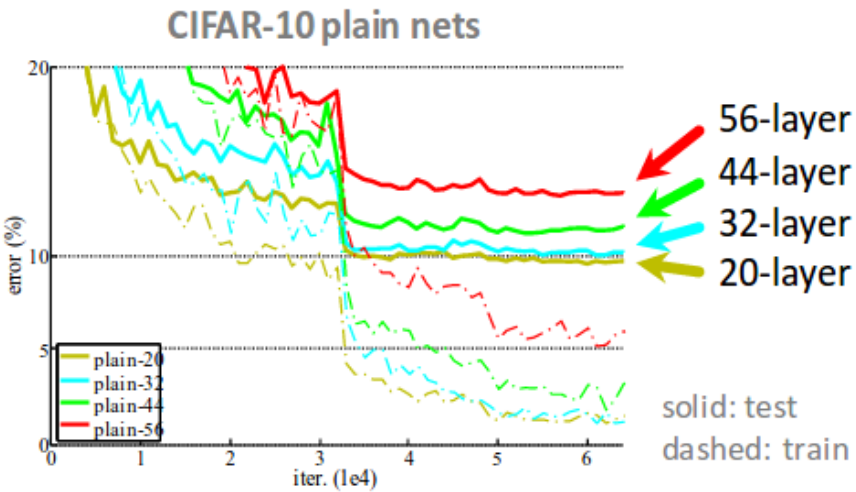
- Trained from scratch, standard hyper-parameters & augmentation
- Batch Normalization (after each conv & before activation), no dropout

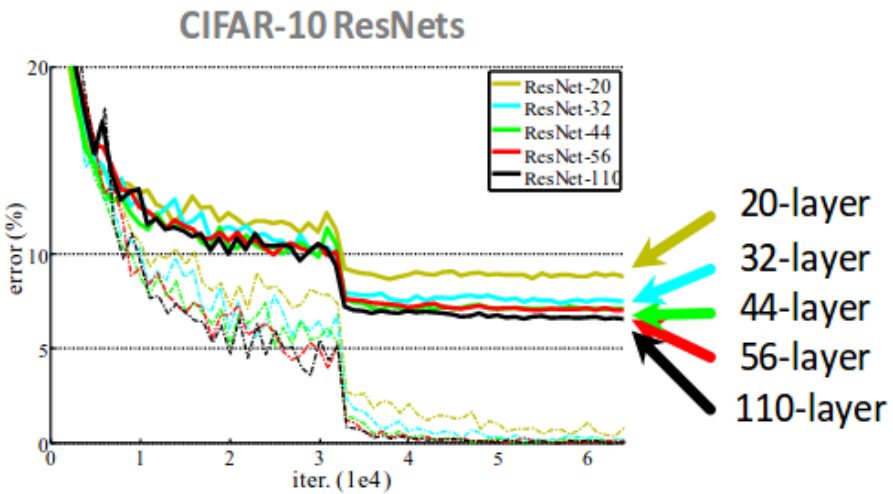
ImageNet'15 : Residual Networks (ResNet) [He et al., 2016]



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ResNet : Results

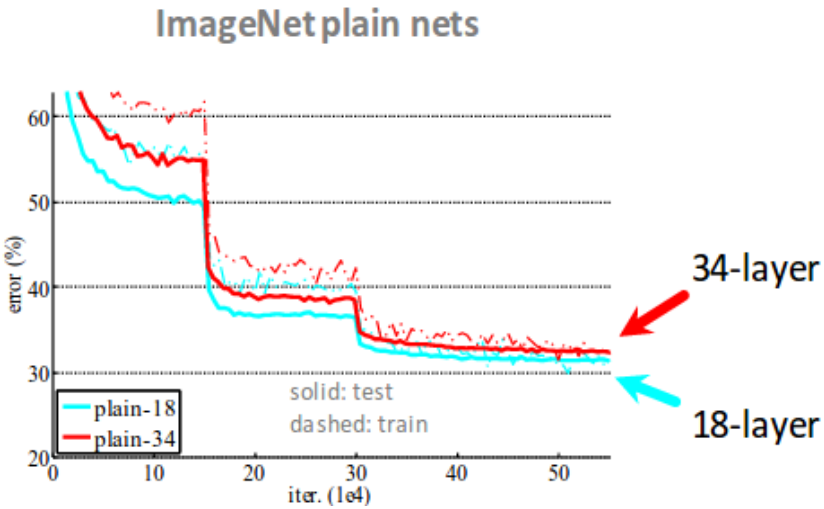


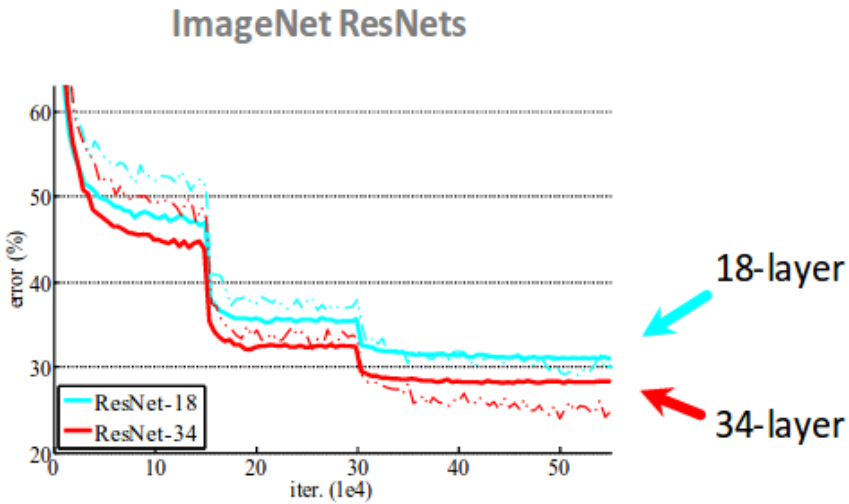


CIFAR-10 results

- **Deep ResNets trained without difficulty :**
  Deeper Models $\Rightarrow$ training & testing error lower

ResNet : Results



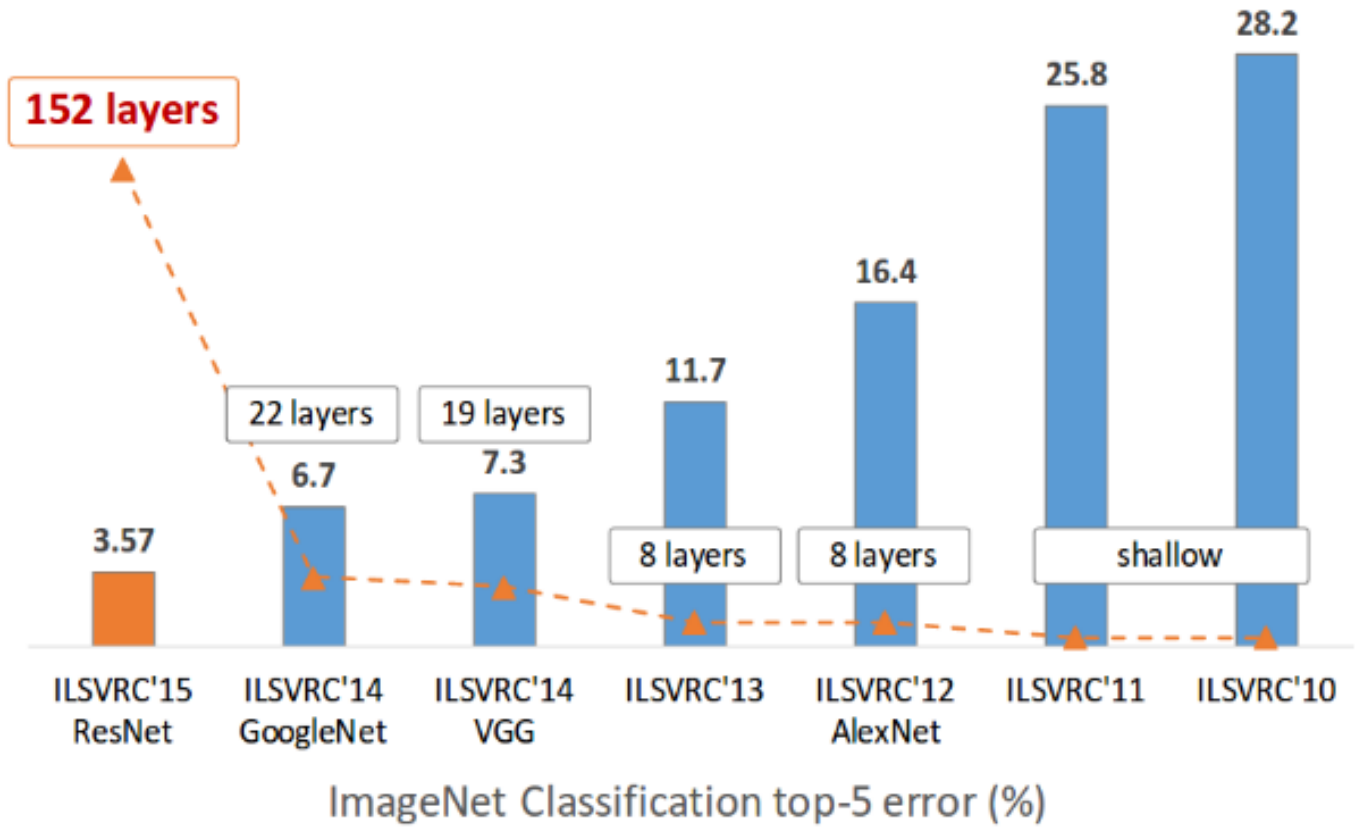


ImageNet

- **Deep ResNets trained without difficulty :**
  Deeper Models $\Rightarrow$ training & testing error lower

ResNet : Conclusion

- ResNet : training deeper models
- **Performance below 4% error at ImageNet Large Scale Visual Recognition Competition'15 !**

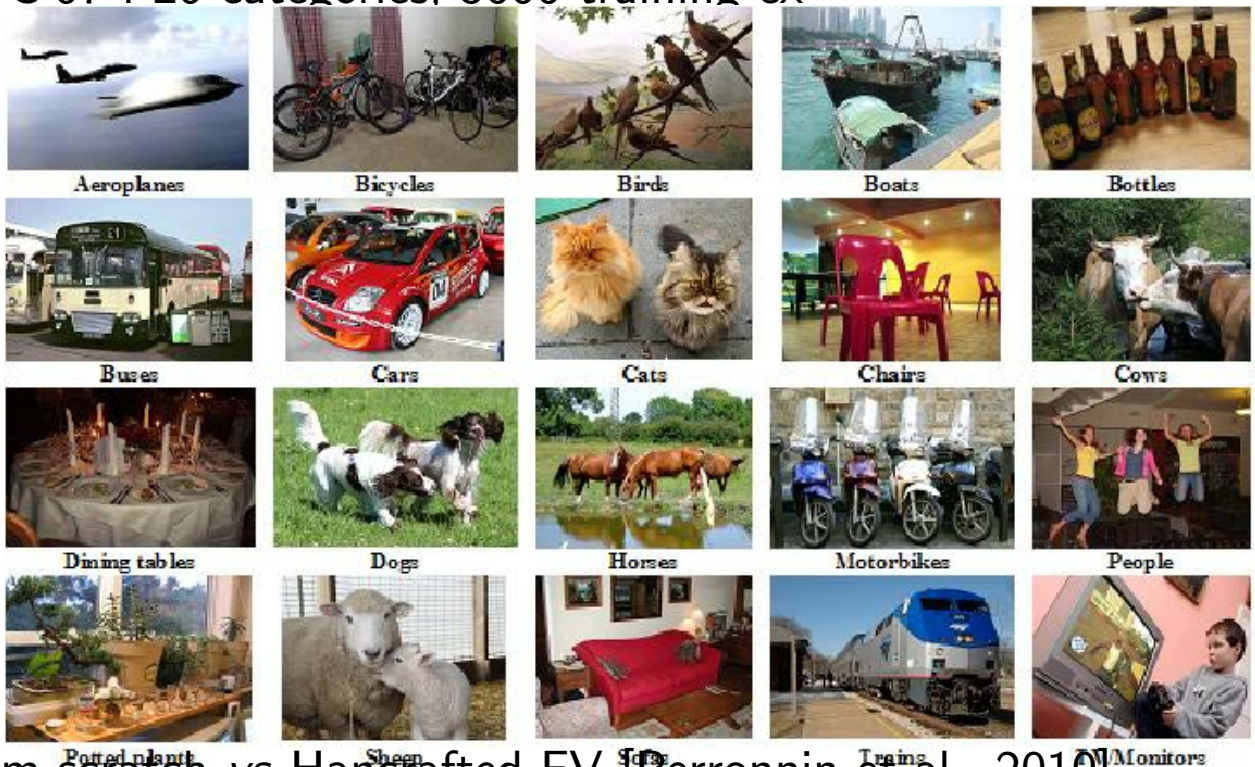


Outline

1. Convolutional Neural Networks
2. Modern Deep Architectures
3. Implementation
4. Transfer Learning

Training Deep ConvNets on Small Datasets

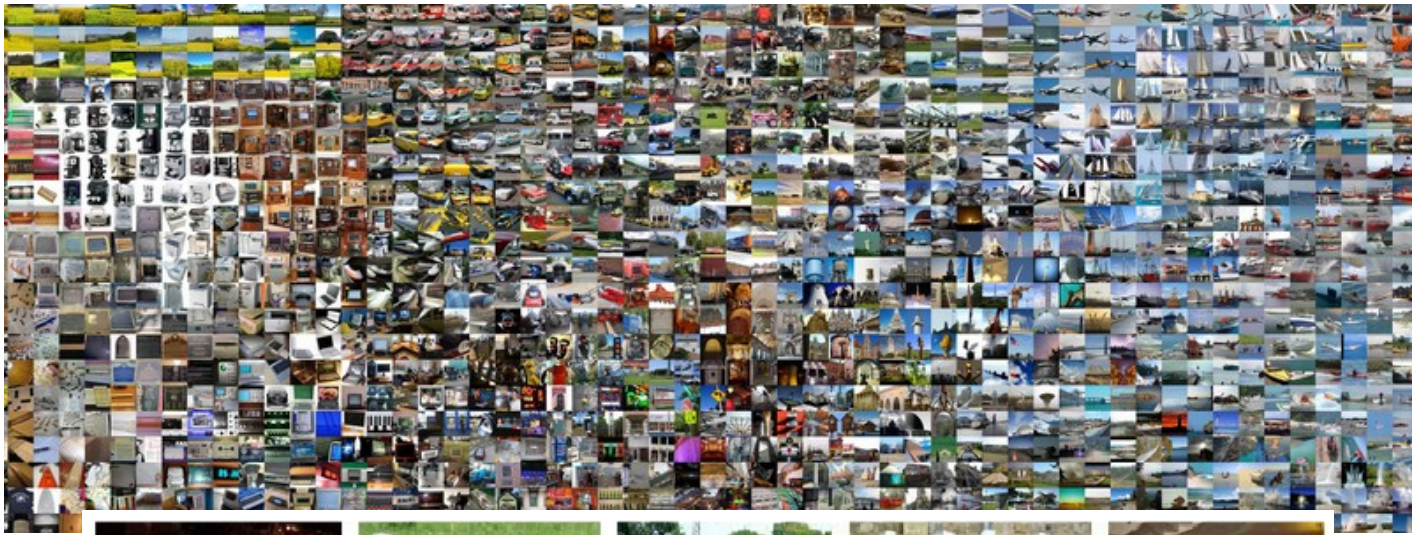
- Ex : PASCAL VOC'07 : 20 categories, 5000 training ex



- Training VGG from scratch *vs* Handcrafted FV [Perronnin et al., 2010]
- Deep ¡¡ Handcrafted

Model	Test mAP (%)
VGG	≈ 40
FV	≈ 70

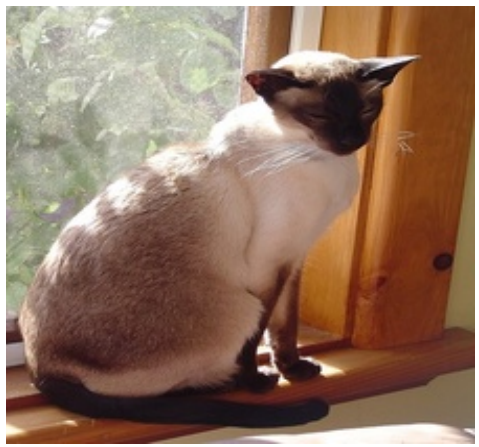
Training Deep ConvNets on Small Datasets



- ImageNet : deep ¿¿ hancrafted
- VOC'07 :deep ¡¡ hancrafted
  - Not enough training ex
  - Complex images

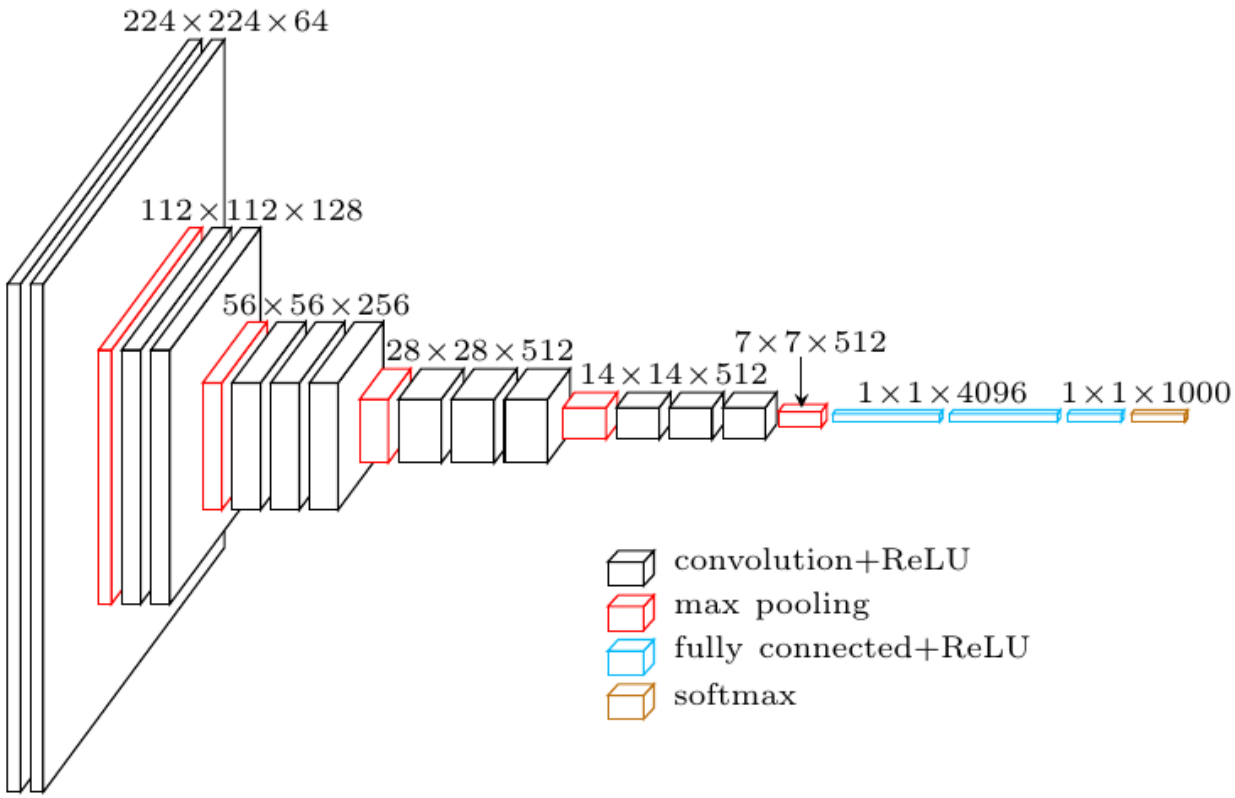
Transfer Learning

- **Idea : export knowledge from source domain to target domain**
  - Source : good performances, *e.g.* many samples
  - Target : more challenging, *e.g.* few samples
    $\Rightarrow$ Deep ConvNet good in imagenet, but not as good in VOC'07
- **Assumption : source and target classes different but related**
  - Ex : Various breeds of cat (tabby, persian) *vs* cat

ImageNet		VOC'07
Tabby Cat	Persian Cat	Cat
		

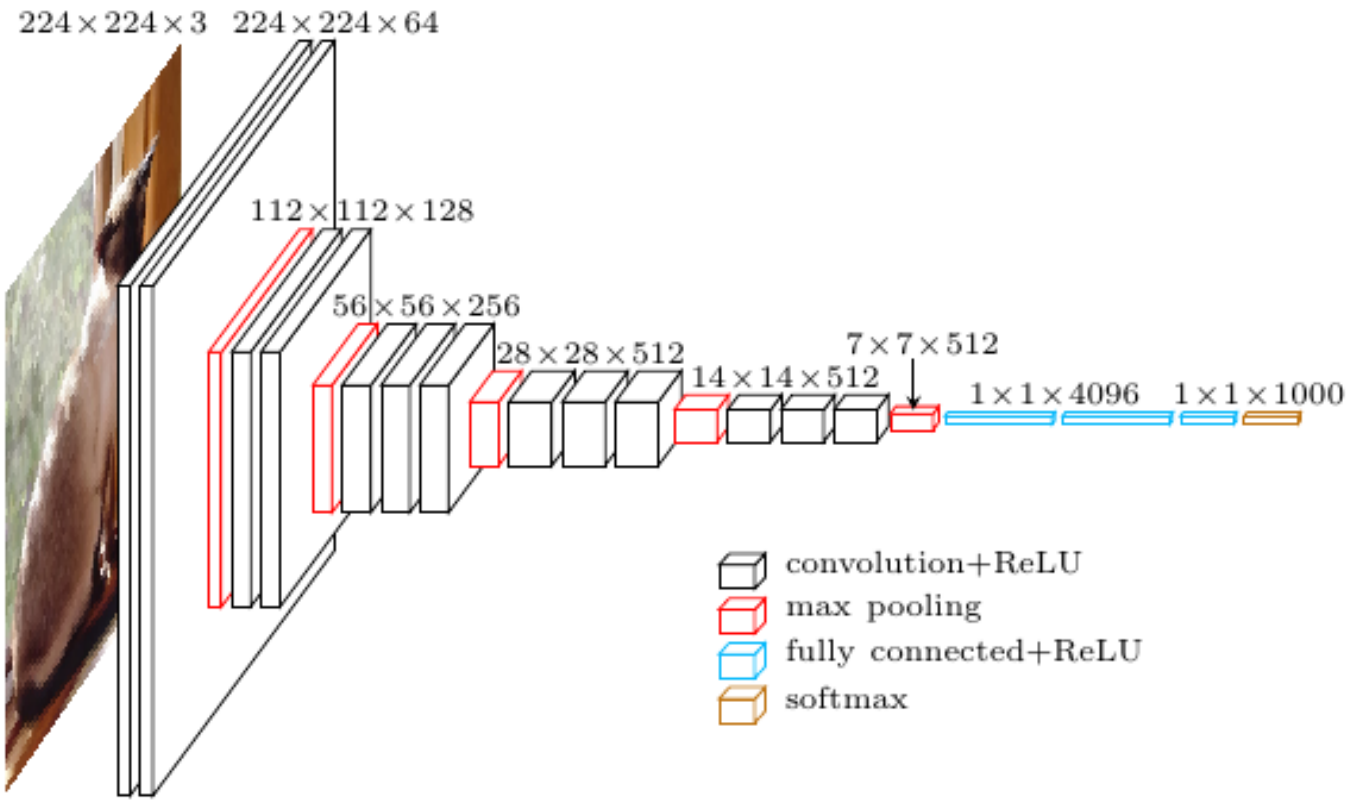
Transferring Representations learned from ImageNet

- Most naive transfer learning approach :
  - Load ConvNet model pre-trained on ImageNet, *e.g.* VGG

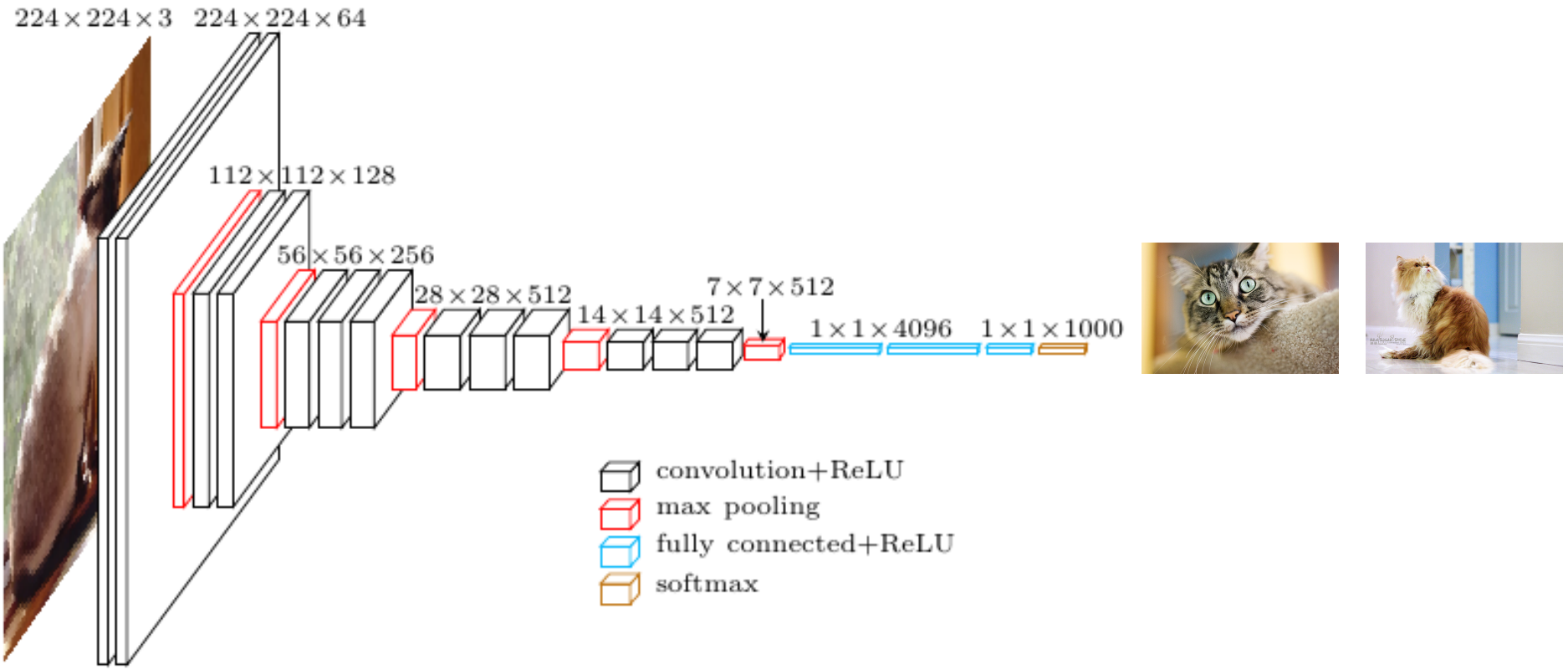


Transferring Representations learned from ImageNet

- Most naive transfer learning approach :
  - Load ConvNet model pre-trained on ImageNet
  - Apply ConvNet on each target dataset image, *e.g.* VOC
  - Extract a given layer activation : **"Deep Features" (DF)**

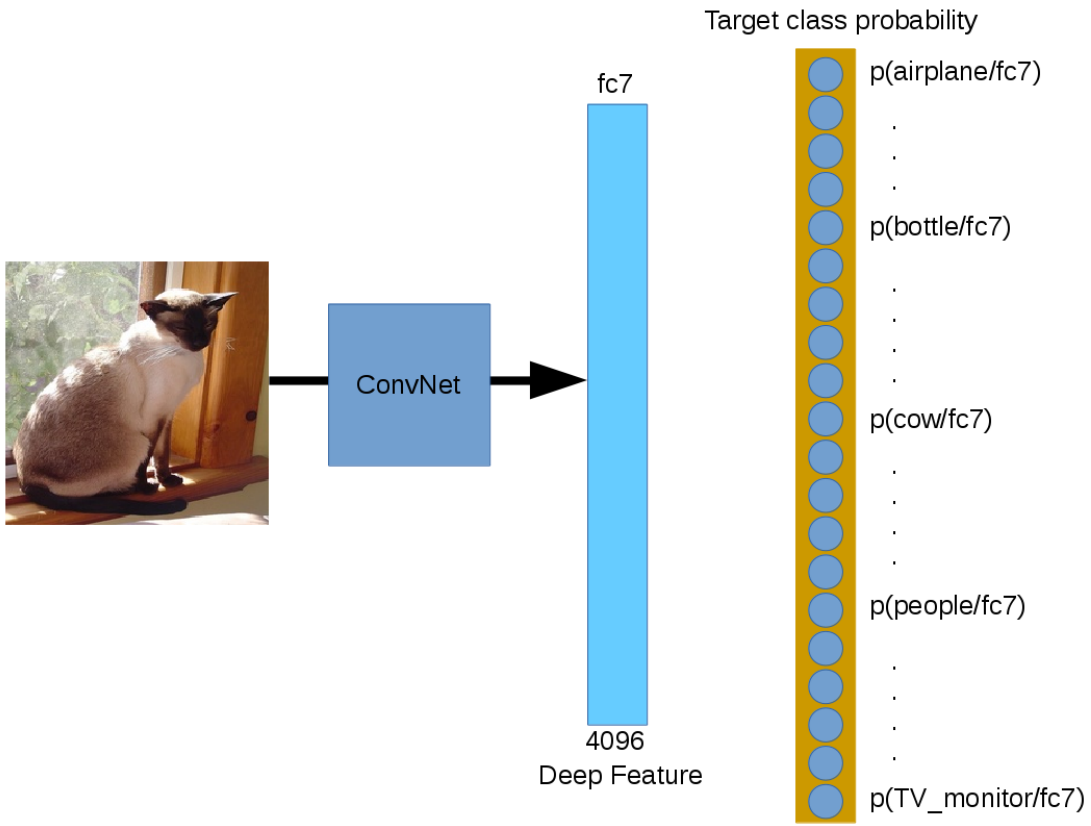


Deep Features (DF) for Classification



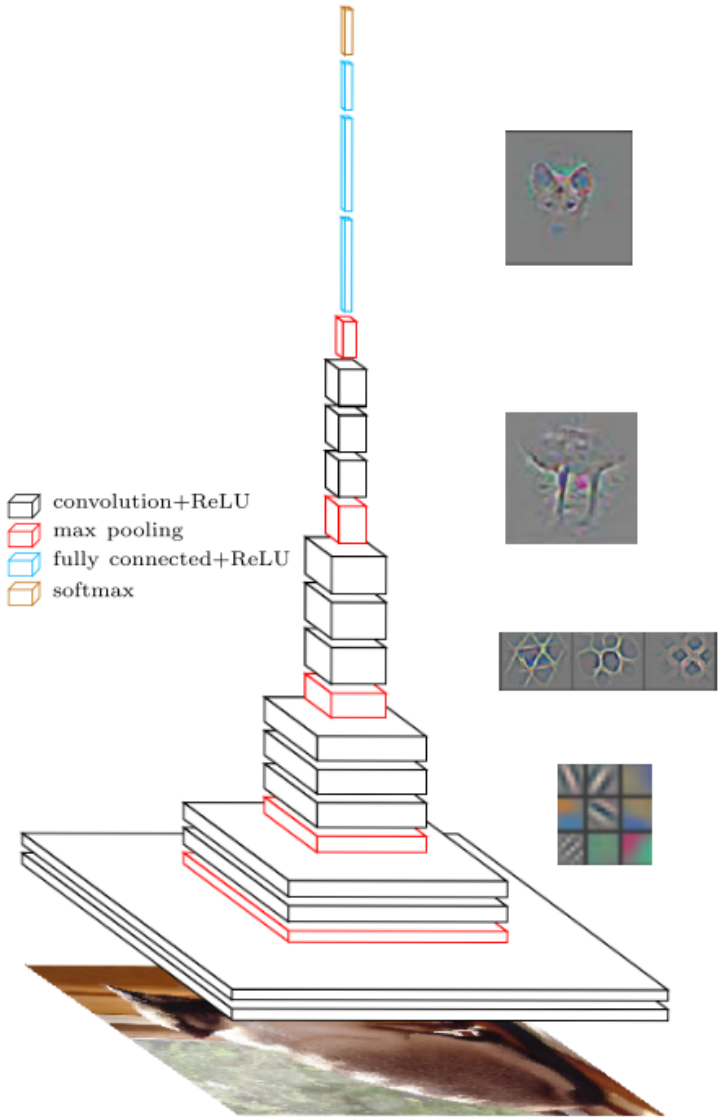
- **Deep Feature (DF), e.g. fc7 : use it as any visual descriptor**
- Fc7 : relevant for discriminating target classes, *e.g.* tabby/persian cat for cat

Deep Features (DF) for Classification



- **Deep Feature (DF), e.g. fc7 : visual descriptor**
  - DF : highly non-linear feature
  - Can use flat ML model for target class prediction

Deep Features (DF) for Classification



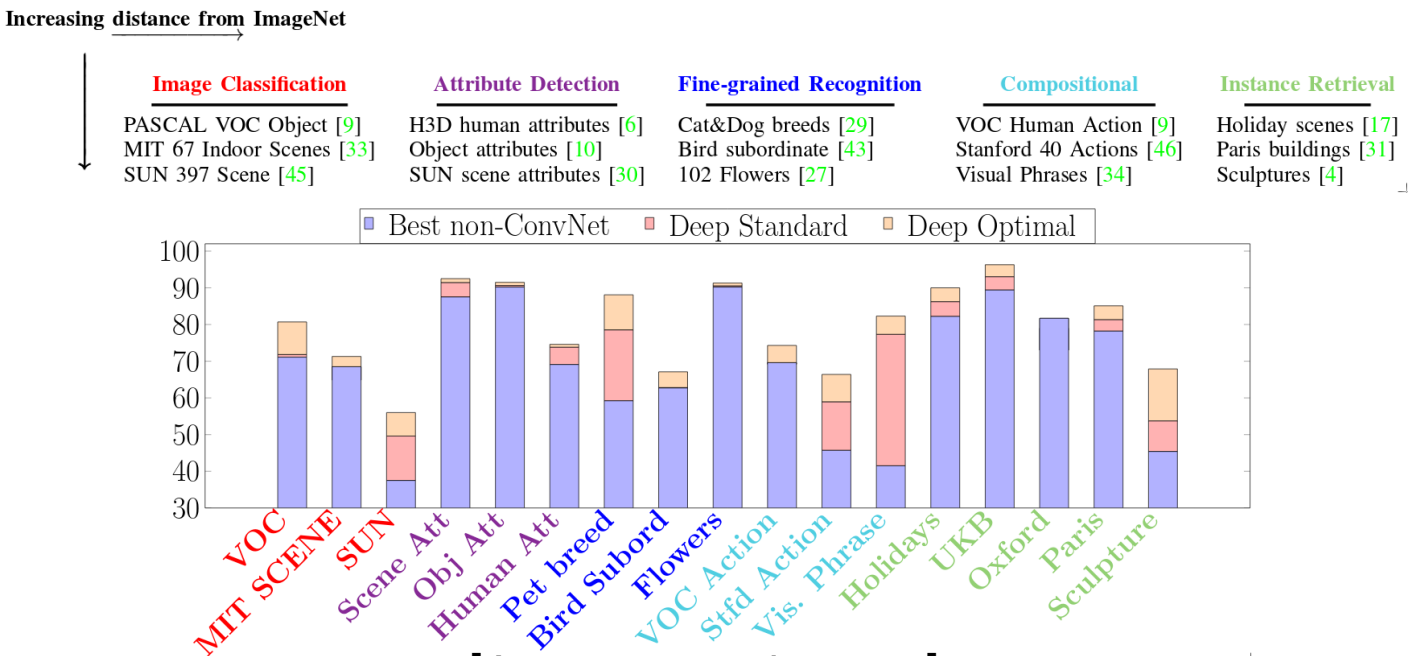
Which layer to use for classification ?

- Layers close to classification : specific to ImageNet
- Layers less close to classification : more generic features

$\Rightarrow$ **Depends on similarity between target & ImageNet**

Deep Features (DF) for Visual Recognition : Conclusion

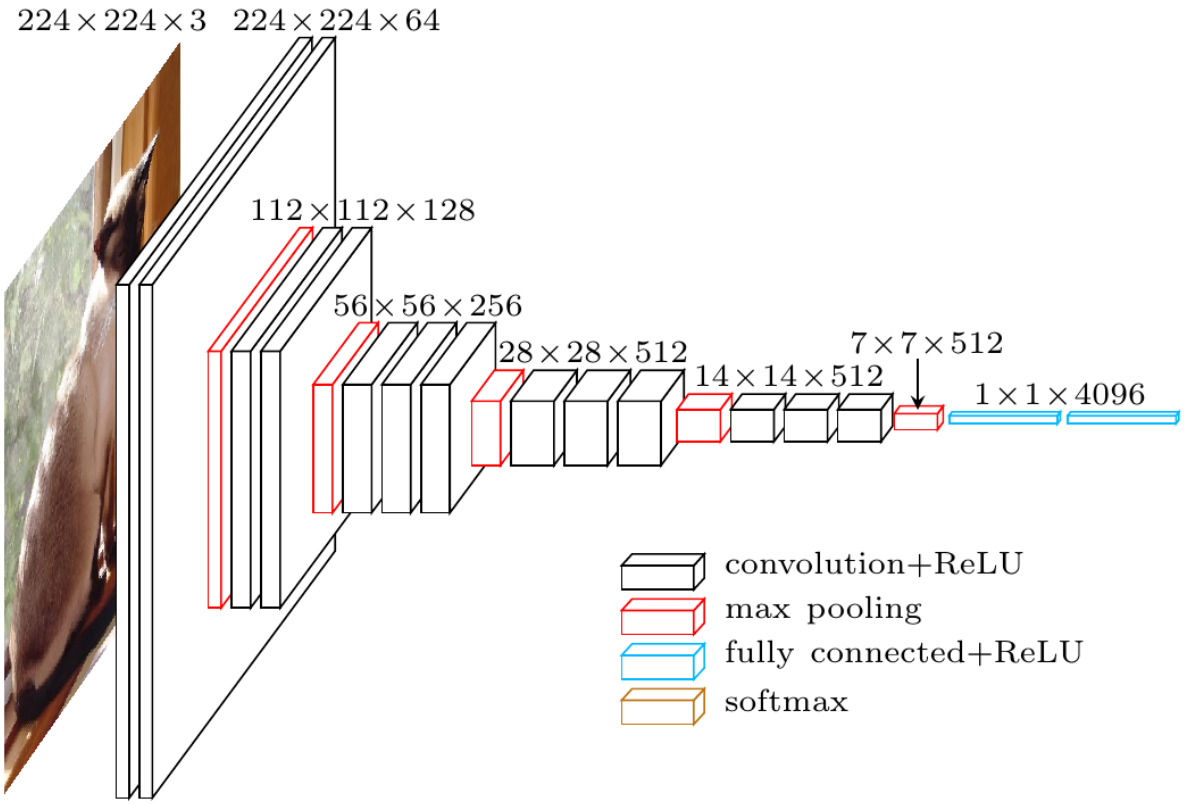
- **Deep Features : ConvNet success beyond ImageNet**
  $\Rightarrow$ No need huge dataset for using / training deep models
- **Off-the-shelf features for any visual recognition task**
- ImageNet : 1000 classes, large set of visual concepts
  - Transfer very well even to tasks with large domain shift, *e.g.* medical images
- **Off-the-shelf features for any visual recognition task**



[Azizpour et al., 2016]

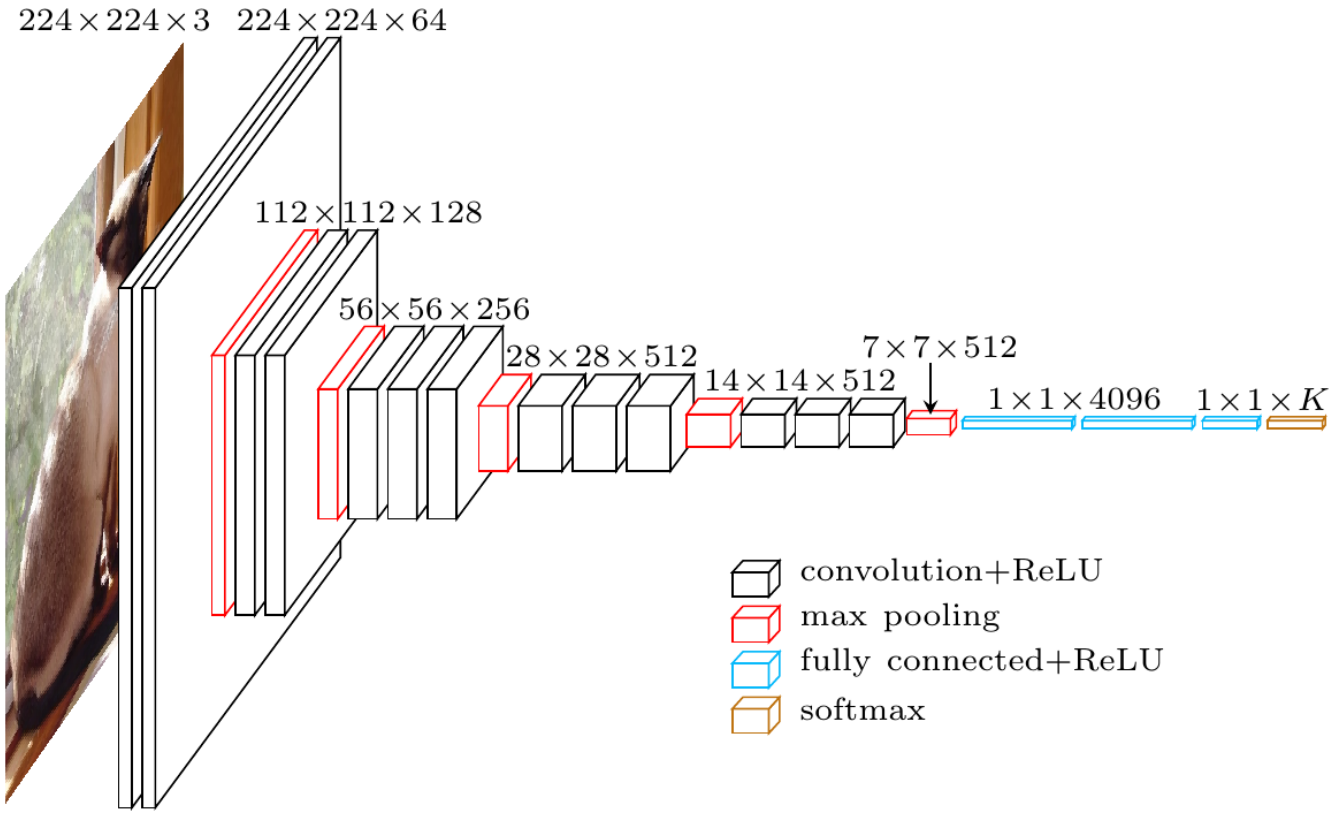
Deep Features (DF) and Transfer Learning for Classification

- **<u>Classification :</u> adapting final layer to match target classes**
  1. Remove last layer $\Leftrightarrow$ ImageNet classes

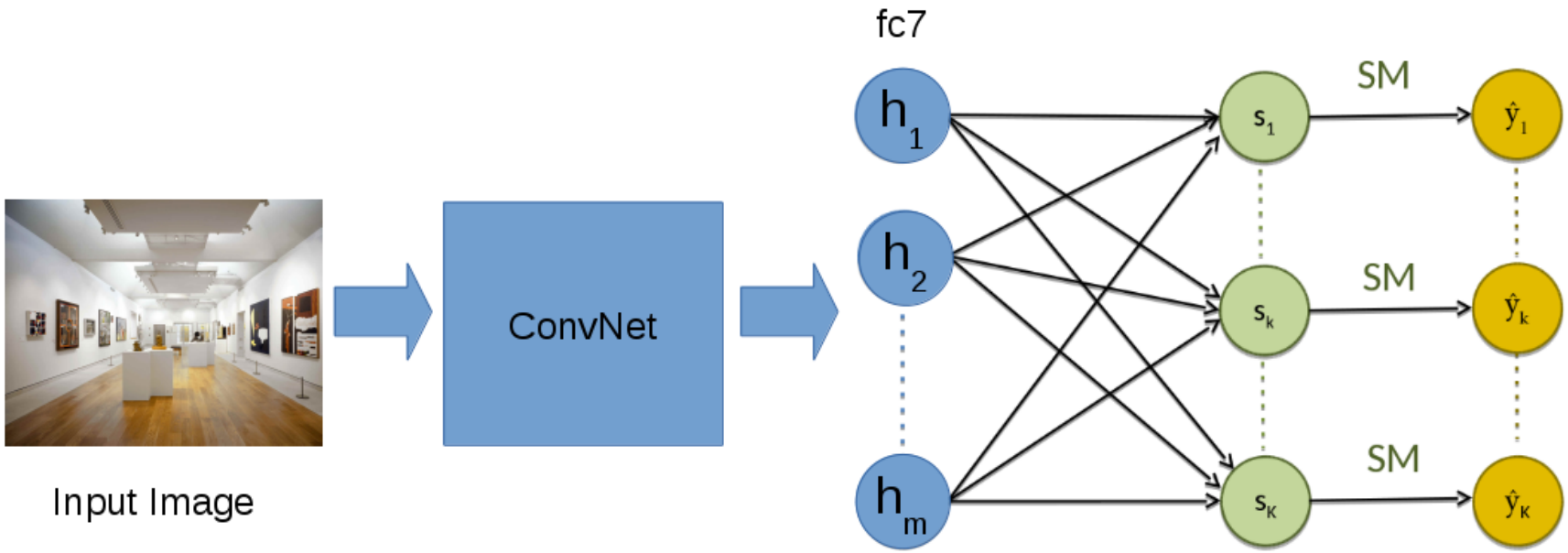


Deep Features (DF) and Transfer Learning for Classification

- **Classification : adapting final layer to match target classes**
  2. Add transfer layer with K target classes, *e.g.* $K = 20$ VOC'07

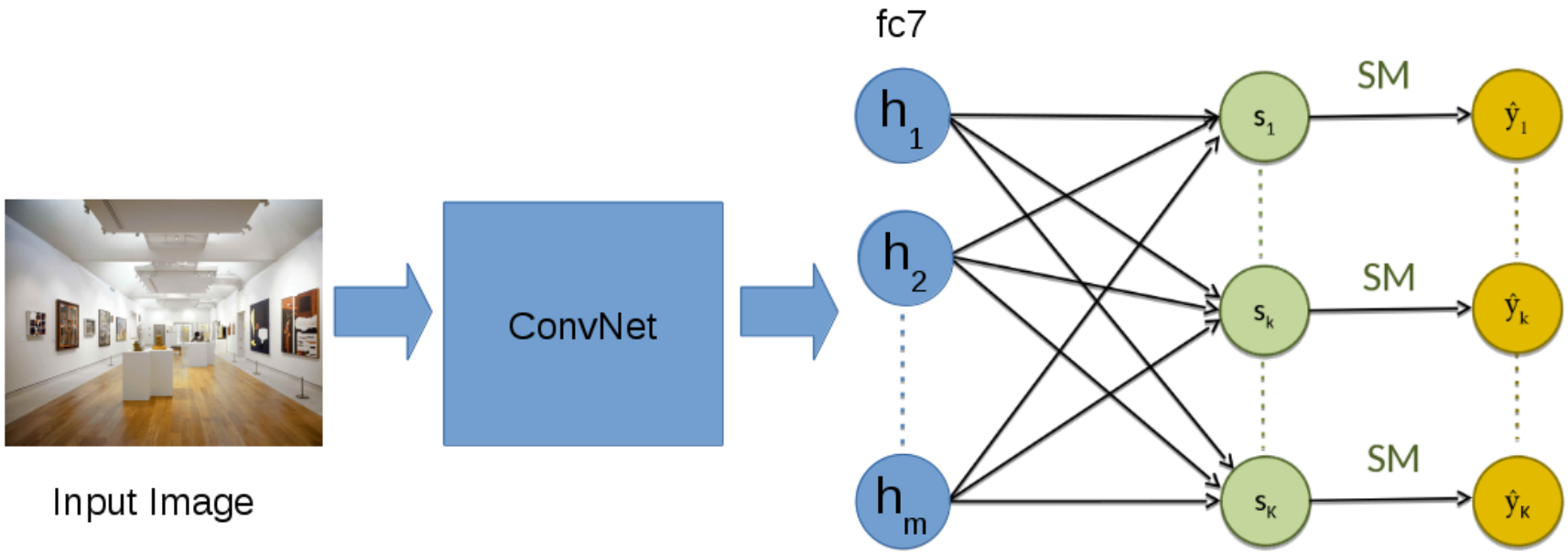


Transfer Learning for multi-class Classification



- Multi-class Classification : only one class per image, *i.e.* exclusive labels (*e.g.* ImageNet, MNIST)
- Ex for transfer in MIT-67 (museum)

Transfer Learning for multi-class Classification



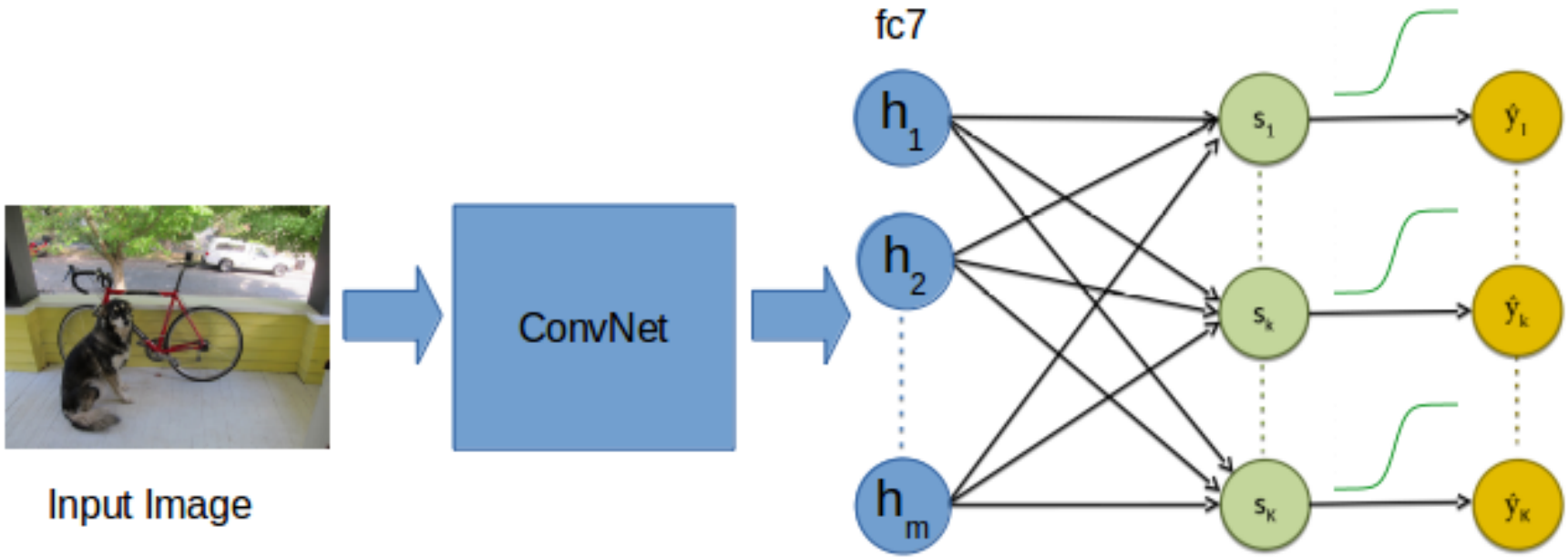
- Transfer layer of K classes + soft-max activation
- $\mathbf{s_i} = \mathbf{x_i W} + \mathbf{b}$, $\hat{\mathbf{y}}_\mathbf{k} \sim P(k/\mathbf{x_i}, \mathbf{W}, \mathbf{b}) = \frac{e^{\mathbf{s_k}}}{\sum_{k'=1}^{K} e^{\mathbf{s_{k'}}}}$
- Class parameters $\mathbf{w}_k$ dependent

Multi-label Classification

- Several labels for a given image, *e.g.* dog + bicycle + car (PASCAL VOC)
- Classification : train K binary models predicting class presence / absence



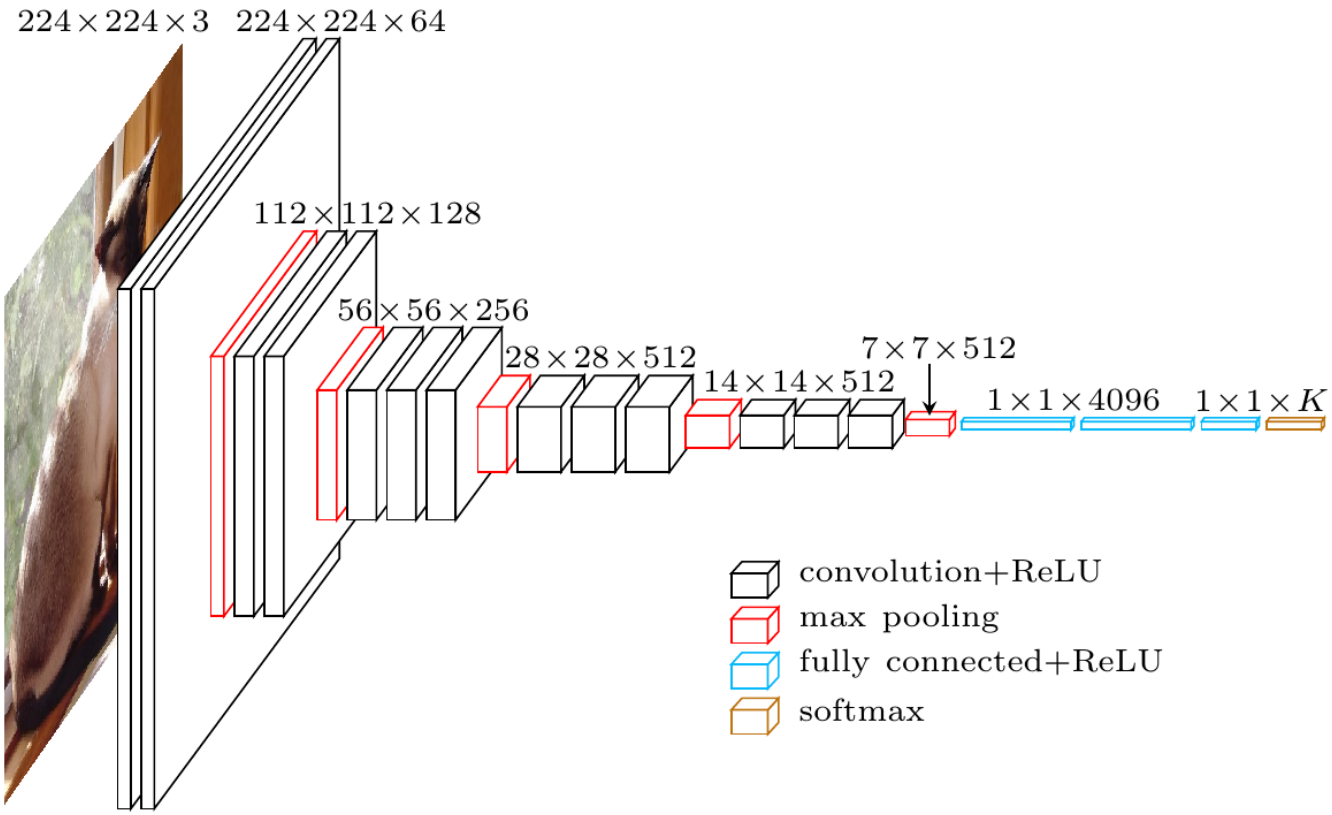
Transfer Learning for multi-label Classification



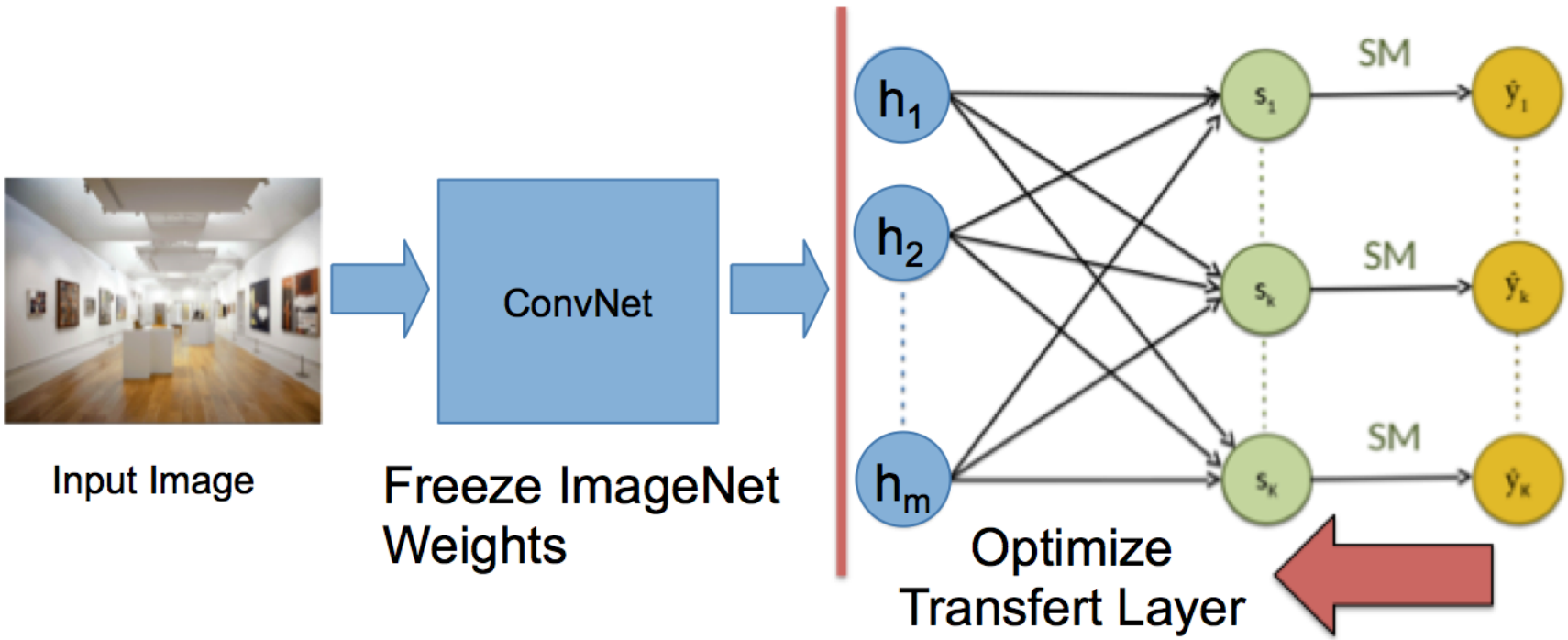
- Transfer layer of K classes + sigmoid activation
- $\mathbf{s}_i = \mathbf{x}_i\mathbf{W} + \mathbf{b}$, $\hat{\mathbf{y}_k} = \left[1 + e^{-\lambda \mathbf{s}_k}\right]^{-1}$
- Class parameters $\mathbf{w}_k$ independent

Deep Features (DF) and Transfer Learning for Classification

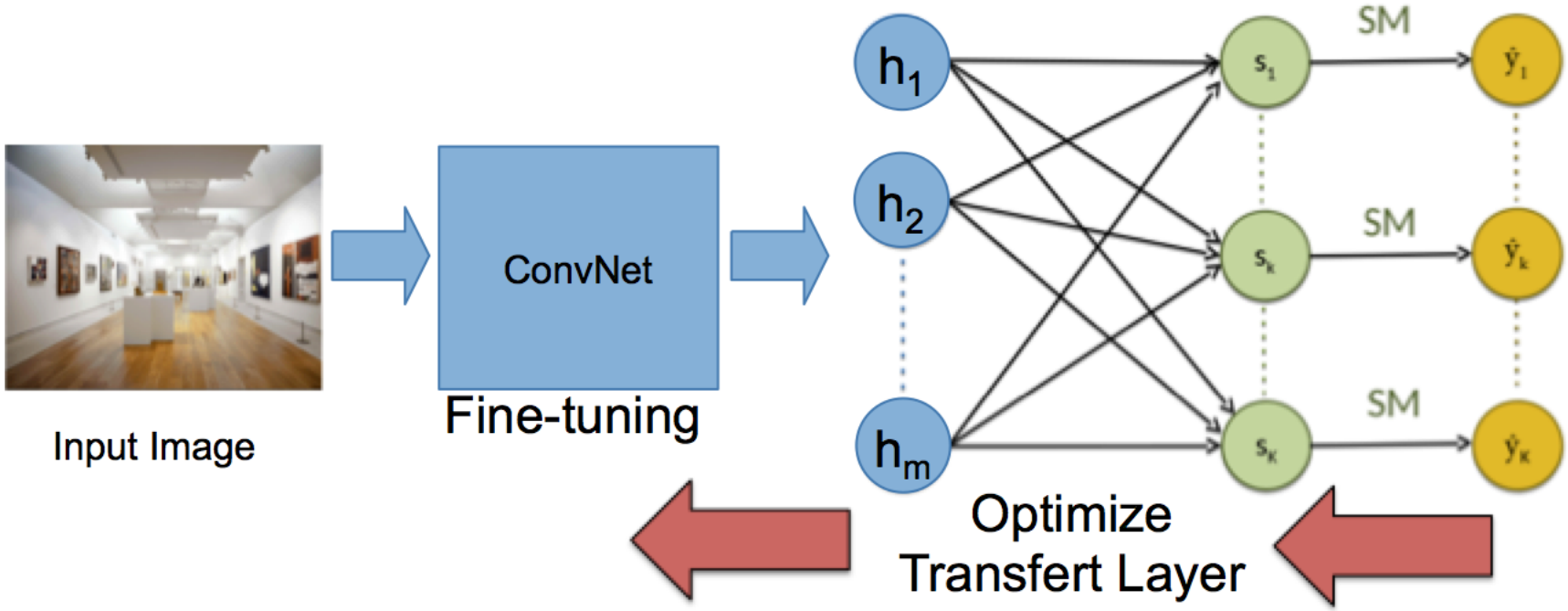
- **Training strategy ?**, depending on :
  a) Volume of the target dataset
  b) Semantic proximity wrt ImageNet



Training Strategy : Pure Transfer



Training Strategy : Fine-Tuning



- Decreased learning rate for fined-tuned wrt from scratch parameters *e.g.* factor 10 $\downarrow$

Training Strategy : Results

- **Small datasets $[10^3, 10^4]$, semantically close to ImageNet**
- Ex : VOC'07 (20 classes, 5000 ex)

Model	mAP (%)
VGG from scratch	≈ 40
Handcrafted FV	≈ 70
VGG pure tranfer	≈ 83
VGG fine-tuning	≈ 85

  - Fine Tuning $>$ Transfer $>>$ Handcrafted (BoW)
  - From scratch low : not enough training data



Training Strategy : Results

- **Medium/large datasets $\geq 10^5$, semantically close to ImageNet**
- Ex : UPMC-food-101 (100 classes, 100000 ex)



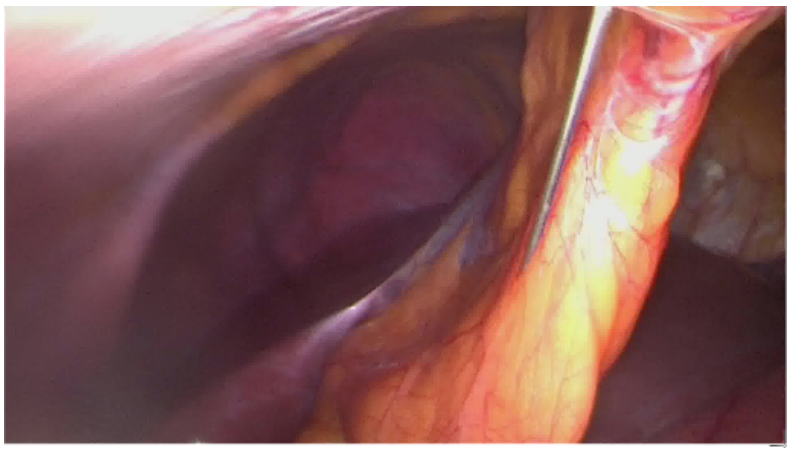
Model	mAP (%)
Handcrafted	≈ 25
VGG pure tranfer	≈ 40
VGG from scratch	≈ 53
VGG fine-tuning	≈ 65

- **From scratch does work (well !)**
- **Fine Tuning >> From scratch >> Transfer >> Handcrafted**

Training Strategy : Results

- **Medium/large datasets $\geq 10^5$, semantically far from ImageNet**
- Ex : M2CAI'16 challenge : 22 videos, $\sim 10^5$ images, 8 classes

Model	Accuracy (%)
VGG pure tranfer	≈ 60
VGG from scratch	≈ 70
VGG fine-tuning	≈ 80



- **Fine Tuning $>>$ From scratch $>>$ Transfer**
- **Transfer already good baseline despite big visual content shift**

Transfer Learning : Conclusion

	Small visual shift	**Large visual shift**
Small dataset	Transfer top layers	Transfer lower layers
Larger dataset	Fine-tune top layers	Fine-tune all layers

- Small dataset, large visual shift : common in medical image classification

- **How to implement transfer for localized tasks ?**
  $\Rightarrow$ **last course !**





References I



Azizpour, H., Razavian, A. S., Sullivan, J., Maki, A., and Carlsson, S. (2016).

Factors of transferability for a generic convnet representation.
IEEE Trans. Pattern Anal. Mach. Intell., 38(9) :1790–1802.



He, K., Zhang, X., Ren, S., and Sun, J. (2016).

Deep residual learning for image recognition.
In CVPR.



Ioffe, S. and Szegedy, C. (2015).

Batch normalization : Accelerating deep network training by reducing internal covariate shift.
In Bach, F. R. and Blei, D. M., editors, ICML, volume 37 of JMLR Workshop and Conference Proceedings, pages 448–456. JMLR.org.



Kanazawa, A., Sharma, A., and Jacobs, D. W. (2014).

Locally scale-invariant convolutional neural networks.
In Deep Learning and Representation Learning Workshop : NIPS 2014.



Lin, M., Chen, Q., and Yan, S. (2013).

Network in network.
CoRR, abs/1312.4400.



Marcos, D., Volpi, M., Komodakis, N., and Tuia, D. (2017).

Rotation equivariant vector field networks.
In The IEEE International Conference on Computer Vision (ICCV).



Perronnin, F., Sánchez, J., and Mensink, T. (2010).

Improving the fisher kernel for large-scale image classification.
In Computer Vision–ECCV 2010, pages 143–156. Springer.



Simonyan, K. and Zisserman, A. (2014).

Very deep convolutional networks for large-scale image recognition.
CoRR, abs/1409.1556.

References II



Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., and Rabinovich, A. (2015).

Going deeper with convolutions.

In 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 1–9. IEEE.



Taigman, Y., Yang, M., Ranzato, M., and Wolf, L. (2014).

Deepface : Closing the gap to human-level performance in face verification.

In The IEEE Conference on Computer Vision and Pattern Recognition (CVPR).



Zeiler, M. and Fergus, R. (2014).

Visualizing and understanding convolutional networks, volume 8689 LNCS of Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), pages 818–833.

Springer Verlag, part 1 edition.